流匹配指南与代码

Flow Matching Guide and Code

Yaron Lipman[1], **Marton Havasi**[1], **Peter Holderrieth**[2], **Neta Shaul**[3], **Matt Le**[1], **Brian Karrer**[1], **Ricky T. Q. Chen**[1], **David Lopez-Paz**[1], **Heli Ben-Hamu**[3], **Itai Gat**[1]

[1]Meta 的 FAIR, [2]MIT CSAIL, [3] 魏茨曼科学研究所

流匹配（FM）是一种新近提出的生成建模框架，已在图像、视频、音频、语音和生物结构等多个领域取得最先进的性能. 本指南全面且自包含地介绍 FM，涵盖其数学基础、设计选择和扩展. 我们还提供一个 PyTorch 程序库及相关示例（例如图像与文本生成），旨在为希望理解、应用和进一步发展 FM 的初学者及有经验的研究者提供参考.

日期: 原文：2024 年 12 月 10 日；arXiv:2412.06264v1

代码: `flow_matching` 程序库，https://github.com/facebookresearch/flow_matching

译本: Codex 翻译及重排；GPT-6 与 GPT-6.1-sol（medium）并行翻译；2026 年 10 月 22 日

Meta

目录

1 引言 **3**

2 快速入门与关键概念 **4**

3 流模型 **7**
3.1 随机向量 . . . 8
3.2 条件密度与条件期望 . . . 8
3.3 微分同胚与推前映射 . . . 9
3.4 作为生成模型的流 . . . 10
3.5 概率路径与连续性方程 . . . 12
3.6 瞬时变量替换公式 . . . 14
3.7 通过模拟训练流模型 . . . 16

4 流匹配 **16**
4.1 数据 . . . 16
4.2 构造概率路径 . . . 17
4.3 推导生成概率路径的速度场 . . . 17
4.4 一般条件化与边缘化技巧 . . . 18
4.5 流匹配损失 . . . 20
4.6 利用条件流求解条件生成问题 . . . 22
4.7 最优传输与线性条件流 . . . 25
4.8 仿射条件流 . . . 27
4.9 数据耦合 . . . 33
4.10 条件生成与引导 . . . 34

5 非欧几里得流匹配 **36**
5.1 黎曼流形 . . . 37
5.2 流形上的概率、流与速度 . . . 37
5.3 流形上的概率路径 . . . 38
5.4 流形上的边缘化技巧 . . . 38
5.5 黎曼流匹配损失 . . . 39
5.6 通过预度量构造条件流 . . . 39

6 连续时间马尔可夫链模型 **41**
6.1 离散状态空间与随机变量 . . . 41
6.2 CTMC 生成模型 . . . 42
6.3 概率路径与柯尔莫哥洛夫方程 . . . 43

7 离散流匹配 **44**
7.1 数据与耦合 . . . 44
7.2 离散概率路径 . . . 44
7.3 边缘化技巧 . . . 44
7.4 离散流匹配损失 . . . 45
7.5 因子化路径与速度 . . . 46

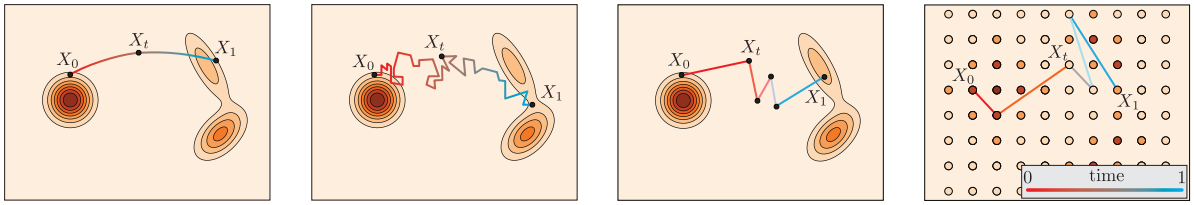
8 连续时间马尔可夫过程模型 **55**
8.1 一般状态空间与随机变量 . . . 55
8.2 CTMP 生成模型 . . . 56
8.3 概率路径与柯尔莫哥洛夫方程 . . . 61
8.4 普适表示定理 . . . 65

9 生成元匹配 **65**
9.1 数据与耦合 . . . 65
9.2 一般概率路径 . . . 65
9.3 通过神经网络参数化生成元 . . . 66
9.4 边缘生成元与条件生成元 . . . 67
9.5 生成元匹配损失 . . . 69
9.6 通过求解 KFE 寻找条件生成元 . . . 70
9.7 组合模型 . . . 72
9.8 多模态模型 . . . 74

10 与扩散模型及其他去噪模型的关系 **75**
10.1 时间约定 . . . 75
10.2 前向过程与概率路径 . . . 75
10.3 训练扩散模型 . . . 76
10.4 采样 . . . 77
10.5 时间反演与反向过程的作用 . . . 78
10.6 与其他去噪模型的关系 . . . 79

A 补充证明 **86**
A.1 离散质量守恒 . . . 86
A.2 流形边缘化技巧 . . . 87
A.3 KFE 的正则性假设 . . . 88

术语对照表 **89**



(a) 流 **(b)** 扩散 **(c)** 跳跃 **(d)** CTMC

图 1 将源样本 X_0 变为目标样本 X_1 的四种时间连续过程 $(X_t)_{0\le t\le 1}$. 它们分别为连续状态空间中的流、连续状态空间中的扩散、连续状态空间中的跳跃过程（用等高线表示密度），以及离散状态空间中的跳跃过程（用圆盘表示状态，以颜色表示概率）.

1 引言

流匹配（Flow Matching，FM）(Lipman et al., 2022; Albergo and Vanden-Eijnden, 2022; Liu et al., 2022) 是一个简单的生成建模框架，已推动多个领域及大规模应用达到最先进的水平，包括图像生成 (Esser et al., 2024)、视频生成 (Polyak et al., 2024)、语音生成 (Le et al., 2024)、音频生成 (Vyas et al., 2023)、蛋白质生成 (Huguet et al., 2024) 以及机器人技术 (Black et al., 2024). 本文及其配套代码库有两个主要目标. 第一，全面且自包含地介绍流匹配，详细说明其设计选择以及研究界开发的诸多扩展. 第二，帮助初学者迅速采用流匹配，并在其基础上构建自己的应用.

流匹配框架的基础是学习一个速度场（也称向量场）. 每个速度场通过求解常微分方程（ODE）定义一个**流** ψ_t，这一过程称为模拟. 流是 d 维欧氏空间 $\mathbb{R}^d$ 上确定性的、时间连续的双射变换. 流匹配的目标是构造一个流，将从源分布 p 抽取的样本 $X_0 \sim p$ 变换为目标样本 $X_1 := \psi_1(X_0)$，使其服从所需的分布 q，即 $X_1 \sim q$，见图 1a. (Chen et al., 2018; Grathwohl et al., 2018) 以连续归一化流（CNF）的形式将流模型引入机器学习领域. 最初，通过最大化训练样本 X_1 的似然 $p(X_1)$ 来训练流，因此在训练期间需要进行模拟并对模拟过程求导. 由于这带来了计算负担，后续工作尝试在不进行模拟的情况下学习 CNF(Rozen et al., 2021; Ben-Hamu et al., 2022)，逐渐发展为如今的流匹配算法 (Lipman et al., 2022; Liu et al., 2022; Albergo and Vanden-Eijnden, 2022; Neklyudov et al., 2023; Heitz et al., 2023; Tong et al., 2023). 所得到的框架包含两个步骤 (Lipman et al., 2022)，见图 2：首先，选择一条在源分布 p 与目标分布 q 之间插值的概率路径 p_t；其次，训练一个速度场（神经网络），使其定义的流变换 ψ_t 实现 p_t.

FM 的原理可以推广到 $\mathbb{R}^d$ 以外的状态空间 $\mathcal{S}$，甚至推广到并非流的演化过程. 最近，**离散流匹配**(Campbell et al., 2024; Gat et al., 2024) 为离散状态空间上的时间连续马尔可夫过程发展了流匹配算法；这类过程也称连续时间马尔可夫链（CTMC），见图 1c. 这一进展使流匹配有望用于语言建模等离散生成任务. **黎曼流匹配**(Chen and Lipman, 2024) 将流匹配推广到黎曼流形 $\mathcal{S} = \mathcal{M}$ 上的流；它如今已成为蛋白质折叠等多种化学机器学习应用中的最先进模型 (Yim et al., 2023; Bose et al., 2023). 更一般地，**生成元匹配**(Holderrieth et al., 2024) 表明，流匹配框架适用于任意模态和一般的**连续时间马尔可夫过程（CTMP）**；如图 1所示，这包括连续空间中的**流**、**扩散**和**跳跃过程**，以及离散空间中的 CTMC. 值得关注的是，对于任意此类 CTMP，流匹配的步骤都相同：首先，选择一条在相应状态空间 $\mathcal{S}$ 上连接源分布 p 与目标分布 q 的路径 p_t；其次，训练一个生成元，它发挥与流的速度类似的作用，定义实现 p_t 的 CTMP 过程. 这一推广使我们能够用统一的观点看待许多现有生成模型，并利用所选择的生成马尔可夫过程，为任意模态开发新的生成模型.

从时间顺序来看，**扩散模型**最早发展了无需模拟的 CTMP 训练方法，其对应的过程是扩散过程，见图 1b. 扩散模型最初以离散时间高斯过程的形式提出 (Sohl-Dickstein et al., 2015; Ho et al.,

2020)，后来被表述为连续时间随机微分方程（SDE）(Song et al., 2021). 从流匹配的视角来看，扩散模型通过由特定 SDE 建模的前向加噪过程，以一种特定方式构造在源分布与目标分布之间插值的概率路径 p_t. 这些 SDE 被选为具有闭式边缘概率的方程，随后通过得分函数利用这些边缘概率来参数化扩散过程的生成元（即漂移和扩散系数）(Song and Ermon, 2019). 这种参数化基于前向加噪过程的反向过程 (Anderson, 1982). 因此，扩散模型学习的是边缘概率的得分函数. 扩散模型文献还提出了得分以外的生成元参数化，包括噪声预测、去噪器 (Kingma et al., 2021) 或v 预测 (Salimans and Ho, 2022)；其中，对于某种特定的概率路径 p_t，后者与速度预测一致. **扩散桥**(Peluchetti, 2023) 提供了另一种设计扩散过程的 p_t 与生成元的方法，将扩散模型推广到任意源与目标耦合. 具体而言，这些构造同样建立在边缘分布有闭式表达的 SDE 上，并同样利用得分表述生成元（使用 Doob 的 h 变换）. Shi et al. (2023); Liu et al. (2023) 表明，线性版本的流匹配可以视为桥匹配的一种极限情形.

本文其余部分安排如下. 节 2给出一个自包含的 "速查指南"，帮助读者理解并使用 PyTorch 实现基本流匹配. 节 3严格讨论连续状态空间中的流模型；可以说，流是所有 CTMP 中最简单的一种. 节 4介绍 $\mathbb{R}^d$ 中的流匹配框架及其各种设计选择与扩展. 我们说明，通过考虑显著更简单的条件情形可以构造流，使设计具有很大灵活性，例如很容易扩展到节 5介绍的黎曼几何. 节 6介绍连续时间马尔可夫链（CTMC）以及它们作为离散状态空间生成模型的用法. 随后，节 7讨论流匹配向 CTMC 过程的扩展. 节 8介绍如何将连续时间马尔可夫过程（CTMP）用作任意状态空间上的生成模型. 节 9介绍生成元匹配（GM）：这是一个适用于任意模态的生成建模框架，提供了可扩展的 CTMP 训练方式，并将前面各节的所有模型统一到一个共同框架中. 最后，鉴于去噪扩散模型的广泛使用，节 10将其作为 FM 模型族的一个具体实例进行讨论.

2 快速入门与关键概念

假设我们能够取得一个训练数据集，其中的样本来自 $\mathbb{R}^d$ 上的某个目标分布 q，我们的目标是构建一个能够从 q 生成新样本的模型. 为完成这一任务，**流匹配（FM）**构造一条从已知源分布 $p_0 = p$ 到数据目标分布 $p_1 = q$ 的**概率路径** $(p_t)_{0\le t\le 1}$，其中每个 p_t 都是 $\mathbb{R}^d$ 上的分布. 具体而言，FM 是一个简单的回归目标，用于训练描述样本瞬时速度的**速度场**神经网络；随后沿概率路径 p_t，利用该网络将源分布 p_0 转换为目标分布 p_1. 训练后，我们通过以下步骤从目标分布生成新样本 $X_1 \sim q$:

1. 从源分布抽取新样本 $X_0 \sim p$;
2. 求解由速度场确定的常微分方程（ODE）.

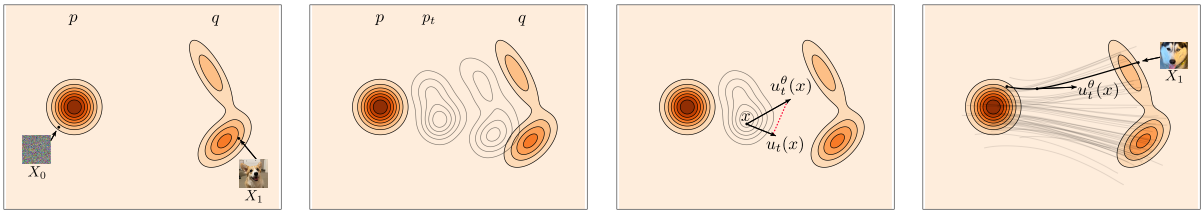
更形式化地说，ODE 由一个随时间变化的向量场 $u : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ 定义；在这里，该向量场是用神经网络建模的速度场. 这个速度场确定一个随时间变化的**流** $\psi : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$，其定义为

$$\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(x) = u_t(\psi_t(x)),$$

其中 $\psi_t := \psi(t,x)$，且 $\psi_0(x) = x$. 如果速度场 u_t 对应的流 ψ_t 满足

$$X_t := \psi_t(X_0) \sim p_t, \text{ 其中} X_0 \sim p_0, \tag{2.1}$$

则称它生成概率路径 p_t. 根据上式，只需速度场 u_t，通过求解上述 ODE 即可从 p_t 采样. 如图 2d所示，将 ODE 求解到 $t = 1$，就能得到近似服从目标分布 q 的样本 $X_1 = \psi_1(X_0)$. 因此，流匹配的目标是学习向量场 u_t^θ，使其流 ψ_t 生成满足 $p_0 = p$ 和 $p_1 = q$ 的概率路径 p_t.



(a) 数据. **(b)** 路径设计. **(c)** 训练. **(d)** 采样.

图 2 流匹配的基本方案. (a) 目标是找到一个流，将已知源分布或噪声分布 q 的样本 X_0 映射为未知目标分布或数据分布 q 的样本 X_1. (b) 为此，设计一条时间连续的概率路径 $(p_t)_{0\le t\le 1}$，在 $p := p_0$ 与 $q := p_1$ 之间插值. (c) 训练期间，用回归估计已知能生成 p_t 的速度场 u_t. (d) 要抽取新的目标样本 $X_1 \sim q$，从新的源样本 $X_0 \sim p$ 出发，将估计的速度场 $u_t^\theta(X_t)$ 从 $t = 0$ 积分至 $t = 1$.

使用上述记号，流匹配的目标是学习由神经网络实现的速度场 u_t^θ 的参数 θ. 正如引言中所述，我们分两步完成这一目标：设计一条在 p 与 q 之间插值的概率路径 p_t（见图 2b），然后通过回归训练一个生成 p_t 的速度场 u_t^θ（见图 2c）.

现在进行第一步：设计概率路径 p_t. 在本例中，令源分布 $p := p_0 = \mathcal{N}(x|0, I)$，并将概率路径 p_t 构造为条件概率路径 $p_{t|1}(x|x_1)$ 的汇集；每条条件路径都以训练数据集中的一个数据样本 $X_1 = x_1$ 为条件. （图 3a展示了其中一条条件路径.）于是概率路径 p_t 的表达式为

$$p_t(x) = \int p_{t|1}(x|x_1)q(x_1)\mathrm{d}x_1, \quad \text{其中} p_{t|1}(x|x_1) = \mathcal{N}(x|tx_1, (1-t)^2 I). \tag{2.2}$$

这条路径也称条件最优传输路径或线性路径，具有一些理想性质，我们将在后文研究这些性质. 使用这条概率路径，可以通过抽取 $X_0 \sim p$ 与 $X_1 \sim q$ 并取它们的线性组合，定义随机变量 $X_t \sim p_t$：

$$X_t = tX_1 + (1-t)X_0 \sim p_t. \tag{2.3}$$

现在进行流匹配的第二步：将速度场 u_t^θ（通常以神经网络实现）回归到一个已知能生成所需概率路径 p_t 的目标速度场 u_t. 为此，**流匹配损失**为

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,X_t}\left\|u_t^\theta(X_t) - u_t(X_t)\right\|^2, \quad \text{其中} t \sim \mathcal{U}[0,1] \text{ 且} X_t \sim p_t. \tag{2.4}$$

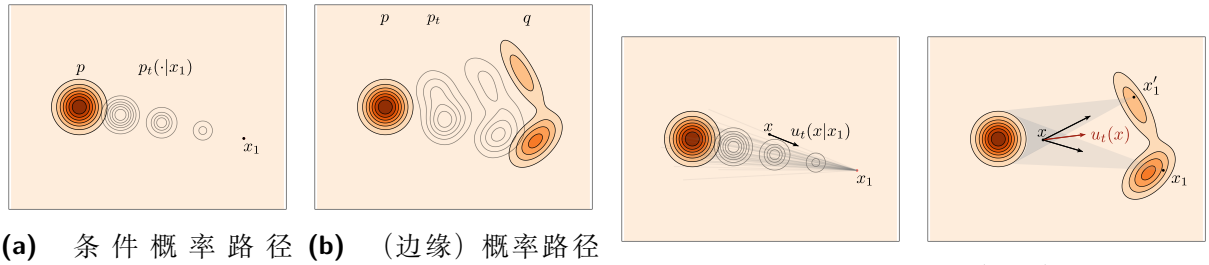
实际上，上述目标很少能直接实现，因为 u_t 是一个复杂对象，支配两个高维分布之间的联合变换. 好在，若以从训练集中随机选取的单个目标样本 $X_1 = x_1$ 为条件，损失目标就会大幅简化. 为说明这一点，利用式 (2.3)构造条件随机变量

$$X_{t|1} = tx_1 + (1-t)X_0 \quad \sim \quad p_{t|1}(\cdot|x_1) = \mathcal{N}(\cdot \mid tx_1, (1-t)^2 I). \tag{2.5}$$

利用这些变量，求解 $\frac{\mathrm{d}}{\mathrm{d}t}X_{t|1} = u_t(X_{t|1}|x_1)$，得到**条件速度场**

$$u_t(x|x_1) = \frac{x_1 - x}{1-t}, \tag{2.6}$$

它生成条件概率路径 $p_{t|1}(\cdot|x_1)$. （这两个条件对象的图示见图 3c.）借助生成所设计条件概率路径的简单条件速度场表达式式 (2.6)，我们可以构造(2.4)中流匹配损失的一个易于计算的版本，



(a) 条件概率路径 $p_t(x|x_1)$. **(b)** （边缘）概率路径 $p_t(x)$. **(c)** 条件速度场 $u_t(x|x_1)$. **(d)** （边缘）速度场 $u_t(x)$.

图 3 流匹配中的路径设计. 给定固定的目标样本 $X = x_1$，其条件速度场 $u_t(x|x_1)$ 生成条件概率路径 $p_t(x|x_1)$. （边缘）速度场 $u_t(x)$ 由所有条件速度场汇集而得；概率路径 $p_t(x)$ 亦然.

即**条件流匹配损失**：

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,X_t,X_1}\|u_t^\theta(X_t) - u_t(X_t|X_1)\|^2, \quad \text{其中} t \sim U[0,1],\ X_0 \sim p,\ X_1 \sim q, \tag{2.7}$$

且 $X_t = (1-t)X_0 + tX_1$. 值得关注的是，(2.4)与(2.7)中的目标为学习 u_t^θ 提供相同的梯度，即

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta). \tag{2.8}$$

最后，将(2.6)中的 $u_t(x|x_1)$ 代入式 (2.7)，得到流匹配最简单的实现：

$$\mathcal{L}_{\text{CFM}}^{\text{OT,Gauss}}(\theta) = \mathbb{E}_{t,X_0,X_1}\|u_t^\theta(X_t) - (X_1 - X_0)\|^2, \quad \text{其中} t \sim U[0,1],\ X_0 \sim \mathcal{N}(0,I),\ X_1 \sim q. \tag{2.9}$$

节 2提供了用纯 PyTorch 实现本节快速入门内容的独立代码. 后文将讨论更复杂的变体与设计选择，它们均在配套的`flow_matching` 程序库中实现.

代码 1: 独立的流匹配代码
flow_matching/examples/standalone_flow_matching.ipynb

```python
import torch
from torch import nn, Tensor
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

class Flow(nn.Module):
  def __init__(self, dim: int = 2, h: int = 64):
    super().__init__()
    self.net = nn.Sequential(
      nn.Linear(dim + 1, h), nn.ELU(),
      nn.Linear(h, h), nn.ELU(),
      nn.Linear(h, h), nn.ELU(),
      nn.Linear(h, dim))

  def forward(self, x_t: Tensor, t: Tensor) -> Tensor:
    return self.net(torch.cat((t, x_t), -1))

  def step(self, x_t: Tensor, t_start: Tensor, t_end: Tensor) -> Tensor:
    t_start = t_start.view(1, 1).expand(x_t.shape[0], 1)
    # 为简化起见，本例使用中点 ODE 求解器
```

```
        return x_t + (t_end - t_start) * self(x_t + self(x_t, t_start) * (t_end -
        ↪  t_start) / 2,
                                              t_start + (t_end - t_start) / 2)

# 训练
flow = Flow()
optimizer = torch.optim.Adam(flow.parameters(), 1e-2)
loss_fn = nn.MSELoss()

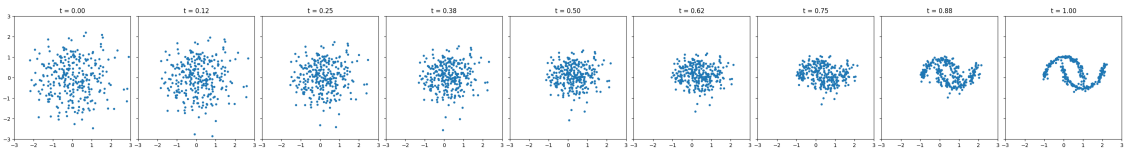
for _ in range(10000):
  x_1  = Tensor(make_moons(256, noise=0.05)[0])
  x_0  = torch.randn_like(x_1)
  t    = torch.rand(len(x_1), 1)
  x_t  = (1 - t) * x_0 + t * x_1
  dx_t = x_1 - x_0
  optimizer.zero_grad()
  loss_fn(flow(x_t, t), dx_t).backward()
  optimizer.step()

# 采样
x = torch.randn(300, 2)
n_steps = 8
fig, axes = plt.subplots(1, n_steps + 1, figsize=(30, 4), sharex=True, sharey=True)
time_steps = torch.linspace(0, 1.0, n_steps + 1)

axes[0].scatter(x.detach()[:, 0], x.detach()[:, 1], s=10)
axes[0].set_title(f't = {time_steps[0]:.2f}')
axes[0].set_xlim(-3.0, 3.0)
axes[0].set_ylim(-3.0, 3.0)

for i in range(n_steps):
  x = flow.step(x, time_steps[i], time_steps[i + 1])
  axes[i + 1].scatter(x.detach()[:, 0], x.detach()[:, 1], s=10)
  axes[i + 1].set_title(f't = {time_steps[i + 1]:.2f}')

plt.tight_layout()
plt.show()
```



3 流模型

本节介绍**流**，它是最简单形式的流匹配所依赖的数学对象. 本文后续部分将讨论比流更一般的马尔可夫过程，由此得到更复杂的生成学习范式，并为流匹配框架引入更多设计选择. 我们从流开始讨论有三个原因. 第一，流可以说是所有 CTMP 中最简单的一类：它们是确定性的，并且可以通过速度进行紧凑的参数化；只要源分布 p 和目标分布 q 都有密度，这些模型就可以将任意 p 变换为任意 q. 第二，通过近似求解 ODE，可以较高效地从流中采样；相比之下，例如描述扩散过程的 SDE 就更难模拟. 第三，流的确定性使模型似然的无偏估计成为可能，而更一般的随机过程则需要使用下界. 要理解流，我们首先必须回顾概率论和微分方程理论中的一些背景概念，下面就进行这一回顾.

3.1 随机向量

考虑 d 维欧氏空间中的数据 $x = (x^1, \ldots, x^d) \in \mathbb{R}^d$，该空间配备标准欧氏内积 $\langle x, y\rangle = \sum_{i=1}^d x^i y^i$ 和范数 $\|x\| = \sqrt{\langle x, x\rangle}$. 我们考虑具有连续概率密度函数（PDF）的随机变量（RV）$X \in \mathbb{R}^d$；该密度是一个连续函数 $p_X : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$，它给出事件 A 的概率

$$\mathbb{P}(X \in A) = \int_A p_X(x)\mathrm{d}x, \tag{3.1}$$

其中 $\int p_X(x)\mathrm{d}x = 1$. 按惯例，对整个空间积分时省略积分区域（$\int \equiv \int_{\mathbb{R}^d}$）. 为简化记号，将随机变量 X_t 的概率密度函数 p_{X_t} 简记为 p_t. 用 $X \sim p$ 或 $X \sim p(X)$ 表示 X 服从分布 p. 生成建模中一种常见的概率密度函数是 d 维各向同性高斯密度：

$$\mathcal{N}(x|\mu, \sigma^2 I) = (2\pi\sigma^2)^{-\frac{d}{2}} \exp\left(-\frac{\|x-\mu\|_2^2}{2\sigma^2}\right), \tag{3.2}$$

其中 $\mu \in \mathbb{R}^d$ 和 $\sigma \in \mathbb{R}_{>0}$ 分别表示该分布的均值和标准差.

随机变量的期望是在最小二乘意义下最接近 X 的常向量：

$$\mathbb{E}[X] = \arg\min_{z \in \mathbb{R}^d} \int \|x - z\|^2 p_X(x)\mathrm{d}x = \int x p_X(x)\mathrm{d}x. \tag{3.3}$$

计算随机变量的函数的期望时，一个有用的工具是无意识统计学家定律：

$$\mathbb{E}[f(X)] = \int f(x) p_X(x)\mathrm{d}x. \tag{3.4}$$

必要时，用 $\mathbb{E}_X f(X)$ 标出取期望所涉及的随机变量.

3.2 条件密度与条件期望

给定两个随机变量 $X, Y \in \mathbb{R}^d$，它们的联合概率密度函数 $p_{X,Y}(x, y)$ 的边缘密度为

$$\int p_{X,Y}(x, y)\mathrm{d}y = p_X(x) \text{ 且 } \int p_{X,Y}(x, y)\mathrm{d}x = p_Y(y). \tag{3.5}$$

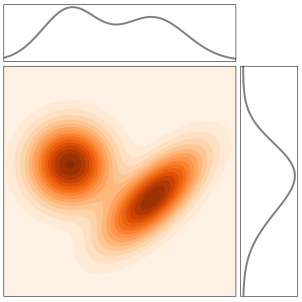


图 4 联合概率密度函数 $p_{X,Y}$（以阴影表示）及其边缘密度 p_X 和 p_Y（以黑线表示）.

图 4展示了 $\mathbb{R}$ 中（$d = 1$）两个随机变量的联合概率密度函数. 条件概率密度函数 $p_{X|Y}$ 描述以满足 $p_Y(y) > 0$ 的事件 $Y = y$ 为条件时，随机变量 X 的概率密度函数：

$$p_{X|Y}(x|y) \coloneqq \frac{p_{X,Y}(x, y)}{p_Y(y)}, \tag{3.6}$$

条件概率密度函数 $p_{Y|X}$ 的定义类似. 贝叶斯公式将条件概率密度函数 $p_{Y|X}$ 用 $p_{X|Y}$ 表示为

$$p_{Y|X}(y|x) = \frac{p_{X|Y}(x|y)p_Y(y)}{p_X(x)}, \tag{3.7}$$

其中 $p_X(x) > 0$.

条件期望 $\mathbb{E}[X|Y]$ 是在最小二乘意义下最佳逼近 X 的函数 $g_\star(Y)$：

$$g_\star \coloneqq \arg\min_{g:\mathbb{R}^d\to\mathbb{R}^d} \mathbb{E}\left[\|X-g(Y)\|^2\right] = \arg\min_{g:\mathbb{R}^d\to\mathbb{R}^d} \int \|x-g(y)\|^2 p_{X,Y}(x,y)\mathrm{d}x\mathrm{d}y$$
$$= \arg\min_{g:\mathbb{R}^d\to\mathbb{R}^d} \int \left[\int \|x-g(y)\|^2 p_{X|Y}(x|y)\mathrm{d}x\right] p_Y(y)\mathrm{d}y. \tag{3.8}$$

因此，对于满足 $p_Y(y)>0$ 的 $y\in\mathbb{R}^d$，条件期望函数为

$$\mathbb{E}[X|Y=y] \coloneqq g_\star(y) = \int x p_{X|Y}(x|y)\mathrm{d}x, \tag{3.9}$$

第二个等号来自对式 (3.8)中内层方括号在 $Y=y$ 时取最小值点，与式 (3.3)类似. 将 $g_\star$ 与随机变量 Y 复合，得到

$$\mathbb{E}[X|Y] \coloneqq g_\star(Y), \tag{3.10}$$

它是一个取值于 $\mathbb{R}^d$ 的随机变量. 容易引起混淆的是，$\mathbb{E}[X|Y=y]$ 和 $\mathbb{E}[X|Y]$ 都经常被称为条件期望，但它们是不同的对象. 具体而言，$\mathbb{E}[X|Y=y]$ 是一个函数 $\mathbb{R}^d\to\mathbb{R}^d$，而 $\mathbb{E}[X|Y]$ 是取值于 $\mathbb{R}^d$ 的随机变量. 为消除这两个术语的歧义，后续讨论将使用此处引入的记号.

*塔式性质*有助于简化涉及两个随机变量 X 与 Y 的条件期望的推导：

$$\mathbb{E}[\mathbb{E}[X|Y]] = \mathbb{E}[X] \tag{3.11}$$

由于 $\mathbb{E}[X|Y]$ 是随机变量，并且本身是随机变量 Y 的函数，外层期望计算的是 $\mathbb{E}[X|Y]$ 的期望. 利用上面的一些定义，可以验证塔式性质：

$$\begin{aligned}\mathbb{E}[\mathbb{E}[X|Y]] &= \int\left(\int x p_{X|Y}(x|y)\mathrm{d}x\right) p_Y(y)\mathrm{d}y\\ &\overset{(3.6)}{=} \int\int x p_{X,Y}(x,y)\mathrm{d}x\mathrm{d}y\\ &\overset{(3.5)}{=} \int x p_X(x)\mathrm{d}x\\ &= \mathbb{E}[X].\end{aligned}$$

最后，考虑涉及两个随机变量 $f(X,Y)$ 与 Y 的一个有用性质，其中 X 与 Y 是任意两个随机变量. 结合(3.9)，使用无意识统计学家定律，得到恒等式

$$\mathbb{E}[f(X,Y)|Y=y] = \int f(x,y)p_{X|Y}(x|y)\mathrm{d}x. \tag{3.12}$$

3.3 微分同胚与推前映射

用 $C^r(\mathbb{R}^m,\mathbb{R}^n)$ 表示具有连续 r 阶偏导数的函数 $f:\mathbb{R}^m\to\mathbb{R}^n$ 的集合：

$$\frac{\partial^r f^k}{\partial x^{i_1}\cdots\partial x^{i_r}}, \qquad k\in[n], i_j\in[m], \tag{3.13}$$

其中 $[n]\coloneqq\{1,2,\ldots,n\}$. 为简化记号，也定义 $C^r(\mathbb{R}^n)\coloneqq C^r(\mathbb{R}^m,\mathbb{R})$；例如，$C^1(\mathbb{R}^m)$ 表示连续可微的标量函数. 一类重要的函数是C^r **微分同胚**；它们是满足 $\psi^{-1}\in C^r(\mathbb{R}^n,\mathbb{R}^n)$ 的可逆函数

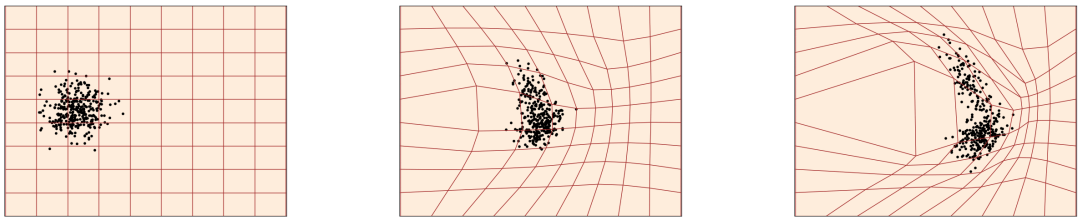


图 5 流模型 $X_t = \psi_t(X_0)$ 由微分同胚 $\psi_t : \mathbb{R}^d \to \mathbb{R}^d$（用棕色方格网表示）定义，将源随机变量 X_0 的样本（左，黑点）推向某个目标分布 q（右）. 图中展示了三个不同的时刻 t.

$\psi \in C^r(\mathbb{R}^n, \mathbb{R}^n)$.

给定具有密度 p_X 的随机变量 $X \sim p_X$，考虑随机变量 $Y = \psi(X)$，其中 $\psi : \mathbb{R}^d \to \mathbb{R}^d$ 是 C^1 微分同胚. Y 的概率密度函数记为 p_Y，也称 p_X 的**推前**. 概率密度函数 p_Y 可以通过变量替换计算：

$$\mathbb{E}\left[f(Y)\right] = \mathbb{E}\left[f(\psi(X))\right] = \int f(\psi(x))p_X(x)\mathrm{d}x = \int f(y)p_X(\psi^{-1}(y))\left|\det \partial_y \psi^{-1}(y)\right|\mathrm{d}y,$$

第三个等号来自变量替换 $x = \psi^{-1}(y)$；$\partial_y \phi(y)$ 表示雅可比矩阵（由一阶偏导数组成），即

$$[\partial_y \phi(y)]_{i,j} = \frac{\partial \phi^i}{\partial x^j},\ i, j \in [d],$$

$\det A$ 表示方阵 $A \in \mathbb{R}^{d\times d}$ 的行列式. 因此，概率密度函数 p_Y 为

$$p_Y(y) = p_X(\psi^{-1}(y))\left|\det \partial_y \psi^{-1}(y)\right|. \tag{3.14}$$

用符号 $\sharp$ 表示推前算子，即

$$\left[\psi_\sharp p_X\right](y) \coloneqq p_X(\psi^{-1}(y))\left|\det \partial_y \psi^{-1}(y)\right|. \tag{3.15}$$

3.4 作为生成模型的流

如节 2所述，生成建模的目标是将**源分布** p 的样本 $X_0 = x_0$ 变换为**目标分布** q 的样本 $X_1 = x_1$. 本节开始构建借助流映射 ψ_t 解决这一问题所需的工具. 更形式化地说，C^r **流**是一个随时间变化的映射 $\psi : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$，其作用为 $\psi : (t, x) \mapsto \psi_t(x)$. 这样的流也是 $C^r([0,1] \times \mathbb{R}^d, \mathbb{R}^d)$ 函数，并且对每个 $t \in [0,1]$，函数 $\psi_t(x)$ 关于 x 是 C^r 微分同胚. **流模型**是通过将流 ψ_t 作用于随机变量 X_0 定义的连续时间马尔可夫过程 $(X_t)_{0\le t\le 1}$：

$$X_t = \psi_t(X_0), \quad t \in [0,1], \text{ 其中} X_0 \sim p. \tag{3.16}$$

流模型的图示见图 5. 要说明 X_t 为马尔可夫过程，注意对任意 $0 \le t < s \le 1$，有

$$X_s = \psi_s(X_0) = \psi_s(\psi_t^{-1}(\psi_t(X_0))) = \psi_{s|t}(X_t), \tag{3.17}$$

最后一个等号使用式 (3.16)中的 $X_t = \psi_t(X_0)$，并定义 $\psi_{s|t} \coloneqq \psi_s \circ \psi_t^{-1}$；它也是一个微分同胚. $X_s = \psi_{s|t}(X_t)$ 意味着 X_t 之后的状态只依赖于 X_t，所以 X_t 为马尔可夫过程. 实际上，对于流

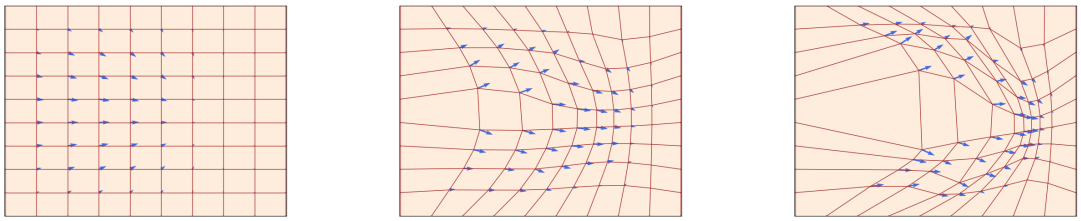


图 6 流 $\psi_t : \mathbb{R}^d \to \mathbb{R}^d$（方格网）由速度场 $u_t : \mathbb{R}^d \to \mathbb{R}^d$（蓝色箭头）定义，该速度场规定了流在各个位置的瞬时运动. 图中展示了三个不同的时刻 t.

模型，这种依赖是确定性的.

因此，**生成式流建模**的目标是找到一个流 ψ_t，使得

$$X_1 = \psi_1(X_0) \sim q. \tag{3.18}$$

3.4.1 流与速度场的等价性

C^r 流 ψ 可以通过下列 ODE，用一个属于 $C^r([0,1] \times \mathbb{R}^d, \mathbb{R}^d)$ 的速度场 $u : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ 定义，该速度场的作用为 $u : (t, x) \mapsto u_t(x)$：

$$\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(x) = u_t(\psi_t(x)) \qquad \text{（流 ODE）} \tag{3.19a}$$

$$\psi_0(x) = x \qquad \text{（流的初始条件）} \tag{3.19b}$$

图 6展示了流及其速度场.

关于式 (3.19)的解 $\psi_t(x)$ 的存在性与唯一性，有如下标准结论（例如见Perko (2013); Coddington et al. (1956)）：

定理 1 (流的局部存在性与唯一性). *如果 $u \in C^r([0,1] \times \mathbb{R}^d, \mathbb{R}^d)$，$r \geq 1$（特别地，$u$ 局部 Lipschitz），那么(3.19)中的 ODE 有唯一解，它是定义在开集 Ω 上的 $C^r(\Omega, \mathbb{R}^d)$ 微分同胚 $\psi_t(x)$，且 Ω 包含 $\{0\} \times \mathbb{R}^d$.*

这个定理只保证 C^r 流的局部存在性与唯一性：对于每个 $x \in \mathbb{R}^d$，流 $\psi_t(x)$ 使它移动的时间可能限于 $t \in [0, t_x)$. 要对所有 $x \in \mathbb{R}^d$ 保证解存在到 $t = 1$，必须在局部 Lipschitz 性之外加入额外假设. 例如，可以考虑全局 Lipschitz 性；在 C^1 情形下，有界的一阶导数可保证这一性质. 不过，后文将依靠另一种条件，即可积性，来保证流几乎处处存在且延续至 $t = 1$.

目前，我们已经说明速度场唯一地定义流. 反过来，给定 C^1 流 ψ_t，对任意 $x \in \mathbb{R}^d$，考虑方程 $\frac{\mathrm{d}}{\mathrm{d}t}\psi_t(x') = u_t(\psi_t(x'))$，并利用每个 $t \in [0,1]$ 时 ψ_t 都是可逆微分同胚这一事实，令 $x' = \psi_t^{-1}(x)$，即可求出定义它的速度场 $u_t(x)$. 因此，确定流 ψ_t 的唯一速度场 u_t 为

$$u_t(x) = \dot{\psi}_t(\psi_t^{-1}(x)), \tag{3.20}$$

其中 $\dot{\psi}_t := \frac{\mathrm{d}}{\mathrm{d}t}\psi_t$. 由此，我们说明了 C^r 流 ψ_t 与 C^r 速度场 u_t 之间的等价性.

3.4.2 由源样本计算目标样本

计算目标样本 X_1，或更一般地计算任意样本 X_t，需要从某个初值 $X_0 = x_0$ 出发，近似求解式 (3.19)中的 ODE. ODE 数值方法是数值分析中一个经典且研究充分的课题，已有许多强大的方法 (Iserles, 2009). 其中最简单的方法之一是欧拉方法，其更新规则为

$$X_{t+h} = X_t + hu_t(X_t) \tag{3.21}$$

其中 $h = n^{-1} > 0$ 为步长超参数，$n \in \mathbb{N}$. 要从目标分布抽取样本 X_1，从某个 $X_0 \sim p$ 出发应用欧拉方法，产生序列 $X_h, X_{2h}, \ldots, X_1$. 欧拉方法与 X_t 的一阶泰勒展开一致：

$$X_{t+h} = X_t + h\dot{X}_t + o(h) = X_t + hu_t(X_t) + o(h),$$

其中 $o(h)$ 表示比 h 增长更慢的函数，即当 $h \to 0$ 时，$o(h)/h \to 0$. 因此，欧拉方法每一步累积 $o(h)$ 的误差，并且可以证明，经过 $n = 1/h$ 步后累积 $o(1)$ 的误差. 所以，当步长减小至 $h \to 0$ 时，欧拉方法的误差趋于零. 欧拉方法只是诸多 ODE 求解器中的一个例子. 小节 3.4.2展示了另一种选择，即二阶中点方法；它在实践中往往优于欧拉方法.

代码 2: 用中点求解器计算 X_1

```python
from flow_matching.solver import ODESolver
from flow_matching.utils import ModelWrapper

class Flow(ModelWrapper):
    def __init__(self, dim=2, h=64):
        super().__init__()
        self.net = torch.nn.Sequential(
            torch.nn.Linear(dim + 1, h), torch.nn.ELU(),
            torch.nn.Linear(h, dim))

    def forward(self, x, t):
        t = t.view(-1, 1).expand(*x.shape[:-1], -1)
        return self.net(torch.cat((t, x), -1))

velocity_model = Flow()

... # 优化模型参数，使 model(x_t, t) = u_t(X_t)

x_0 = torch.randn(batch_size, *data_dim)  # 指定初始条件

solver = ODESolver(velocity_model=velocity_model)
num_steps = 100
x_1 = solver.sample(x_init=x_0, method='midpoint', step_size=1.0 / num_steps)
```

3.5 概率路径与连续性方程

将随时间变化的概率 $(p_t)_{0 \le t \le 1}$ 称为**概率路径**. 在这里，一条重要的概率路径是流模型 $X_t = \psi_t(X_0)$ 在时刻 t 的边缘概率密度函数：

$$X_t \sim p_t. \tag{3.22}$$

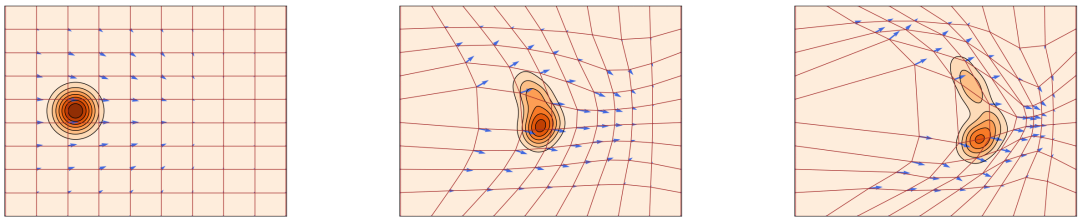


图 7 如果由速度场 u_t（蓝色）定义的流（方格网）在每个时刻 $t \in [0,1)$ 都将 p（左）变为 p_t，则称 u_t 生成概率路径 p_t（以等高线表示概率密度函数）.

对于每个时刻 $t \in [0,1]$，这些边缘概率密度函数可由式 (3.15)的推前公式得到，即

$$p_t(x) = [\psi_{t\sharp} p](x). \tag{3.23}$$

给定任意概率路径 p_t，定义

$$u_t \text{ 生成 } p_t, \text{ 若} X_t = \psi_t(X_0) \sim p_t \text{ 对所有} t \in [0,1). \tag{3.24}$$

由此，速度场、它们对应的流与所生成的概率路径之间建立了紧密联系，图 7给出了示意. 注意，此处采用右端开口的时间区间 $[0,1)$，以允许处理具有紧支撑的目标分布 q；在这种情形下，速度可能恰好在 $t=1$ 时没有定义.

要验证速度场 u_t 生成概率路径 p_t，可以检验二元组 (u_t, p_t) 是否满足称为连续性方程的偏微分方程（PDE）：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) + \operatorname{div}(p_t u_t)(x) = 0, \tag{3.25}$$

其中 $\operatorname{div}(v)(x) = \sum_{i=1}^{d} \partial_{x^i} v^i(x)$，且 $v(x) = (v^1(x), \ldots, v^d(x))$. 下面的定理是质量守恒公式 (Villani et al., 2009) 的一种重新表述，它说明连续性方程的解 u_t 生成概率路径 p_t：

定理 2 (质量守恒). *设 p_t 是概率路径，u_t 是局部 Lipschitz 且可积的向量场. 则以下两个命题等价：*

1. *连续性方程(3.25)对 $t \in [0,1)$ 成立.*
2. *u_t 在(3.24)的意义下生成 p_t.*

上述定理中的局部 Lipschitz 性是指，对每个 (t,x)，存在一个局部邻域，使 $u_t(x)$ 在该邻域上满足 Lipschitz 条件. 假设 u 可积是指：

$$\int_0^1 \int \|u_t(x)\| \, p_t(x) \mathrm{d}x \mathrm{d}t < \infty. \tag{3.26}$$

具体而言，在时间区间 $[0,t]$ 上对流 ODE (3.19a)的解积分，得到积分方程

$$\psi_t(x) = x + \int_0^t u_s(\psi_s(x)) \mathrm{d}s. \tag{3.27}$$

因此，可积性意味着

$$\begin{aligned}\mathbb{E}\,\|X_t\| &\overset{(3.16)}{=} \int \|\psi_t(x)\|\, p(x)\mathrm{d}x \\ &= \int \left\| x + \int_0^t u_s(\psi_s(x))ds \right\| p(x)\mathrm{d}x \\ &\overset{(i)}{\leq} \mathbb{E}\,\|X_0\| + \int_0^1 \int \|u_s(x)\|\, p_t(x)\mathrm{d}t \\ &\overset{(ii)}{<} \infty,\end{aligned}$$

其中（i）来自三角不等式，（ii）使用可积性条件(3.26)和 $\mathbb{E}\,\|X_0\| < \infty$. 因此，如果 X_0 的范数期望有界，可积性使我们能够假定 X_t 的范数期望也有界.

为进一步理解连续性方程的含义，可以利用散度定理将其写成积分形式；直观介绍见Matthews (2012)，严格论述见Loomis and Sternberg (1968). 这个结果说明，对于某个区域 $\mathcal{D}$ 和某个光滑向量场 $u : \mathbb{R}^d \to \mathbb{R}^d$，将 $\mathcal{D}$ 内 u 的散度累积起来，等于沿垂直于边界 $\partial\mathcal{D}$ 的方向穿过该边界而离开 $\mathcal{D}$ 的通量：

$$\int_{\mathcal{D}} \mathrm{div}(u)(x)\mathrm{d}x = \int_{\partial\mathcal{D}} \langle u(y), n(y)\rangle \,\mathrm{d}s_y, \tag{3.28}$$

其中 $n(y)$ 是区域边界 $\partial\mathcal{D}$ 上指向外侧的单位法向量场，$\mathrm{d}s_y$ 为边界的面积元. 为将上述认识应用于连续性方程，对一个小区域 $\mathcal{D} \subset \mathbb{R}^d$（例如立方体）上的(3.25)积分，并应用散度定理，得到

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}t}\int_{\mathcal{D}} p_t(x)\mathrm{d}x &= -\int_{\mathcal{D}} \mathrm{div}(p_t u_t)(x)\mathrm{d}x \\ &= -\int_{\partial\mathcal{D}} \langle p_t(y)u_t(y), n(y)\rangle \,\mathrm{d}s_y.\end{aligned} \tag{3.29}$$

这个方程将体积区域 $\mathcal{D}$ 中总概率质量的变化率（左侧）表示为离开该区域的概率通量的相反数（右侧）. 概率通量定义为 $j_t(y) = p_t(y)u_t(y)$，表示每单位时间、每单位面积（可能是高维面积）流过垂直于 $n(y)$ 的超平面的概率质量. 图示见图 8.

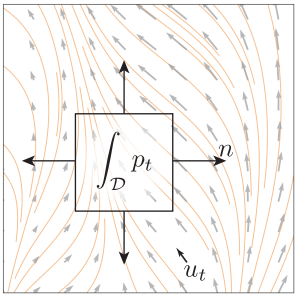


图 8 连续性方程说明，概率的局部变化等于净流出概率通量的相反数.

3.6 瞬时变量替换公式

使用流作为生成模型的一个重要优点是，对所有 $x \in \mathbb{R}^d$，都可以有效计算精确似然 $\log p_1(x)$. 这一性质来自连续性方程的一个推论，称为瞬时变量替换公式 (Chen et al., 2018)：

$$\frac{\mathrm{d}}{\mathrm{d}t}\log p_t(\psi_t(x)) = -\mathrm{div}(u_t)(\psi_t(x)). \tag{3.30}$$

这是一个 ODE，它支配沿流 ODE (3.19a)定义的采样轨迹 $\psi_t(x)$ 的对数似然 $\log p_t(\psi_t(x))$ 的变化. 为推导(3.30)，对 $\log p_t(\psi_t(x))$ 关于时间求导，再应用连续性方程(3.25)和流 ODE (3.19a). 将(3.30)从 $t = 0$ 积分到 $t = 1$ 并整理，得到

$$\log p_1(\psi_1(x)) = \log p_0(\psi_0(x)) - \int_0^1 \mathrm{div}(u_t)(\psi_t(x))\mathrm{d}t. \tag{3.31}$$

实际上，$\operatorname{div}(u_t)$ 等于雅可比矩阵 $\partial_x u_t(x) \in \mathbb{R}^{d\times d}$ 的迹；随着维数 d 增大，计算它越来越困难. 因此，已有工作使用无偏估计量，例如 Hutchinson 迹估计量 (Grathwohl et al., 2018)：

$$\operatorname{div}(u_t)(x) = \operatorname{tr}\left[\partial_x u_t(x)\right] = \mathbb{E}_Z \operatorname{tr}\left[Z^T \partial_x u_t(x) Z\right], \tag{3.32}$$

其中 $Z \in \mathbb{R}^{d\times d}$ 是满足 $\mathbb{E}[Z] = 0$ 和 $\operatorname{Cov}(Z, Z) = I$ 的任意随机变量（例如 $Z \sim \mathcal{N}(0, I)$），且 $\operatorname{tr}[Z] = \sum_{i=1}^{d} Z_{i,i}$. 将上式代入(3.31)，并交换积分与期望的次序，得到如下无偏对数似然估计量：

$$\log p_1(\psi_1(x)) = \log p_0(\psi_0(x)) - \mathbb{E}_Z \int_0^1 \operatorname{tr}\left[Z^T \partial_x u_t(\psi_t(x)) Z\right] \mathrm{d}t. \tag{3.33}$$

与(3.30)中的 $\operatorname{div}(u_t)(\psi_t(x))$ 相比，对于上式中固定的样本 Z，计算 $\operatorname{tr}\left[Z^T \partial_x u_t(\psi_t(x)) Z\right]$ 只需借助向量与雅可比矩阵乘积（JVP）进行一次反向传播[1].

因此，计算 $\log p_1(x)$ 的无偏估计，需要模拟 ODE

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} f(t) \\ g(t) \end{bmatrix} = \begin{bmatrix} u_t(f(t)) \\ -\operatorname{tr}\left[Z^T \partial_x u_t(f(t)) Z\right] \end{bmatrix}, \tag{3.34a}$$

$$\begin{bmatrix} f(1) \\ g(1) \end{bmatrix} = \begin{bmatrix} x \\ 0 \end{bmatrix}, \tag{3.34b}$$

沿时间反向从 $t = 1$ 到 $t = 0$，并令：

$$\widehat{\log p_1}(x) = \log p_0(f(0)) - g(0). \tag{3.35}$$

小节 3.6给出了利用`flow_matching` 程序库从流模型获得对数似然估计的示例.

代码 3: 计算似然

```python
from flow_matching.solver import ODESolver
from flow_matching.utils import ModelWrapper
from torch.distributions.normal import Normal

velocity_model: ModelWrapper = ...  # 训练模型参数，使 model(x_t, t) = u_t(x_t)

x_1 = torch.randn(batch_size, *data_dim)  # 希望计算 log p_1(x) 的点 X_1

# 定义 log p_0(x)
gaussian_log_density = Normal(torch.zeros(size=data_dim),
    ↪ torch.ones(size=data_dim)).log_prob

solver = ODESolver(velocity_model=velocity_model)
num_steps = 100
x_0, log_p1 = solver.compute_likelihood(
    x_1=x_1,
    method='midpoint',
    step_size=1.0 / num_steps,
    log_p0=gaussian_log_density
)
```

[1]例如见https://pytorch.org/docs/stable/generated/torch.autograd.functional.vjp.html.

3.7 通过模拟训练流模型

瞬时变量替换公式及其导出的 ODE 系统(3.34)，使我们能够通过最大化训练数据的对数似然来训练流模型 (Chen et al., 2018; Grathwohl et al., 2018). 具体而言，设 u_t^θ 为具有可学习参数 $\theta \in \mathbb{R}^p$ 的速度场，考虑学习 θ 使得

$$p_1^\theta \approx q. \tag{3.36}$$

例如，可以通过最小化 p_1^θ 与 q 的 KL 散度实现这一目标：

$$\mathcal{L}(\theta) = D_{\mathrm{KL}}(q, p_1^\theta) = -\mathbb{E}_{Y\sim q} \log p_1^\theta(Y) + \text{常数}, \tag{3.37}$$

其中 p_1^θ 是 $X_1 = \psi_1^\theta(X_0)$ 的分布，ψ_t^θ 由 u_t^θ 定义；通过求解 ODE 系统(3.34)，可以得到 $\log p_1^\theta(Y)$ 的无偏估计. 不过，计算这一损失及其梯度需要在训练期间进行精确的 ODE 模拟；只有无误差的解才能产生无偏梯度. 接下来介绍的流匹配是一个无需模拟的流生成模型训练框架，训练期间无需求解 ODE.

4 流匹配

给定源分布 p 和目标分布 q，流匹配（Flow Matching，FM）(Lipman et al., 2022; Liu et al., 2022; Albergo and Vanden-Eijnden, 2022) 是一种可扩展的方法，用于训练由可学习速度 u_t^θ 定义的流模型，并求解**流匹配问题**：

$$\text{寻找} u_t^\theta\text{，使其生成} p_t\text{，其中} p_0 = p \text{ 且} p_1 = q. \tag{4.1}$$

上式中的 “生成” 采用 式 (3.24) 中的含义. 回顾 图 2 中流匹配的蓝图，FM 框架包含以下步骤：(a) 确定已知的源分布 p 和未知的数据目标分布 q；(b) 指定从 $p_0 = p$ 插值到 $p_1 = q$ 的概率路径 p_t；(c) 学习由神经网络实现、能够生成路径 p_t 的速度场 u_t^θ；(d) 通过求解由 u_t^θ 定义的常微分方程，从学得的模型中采样. 为学习步骤 (c) 中的速度场 u_t^θ，FM 最小化以下回归损失：

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{X_t\sim p_t} D\left(u_t(X_t), u_t^\theta(X_t)\right), \tag{4.2}$$

其中 D 是向量之间的相异性度量，例如平方 ℓ_2 范数 $D(u, v) = \|u - v\|^2$. 直观地说，FM 损失促使可学习速度场 u_t^θ 与真实速度场 u_t 相匹配，后者已知能够生成所需的概率路径 p_t. 图 9 展示了流匹配框架中的主要对象及其依赖关系. 我们首先说明如何构造 p_t 和 u_t，以及如何实际实现损失 (4.2)，由此展开对流匹配的介绍.

4.1 数据

再次说明，令源样本为随机变量 $X_0 \sim p$，目标样本为随机变量 $X_1 \sim q$. 通常，源样本服从易于采样的已知分布，而目标样本则以有限大小的数据集形式给出. 根据具体应用，目标样本可以是图像、视频、音频片段，或其他类型的高维、结构丰富的数据. 源样本与目标样本可以相互独立，也可以来自称为**耦合**的一般联合分布

$$(X_0, X_1) \sim \pi_{0,1}(X_0, X_1), \tag{4.3}$$

其中，若没有已知耦合，源样本与目标样本服从独立耦合 $\pi_{0,1}(X_0, X_1) = p(X_0)q(X_1)$. 独立源分布与目标分布的一个常见例子，是从随机高斯噪声向量 $X_0 \sim \mathcal{N}(0, I)$ 生成图像 X_1. 作为非独立

耦合的例子，可以考虑从低分辨率图像 X_0 生成其高分辨率版本 X_1，或从灰度视频 X_0 生成对应的彩色视频 X_1.

4.2 构造概率路径

流匹配采用条件策略，大幅简化了设计概率路径 p_t 及其对应速度场 u_t 的问题. 首先，考虑以单个目标样本 $X_1 = x_1$ 为条件设计 p_t，得到 图 3a 所示的**条件概率路径** $p_{t|1}(x|x_1)$. 随后，可以通过汇总这些条件概率路径 $p_{t|1}$，构造整体的**边缘概率路径** p_t：

$$p_t(x) = \int p_{t|1}(x|x_1)q(x_1)\mathrm{d}x_1, \tag{4.4}$$

如 图 3b 所示. 为求解流匹配问题，我们希望 p_t 满足以下边界条件：

$$p_0 = p, \quad p_1 = q, \tag{4.5}$$

也就是说，边缘概率路径 p_t 从时刻 $t = 0$ 的源分布 p 插值到时刻 $t = 1$ 的目标分布 q. 要求条件概率路径满足以下条件，即可保证这些边界条件成立：

$$p_{0|1}(x|x_1) = \pi_{0|1}(x|x_1), \text{ 且} p_{1|1}(x|x_1) = \delta_{x_1}(x), \tag{4.6}$$

其中，条件耦合为 $\pi_{0|1}(x_0|x_1) = \pi_{0,1}(x_0, x_1)/q(x_1)$，$\delta_{x_1}$ 是以 x_1 为中心的狄拉克测度. 对于独立耦合 $\pi_{0,1}(x_0, x_1) = p(x_0)q(x_1)$，上述第一个约束化为 $p_{0|1}(x|x_1) = p(x)$. 由于狄拉克测度没有密度，第二个约束应理解为：对于连续函数 f，当 $t \to 1$ 时，$\int p_{t|1}(x|y)f(y)\mathrm{d}y \to f(x)$. 注意，将 (4.6) 代入 (4.4) 即可验证边界条件 (4.5).

(2.2) 给出了满足 (4.6) 中条件的一种常用条件概率路径：

$$\mathcal{N}(\cdot \,|\, tx_1, (1-t)^2 I) \to \delta_{x_1}(\cdot) \text{ 当} t \to 1.$$

4.3 推导生成概率路径的速度场

有了边缘概率路径 p_t，现在我们构造能够生成 p_t 的速度场 u_t. 生成速度场 u_t 是多个**条件速度场** $u_t(x|x_1)$ 的平均，后者如 图 3c 所示，并满足：

$$u_t(\cdot|x_1) \text{ 生成} p_{t|1}(\cdot|x_1). \tag{4.7}$$

随后，通过对各个目标样本的条件速度场 $u_t(x|x_1)$ 取平均，得到生成边缘路径 $p_t(x)$ 的**边缘速度场** $u_t(x)$，如 图 3d 所示：

$$u_t(x) = \int u_t(x|x_1)p_{1|t}(x_1|x)\mathrm{d}x_1. \tag{4.8}$$

为用已知量表示上式，回顾贝叶斯公式

$$p_{1|t}(x_1|x) = \frac{p_{t|1}(x|x_1)q(x_1)}{p_t(x)}, \tag{4.9}$$

该式对所有满足 $p_t(x) > 0$ 的 x 定义. 式 (4.8) 可以解释为条件速度 $u_t(x|x_1)$ 的加权平均，其中权重 $p_{1|t}(x_1|x)$ 表示给定当前样本 x 时目标样本 x_1 的后验概率. 还可以用条件期望解释 (4.8)（见 小节 3.2）. 具体来说，若 X_t 是满足 $X_t \sim p_{t|1}(\cdot|X_1)$ 的任意随机变量，或者等价地，(X_t, X_1)

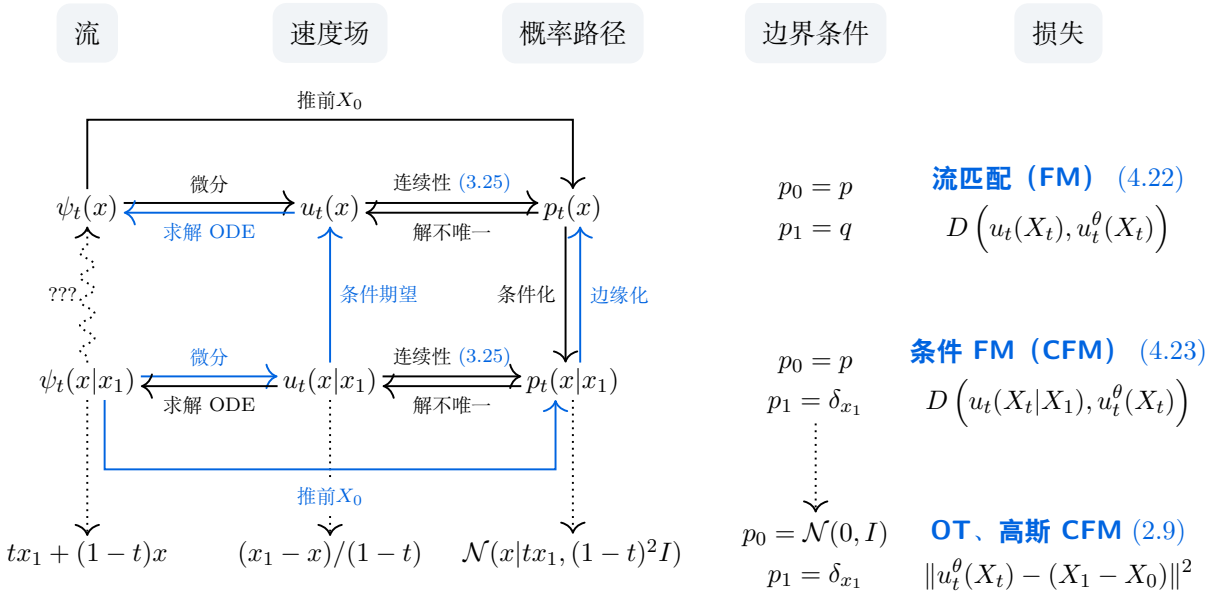


图 9 流匹配框架中的主要对象及其关系. 流 由一个 速度场 表示，该速度场定义了生成 概率路径 的随机过程. 流匹配的主要思想，是将满足所需 边界条件 的复杂流的构造（顶行），分解为满足更简单 边界条件 、因而更容易求解的条件流（中间行）. 箭头表示不同对象之间的依赖关系；**蓝色箭头**表示流匹配框架采用的关系. 损失 一列列出了用于学习 速度场 的损失，其中 CFM 损失（中间行和底行）是实际使用的损失. 底行列出了 节 2 中所述最简单的 FM 算法实例.

的联合分布具有密度 $p_{t,1}(x, x_1) = p_{t|1}(x|x_1)q(x_1)$，则利用 (3.12) 将 (4.8) 写成条件期望，得到

$$u_t(x) = \mathbb{E}\left[u_t(X_t|X_1) \,|\, X_t = x\right], \tag{4.10}$$

由此得到一个有用的解释：$u_t(x)$ 是给定 $X_t = x$ 时对 $u_t(X_t|X_1)$ 的最小二乘近似，见 小节 3.2. 注意，(4.10) 中的 X_t 一般与最终流模型 (3.16) 定义的 X_t 是不同的随机变量，尽管它们具有相同的边缘概率 $p_t(x)$.

4.4 一般条件化与边缘化技巧

为证明上述构造合理，我们需要说明：在温和假设下，式 (4.8) and (4.10) 中的边缘速度场 u_t 能够生成 式 (4.4) 中的边缘概率路径 p_t. 用于证明这一结论的数学工具是质量守恒定理（定理 2）. 接下来，考虑一个稍为一般的设定，它将在本文后续内容中发挥作用. 特别地，以 $X_1 = x_1$ 为条件构造条件概率路径和速度场，并没有特殊之处. 如 Tong et al. (2023) 所指出，上一节的分析同样适用于以任意具有概率密度函数 p_Z 的随机变量 $Z \in \mathbb{R}^m$ 为条件的情形. 由此得到**边缘概率路径**

$$p_t(x) = \int p_{t|Z}(x|z)p_Z(z)dz, \tag{4.11}$$

它由以下**边缘速度场**生成：

$$u_t(x) = \int u_t(x|z)p_{Z|t}(z|x)dz = \mathbb{E}\left[u_t(X_t|Z) \mid X_t = x\right], \tag{4.12}$$

其中 $u_t(\cdot|z)$ 生成 $p_{t|Z}(\cdot|z)$，在 $p_t(x) > 0$ 时由贝叶斯公式得到 $p_{Z|t}(z|x) = \frac{p_{t|Z}(x|z)p_Z(z)}{p_t(x)}$，并且 $X_t \sim p_{t|Z}(\cdot|Z)$. 自然地，令 $Z = X_1$ 即可恢复前面各节中的构造. 在证明主要结果之前，需要一些正则性假设，概括如下.

假设 1. 作为 (t, x) 的函数，$p_{t|Z}(x|z)$ 属于 $C^1([0,1) \times \mathbb{R}^d)$，$u_t(x|z)$ 属于 $C^1([0,1) \times \mathbb{R}^d, \mathbb{R}^d)$. 此外，$p_Z$ 具有有界支撑，即在 $\mathbb{R}^m$ 中某个有界集合之外，$p_Z(x) = 0$. 最后，对所有 $x \in \mathbb{R}^d$ 和 $t \in [0,1)$，都有 $p_t(x) > 0$.

这些假设是温和的. 例如，找到满足 $p_Z(z) > 0$ 且 $p_{t|Z}(\cdot|z) > 0$ 的条件 z，即可证明 $p_t(x) > 0$. 实践中，对于任意小的 $\epsilon > 0$，考虑 $(1 - (1-t)\epsilon)p_{t|Z} + (1-t)\epsilon\mathcal{N}(0, I)$ 即可满足这一要求. 满足该假设的 $p_{t|Z}(\cdot|z)$ 的一个例子，是 (2.2) 中的路径，其中令 $Z = X_1$. 现在可以陈述主要结果：

定理 3 (边缘化技巧)**.** *在 假设 1 下，若 $u_t(x|z)$ 条件可积，并生成条件概率路径 $p_t(\cdot|z)$，则边缘速度场 u_t 对所有 $t \in [0,1)$ 都生成边缘概率路径 p_t.*

上述定理中的条件可积，指质量守恒定理中可积性条件 (3.26) 的条件版本，即：

$$\int_0^1 \int \int \|u_t(x|z)\| \, p_{t|Z}(x|z)p_Z(x)dzdxdt < \infty. \tag{4.13}$$

证明. 验证 定理 2 中质量守恒的两个条件即可得到结论. 首先，验证 (u_t, p_t) 满足连续性方程 (3.25). 由于 $u_t(\cdot|x_1)$ 生成 $p_t(\cdot|x_1)$，有

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}t}p_t(x) &\overset{(i)}{=} \int \frac{\mathrm{d}}{\mathrm{d}t}p_{t|Z}(x|z)p_Z(x)dz && (4.14)\\
&\overset{(ii)}{=} -\int \mathrm{div}_x \left[u_t(x|z)p_{t|Z}(x|z)\right] p_Z(z)dz && (4.15)\\
&\overset{(i)}{=} -\mathrm{div}_x \int u_t(x|z)p_{t|Z}(x|z)p_Z(z)dz && (4.16)\\
&\overset{(iii)}{=} -\mathrm{div}_x \left[u_t(x)p_t(x)\right]. && (4.17)
\end{aligned}$$

等式 (i) 来自交换微分（分别为 $\frac{\mathrm{d}}{\mathrm{d}t}$ 和 div_x）与积分. 这一交换由莱布尼茨法则、$p_{t|Z}(x|z)$ 和 $u_t(x|z)$ 关于 t, x 为 C^1，以及 p_Z 具有有界支撑这些事实保证（因此所有被积函数都是有界集合上的连续函数，从而可积）. 等式 (ii) 来自 $u_t(\cdot|z)$ 生成 $p_{t|Z}(\cdot|z)$ 这一事实以及 定理 2. 等式 (iii) 来自乘以并除以 $p_t(x)$（由假设严格为正），再利用 u_t 的公式 (4.12).

为验证 定理 2 中第二个、也是最后一个条件，我们将证明 u_t 可积且局部 Lipschitz 连续. 由于 C^1 函数局部 Lipschitz 连续，只需验证 $u_t(x)$ 对所有 (t, x) 都是 C^1 的. 这一点可由验证 $u_t(x|z)$ 和 $p_{t|Z}(x|z)$ 为 C^1 且 $p_t(x) > 0$ 得到，而这些均由假设保证. 此外，由于 $u_t(x|z)$ 条件可积，$u_t(x)$ 也可积：

$$\int_0^1 \int \|u_t(x)\| \, p_t(x)\mathrm{d}x\mathrm{d}t \le \int_0^1 \int \int \|u_t(x|z)\| \, p_{t|Z}(x|z)p_Z(z)dzdxdt < \infty, \tag{4.18}$$

其中第一个不等式来自向量形式的 Jensen 不等式. □

4.5 流匹配损失

已经证明目标速度场 u_t 生成从 p 到 q 的指定概率路径 p_t 后，还缺少一个易于计算的损失函数，用于学习尽可能接近目标 u_t 的速度场模型 u_t^θ. 直接给出这一损失函数的主要障碍之一，是目标 u_t 无法实际计算，因为它需要对整个训练集进行边缘化（即在 式 (4.8) 中对 x_1 积分，或在 式 (4.12) 中对 z 积分）. 幸运的是，一族称为**Bregman 散度**的损失函数提供了无偏梯度，仅凭条件速度 $u_t(x|z)$ 就能学习 $u_t^\theta(x)$.

Bregman 散度用下式度量两个向量 $u, v \in \mathbb{R}^d$ 之间的相异性：

$$D(u,v) \coloneqq \Phi(u) - [\Phi(v) + \langle u - v, \nabla\Phi(v)\rangle], \tag{4.19}$$

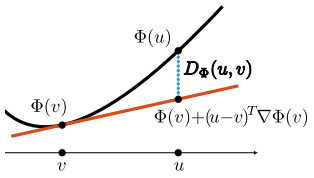


图 10 Bregman 散度.

其中 $\Phi : \mathbb{R}^d \to \mathbb{R}$ 是定义在某个凸集 $\Omega \subset \mathbb{R}^d$ 上的严格凸函数. 如 图 10 所示，Bregman 散度度量 $\Phi(u)$ 与在 v 附近展开、并在 u 处求值的 Φ 的线性近似之间的差. 由于线性近似是凸函数的全局下界，有 $D(u,v) \geq 0$. 此外，由于 Φ 严格凸，$D(u,v) = 0$ 当且仅当 $u = v$. 最基本的 Bregman 散度是平方欧氏距离 $D(u,v) = \|u - v\|^2$，它由选择 $\Phi(u) = \|u\|^2$ 得到. Bregman 散度适用于流匹配的关键性质，在于其关于第二个变量的梯度具有仿射不变性 (Holderrieth et al., 2024)：

$$\nabla_v D(au_1 + bu_2, v) = a\nabla_v D(u_1, v) + b\nabla_v D(u_2, v), \text{ 对任意} a + b = 1, \tag{4.20}$$

这可以由 式 (4.19) 验证. 仿射不变性允许如下交换期望与梯度：

$$\nabla_v D(\mathbb{E}[Y], v) = \mathbb{E}[\nabla_v D(Y, v)] \text{ 对任意随机变量} Y \in \mathbb{R}^d. \tag{4.21}$$

流匹配损失采用 Bregman 散度，沿概率路径 p_t 将可学习速度 $u_t^\theta(x)$ 回归到目标速度 $u_t(x)$：

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, X_t \sim p_t} D(u_t(X_t), u_t^\theta(X_t)), \tag{4.22}$$

其中时间 $t \sim U[0,1]$. 然而，如前所述，目标速度 u_t 无法实际计算，因此不能直接计算上述损失. 为此，考虑更简单且易于计算的**条件流匹配（Conditional Flow Matching，CFM）损失**：

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, Z, X_t \sim p_{t|Z}(\cdot|Z)} D(u_t(X_t|Z), u_t^\theta(X_t)). \tag{4.23}$$

这两个损失对于学习而言等价，因为它们的梯度相同 (Holderrieth et al., 2024)：

定理 4. *流匹配损失与条件流匹配损失的梯度相同：*

$$\nabla_\theta \mathcal{L}_{FM}(\theta) = \nabla_\theta \mathcal{L}_{CFM}(\theta). \tag{4.24}$$

特别地，条件流匹配损失的极小化解是边缘速度 $u_t(x)$.

证明. 证明由直接计算得到：

$$
\begin{aligned}
\nabla_\theta \mathcal{L}_{\mathrm{FM}}(\theta) &= \nabla_\theta \mathbb{E}_{t, X_t \sim p_t} D(u_t(X_t), u_t^\theta(X_t)) \\
&= \mathbb{E}_{t, X_t \sim p_t} \nabla_\theta D(u_t(X_t), u_t^\theta(X_t)) \\
&\overset{(i)}{=} \mathbb{E}_{t, X_t \sim p_t} \nabla_v D(u_t(X_t), u_t^\theta(X_t)) \nabla_\theta u_t^\theta(X_t) \\
&\overset{(4.12)}{=} \mathbb{E}_{t, X_t \sim p_t} \nabla_v D(\mathbb{E}_{Z \sim p_{Z|t}(\cdot|X_t)}[u_t(X_t|Z)], u_t^\theta(X_t)) \nabla_\theta u_t^\theta(X_t) \\
&\overset{(ii)}{=} \mathbb{E}_{t, X_t \sim p_t} \mathbb{E}_{Z \sim p_{Z|t}(\cdot|X_t)} \left[\nabla_v D(u_t(X_t|Z), u_t^\theta(X_t)) \nabla_\theta u_t^\theta(X_t) \right] \\
&\overset{(iii)}{=} \mathbb{E}_{t, X_t \sim p_t} \mathbb{E}_{Z \sim p_{Z|t}(\cdot|X_t)} [\nabla_\theta D(u_t(X_t|Z), u_t^\theta(X_t))] \\
&\overset{(iv)}{=} \nabla_\theta \mathbb{E}_{t, Z \sim q, X_t \sim p_{t|Z}(\cdot|Z)} [D(u_t(X_t|Z), u_t^\theta(X_t))] \\
&= \nabla_\theta \mathcal{L}_{\mathrm{CFM}}(\theta)
\end{aligned}
$$

其中（i）和（iii）使用链式法则；（ii）来自在给定 X_t 的条件下应用 式 (4.21)；（iv）使用贝叶斯公式. □

用于学习条件期望的 *Bregman* 散度. 定理 4 是下面将介绍的一个更一般结果的特例，该结果利用 Bregman 散度学习条件期望. 它将在本文各处使用，并为流匹配背后的所有可扩展损失提供基础：

命题 1 (用于学习条件期望的 Bregman 散度)**.** 令 $X \in \mathcal{S}_X, Y \in \mathcal{S}_Y$ 为状态空间 $\mathcal{S}_X, \mathcal{S}_Y$ 上的随机变量，并令 $g : \mathbb{R}^p \times \mathcal{S}_X \to \mathbb{R}^n$，$(\theta, x) \mapsto g^\theta(x)$，其中 $\theta \in \mathbb{R}^p$ 表示可学习参数. 令 $D_x(u, v)$，$x \in \mathcal{S}_X$，为包含 f 的像的凸集 $\Omega \subset \mathbb{R}^n$ 上的 *Bregman* 散度. 则

$$
\nabla_\theta \mathbb{E}_{X,Y} D_X \left(Y, g^\theta(X) \right) = \nabla_\theta \mathbb{E}_X D_X \left(\mathbb{E}[Y \mid X], g^\theta(X) \right). \tag{4.25}
$$

特别地，对于所有满足 $p_X(x) > 0$ 的 x，关于 θ 的全局极小解 $g^\theta(x)$ 满足

$$
g^\theta(x) = \mathbb{E}[Y \mid X = x]. \tag{4.26}
$$

证明. 假设 g^θ 关于 θ 可微，且 X、Y 的分布以及 D_x 和 g 允许交换微分与积分，展开得：

$$
\begin{aligned}
\nabla_\theta \mathbb{E}_{X,Y} D_X \left(Y, g^\theta(X) \right) &\overset{(i)}{=} \mathbb{E}_X \left[\mathbb{E} \left[\nabla_v D_X \left(Y, g^\theta(X) \right) \nabla_\theta g^\theta(X) \mid X \right] \right] \\
&\overset{(ii)}{=} \mathbb{E}_X \left[\nabla_v D_X \left(\mathbb{E}[Y \mid X], g^\theta(X) \right) \nabla_\theta g^\theta(X) \right] \\
&\overset{(iii)}{=} \mathbb{E}_X \left[\nabla_\theta D_X \left(\mathbb{E}[Y \mid X], g^\theta(X) \right) \right] \\
&= \nabla_\theta \mathbb{E}_X D_X \left(\mathbb{E}[Y \mid X], g^\theta(X) \right),
\end{aligned}
$$

其中（i）来自链式法则和期望的塔式性质 (3.11)．等式（ii）来自 (4.21)．等式（iii）再次使用链式法则．最后，对每个满足 $p_X(x) > 0$ 的 $x \in \mathcal{S}_X$，可以选择 $g^\theta(x) = \mathbb{E}[Y|X = x]$，从而得到 $\mathbb{E}_X D_X \left(\mathbb{E}[Y \mid X], g^\theta(X) \right) = 0$，这必定是关于 θ 的全局最小值. □

在 命题 1 中选择 $X = X_t$、$Y = u_t(X_t|Z)$、$g^\theta(x) = u_t^\theta(x)$，并对 $t \sim U[0, 1]$ 取期望，即可直接得到 定理 4.

一般时间分布　FM 损失的一种有用变体，是从均匀分布以外的分布采样时间 t. 具体而言，考虑 $t \sim \omega(t)$，其中 ω 是 $[0,1]$ 上的概率密度函数. 由此得到以下加权目标：

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\omega,Z,X_t} D(u_t(X_t|Z), u_t^\theta(X_t)) = \mathbb{E}_{t\sim U,Z,X_t} \omega(t) D(u_t(X_t|Z), u_t^\theta(X_t)). \tag{4.27}$$

尽管在数学上等价，在大规模图像生成任务中，采样 $t \sim \omega$ 比使用权重 $\omega(t)$ 能取得更好的性能 (Esser et al., 2024).

4.6　利用条件流求解条件生成问题

到目前为止，我们已将训练流模型 u_t^θ 的问题归结为：

1. 找到条件概率路径 $p_{t|Z}(x|z)$，使其产生的边缘概率路径 $p_t(x)$ 满足 (4.5) 中的边界条件.
2. 找到生成该条件概率路径的条件速度场 $u_t(x|z)$.
3. 使用条件流匹配损失训练（见 式 (4.23)）.

现在讨论完成前两步的一种具体选择，即如何设计这样的条件概率路径与速度场.

现在提出一种灵活的方法，通过**条件流**这一具体构造来设计上述条件概率路径和速度场. 思路如下：定义一个满足边界条件 (4.6) 的流模型 $X_{t|1}$（类似于 (3.16)），并通过微分 (3.20) 从 $X_{t|1}$ 提取速度场. 这一过程同时定义了 $p_{t|1}(x|x_1)$ 和 $u_t(x|x_1)$. 更具体地，定义**条件流模型**

$$X_{t|1} = \psi_t(X_0|x_1), \quad \text{其中} X_0 \sim \pi_{0|1}(\cdot\,|\,x_1), \tag{4.28}$$

其中 $\psi : [0,1) \times \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ 是如下定义的**条件流**：

$$\psi_t(x|x_1) = \begin{cases} x & t = 0 \\ x_1 & t = 1 \end{cases}, \tag{4.29}$$

它关于 (t,x) 光滑，且关于 x 是微分同胚.（这里的光滑指 $\psi_t(x|x_1)$ 关于 t 和 x 的所有阶导数均存在且连续，即 $C^\infty([0,1)\times\mathbb{R}^d, \mathbb{R}^d)$. 这些条件还可以放宽为 $C^2([0,1)\times\mathbb{R}^d, \mathbb{R}^d)$，但会牺牲论述的简洁性.）推前公式 (3.15) 将 $X_{t|1}$ 的概率密度定义为

$$p_{t|1}(x|x_1) := \left[\psi_t(\cdot|x_1)_\sharp \pi_{0|1}(\cdot|x_1)\right](x), \tag{4.30}$$

尽管在实际优化 CFM 损失时并不需要这一表达式，但在理论上可以用它证明 $p_{t|1}$ 满足两个边界条件 (4.6). 首先，根据 (4.29)，$\psi_0(\cdot|x_1)$ 是恒等映射，因此在时刻 $t = 0$ 保持 $\pi_{0|1}(\cdot|x_1)$ 不变. 其次，$\psi_1(\cdot|x_1) = x_1$ 是常值映射，当 $t \to 1$ 时将全部概率质量集中在 x_1. 此外，注意对于 $t \in [0,1)$，$\psi_t(\cdot|x_1)$ 是光滑微分同胚. 因此，由流与速度场的等价性（小节 3.4.1），存在唯一的光滑条件速度场（见 式 (3.20)），其形式为：

$$u_t(x|x_1) = \dot{\psi}_t(\psi_t^{-1}(x|x_1)|x_1). \tag{4.31}$$

综上，我们进一步将寻找条件路径及相应生成速度的任务，归结为构造满足 (4.29) 的条件流 $\psi_t(\cdot|x_1)$. 在 小节 4.7 中，我们将选择一个特别简单且具有若干理想性质的 $\psi_t(x|x_1)$（条件最优传输流），从而得到 节 1 中的标准流匹配算法. 在 小节 4.8 中，将讨论一族特定且广为人知的条件流，即仿射流，其中包含扩散模型文献中的一些已知例子. 在 节 5 中，将利用条件流定义流形上的流匹配，展示这种方法的灵活性.

4.6.1 重新考察条件流匹配损失

令 $Z = X_1$，并通过条件流定义条件概率路径和速度，重新考察 CFM 损失 (4.23)：

$$\begin{aligned}\mathcal{L}_{\mathrm{CFM}}(\theta) &= \mathbb{E}_{t,X_1,X_t\sim p_t(\cdot|X_1)} D\left(u_t(X_t|X_1), u_t^\theta(X_t)\right) \\ &\overset{(3.4)}{=} \mathbb{E}_{t,(X_0,X_1)\sim\pi_{0,1}} D\left(\dot{\psi}_t(X_0|X_1), u_t^\theta(X_t)\right)\end{aligned} \tag{4.32}$$

其中第二个等式使用了无意识统计学家法则，取 $X_t = \psi_t(X_0|X_1)$，以及

$$u_t(X_t|X_1) \overset{(4.31)}{=} \dot{\psi}_t\left(\psi_t^{-1}\left(\psi_t\left(X_0|X_1\right)|X_1\right)\middle|X_1\right) = \dot{\psi}_t(X_0|X_1). \tag{4.33}$$

根据 命题 1，损失 (4.32) 的极小化解具有 (Liu et al., 2022) 中的形式：

$$u_t(x) = \mathbb{E}\left[\dot{\psi}_t(X_0|X_1)\middle|X_t = x\right]. \tag{4.34}$$

在 `flow_matching` 程序库 中，`ProbPath` 对象定义概率路径. 可以在（t、X_0、X_1）处对这条概率路径采样，得到 X_t 和 $\dot{\psi}_t(X_0|X_1)$. 随后即可计算 CFM 损失 $\mathcal{L}_{\mathrm{CFM}}(\theta)$ 的蒙特卡洛估计. 小节 4.6.1 给出了使用 CFM 目标的训练循环示例.

代码 4: 使用条件流匹配（CFM）损失训练

```python
import torch
from flow_matching.path import ProbPath
from flow_matching.path.path_sample import PathSample

path: ProbPath = ...  # flow_matching 库实现了最常用的概率路径
velocity_model: torch.nn.Module = ...  # 初始化速度模型
optimizer = torch.optim.Adam(velocity_model.parameters())

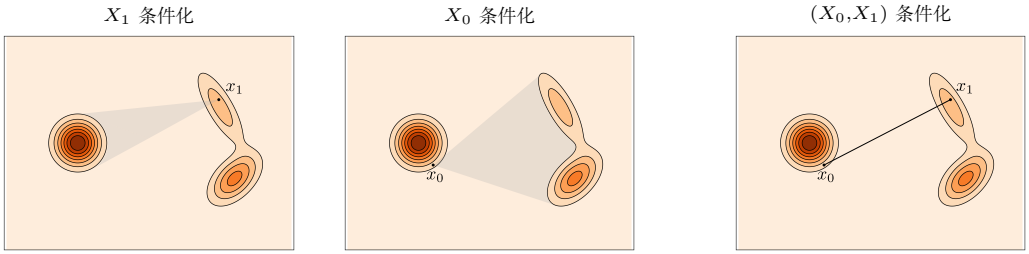
for x_0, x_1 in dataloader:  # 来自 π_{0,1} 的样本，形状为 [batch_size, *data_dim]
    t = torch.rand(batch_size)  # 随机采样时间 t ~ U[0,1]
    sample: PathSample = path.sample(t=t, x_0=x_0, x_1=x_1)
    x_t = sample.x_t
    dx_t = sample.dx_t  # dX_t 是 ψ̇_t(X_0|X_1).
    # 若 D 为欧氏距离，CFM 目标对应于均方误差
    cfm_loss = torch.pow(velocity_model(x_t, t) - dx_t, 2).mean()  # 蒙特卡洛估计
    optimizer.zero_grad()
    cfm_loss.backward()
    optimizer.step()
```

4.6.2 由条件流构造的概率路径的边缘化技巧

接下来介绍适用于由条件流构造的概率路径的边缘化技巧. 为此，注意若 $\pi_{0|1}(\cdot|x_1)$ 为 C^1，则由构造可知 $p_t(x|x_1)$ 也为 C^1；此外，若

$$\mathbb{E}_{t,(X_0,X_1)\sim\pi_{0,1}}\left\|\dot{\psi}_t(X_0|X_1)\right\| < \infty. \tag{4.35}$$

则 $u_t(x|x_1)$ 条件可积. 因此，令 $Z = X_1$，即可得到 定理 3 的以下推论.



$$
\begin{array}{rccccc}
 & \psi_t(X_0|x_1) \sim p_{t|1}(\cdot|x_1) & & \psi_t(X_1|x_0) \sim p_{t|0}(\cdot|x_0) & & \psi_t(x_0,x_1) \sim p_{t|0,1}(\cdot|x_0,x_1) \\
p_t(x) = & \int p_{t|1}(x|x_1) q(x_1) \mathrm{d}x_1 & = & \int p_{t|0}(x|x_0) p(x_0) \mathrm{d}x_1 & = & \int p_{t|0,1}(x|x_0,x_1) \pi_{0,1}(x_0,x_1) \mathrm{d}x_0 \mathrm{d}x_1 \\
u_t(x) = & \mathbb{E}\left[u_t(X_t|X_1)|X_t = x\right] & = & \mathbb{E}\left[u_t(X_t|X_0)|X_t = x\right] & = & \mathbb{E}\left[u_t(X_t|X_0,X_1)|X_t = x\right]
\end{array}
$$

图 11 流匹配中不同形式的条件化，以及利用相应条件流进行的路径设计. 当条件流是微分同胚时，所有构造等价. 当条件流不是微分同胚时，需要额外条件来保证边缘速度生成边缘路径；更多细节见正文.

推论 1. 假设 q 具有有界支撑，$\pi_{0|1}(\cdot|x_1)$ 属于 $C^1(\mathbb{R}^d)$ 且对某个满足 $q(x_1) > 0$ 的 x_1 严格为正，$\psi_t(x|x_1)$ 是满足 (4.29) 和 (4.35) 的条件流. 则分别由 (4.30) 和 (4.31) 定义的 $p_{t|1}(x|x_1)$ 与 $u_t(x|x_1)$，定义了边缘速度场 $u_t(x)$，它生成在 p 和 q 之间插值的边缘概率路径 $p_t(x)$.

证明. 若对某个满足 $q(x_1) > 0$ 的 $x_1 \in \mathbb{R}^d$ 有 $\pi_{0|1}(\cdot|x_1) > 0$，则对所有 $x \in \mathbb{R}^d$ 都有 $p_{t|1}(x|x_1) > 0$，并且它属于 $C^1([0,1) \times \mathbb{R}^d)$（定义见 (4.30) 和 (3.15)）. 此外，$u_t(x|x_1)$（定义于 (4.31)）光滑，并满足

$$
\begin{aligned}
\int_0^1 \int \|u_t(x|x_1)\| \, p_{t|1}(x|x_1) q(x_1) \mathrm{d}x_1 \mathrm{d}x \mathrm{d}t &= \mathbb{E}_{t, X_1 \sim q, X_t \sim p_{t|1}(\cdot|X_1)} \|u_t(X_t|X_1)\| \\
&\overset{(3.4)}{=} \mathbb{E}_{t, X_1 \sim q, X_0 \sim \pi_{0|1}(\cdot|X_1)} \|u_t(\psi_t(X_0|X_1)|X_1)\| \\
&\overset{(4.33)}{=} \mathbb{E}_{t, (X_0,X_1) \sim \pi_{0,1}} \left\|\dot{\psi}_t(X_0|X_1)\right\| \\
&< \infty.
\end{aligned}
$$

因此，$u_t(x|x_1)$ 条件可积（见 (4.13)）. 由 定理 3，边缘速度 u_t 生成 p_t. 由于 (4.30) 定义的 $p_{t|1}(x|x_1)$ 满足 (4.6)，可知 p_t 在 p 与 q 之间插值. □

这一结果将用作工具，证明条件流的特定选择产生的边缘速度 $u_t(x)$ 能够生成边缘概率路径 $p_t(x)$.

4.6.3 采用其他条件的条件流

可以选择不同的条件 Z，但它们本质上均等价. 如 图 11 所示，主要选择包括固定目标样本 $Z = X_1$ (Lipman et al., 2022)、固定源样本 $Z = X_0$ (Esser et al., 2024)，或采用双侧条件 $Z = (X_0, X_1)$ (Albergo and Vanden-Eijnden, 2022; Liu et al., 2022; Pooladian et al., 2023; Tong et al., 2023).

下面重点考虑双侧条件 $Z = (X_0, X_1)$. 按照上述 FM 蓝图，现在希望构造条件概率路径 $p_{t|0,1}(x|x_0,x_1)$ 及相应生成速度 $u_t(x|x_0,x_1)$，使得

$$
p_{0|0,1}(x|x_0,x_1) = \delta_{x_0}(x), \text{ 且} p_{1|0,1}(x|x_0,x_1) = \delta_{x_1}(x). \tag{4.36}
$$

这里的讨论将保持形式化，因为它需要使用狄拉克函数 δ，而迄今的推导仅处理概率密度（并非一般分布）. 为构造这样的路径，可以考虑一个**插值函数** (Albergo and Vanden-Eijnden, 2022)，定义为 $X_{t|0,1} = \psi_t(x_0, x_1)$，其中函数 $\psi : [0,1] \times \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ 满足类似于 (4.29) 的条件：

$$\psi_t(x_0, x_1) = \begin{cases} x_0 & t = 0 \\ x_1 & t = 1. \end{cases} \tag{4.37}$$

因此，$\psi_t(\cdot, x_1)$ 将 $\delta_{x_0}(x)$ 推前至 $\delta_{x_1}(x)$. 类似于前面的做法，定义条件概率路径为

$$p_{t|0,1}(\cdot|x_0, x_1) \coloneqq \psi_t(\cdot, x_1)_\sharp \delta_{x_0}(\cdot) \tag{4.38}$$

它满足 (4.36) 中的边界约束. Albergo and Vanden-Eijnden (2022) 中的**随机插值**定义为

$$X_t = \psi_t(X_0, X_1) \sim p_t(\cdot) = \int p_{t|0,1}(\cdot|x_0, x_1)\pi_{0,1}(x_0, x_1)\mathrm{d}x_0\mathrm{d}x_1. \tag{4.39}$$

接下来，该路径上的条件速度同样可以通过 (3.20) 计算，得到

$$u_t(x|x_0, x_1) = \dot{\psi}_t(x_0, x_1) \tag{4.40}$$

该式仅在 $x = \psi_t(x_0, x_1)$ 时定义. 暂时忽略额外条件，定理 3 似乎表明，生成 $p_t(x)$ 的边缘速度为

$$\begin{aligned} u_t(x) &= \mathbb{E}\left[u_t(X_t|X_0, X_1) \,|\, X_t = x\right] \\ &= \mathbb{E}\left[\dot{\psi}_t(X_0, X_1) \,|\, X_t = x\right], \end{aligned}$$

这得到了与以 X_1 为条件的情形 (4.34) 相同的边缘公式，但条件流 $\psi_t(x_0, x_1)$ 的要求似乎更宽松：现在只要求它是插值函数，从而放宽了较严格的微分同胚条件. 然而，更仔细地考察会发现，要使 $u_t(x)$ 成为 $p_t(x)$ 的生成速度，仍需一些额外条件；简单插值（如 (4.37) 所定义）不足以保证这一点，即使再加上 定理 3 所要求的光滑性条件也不够. 为说明这一点，考虑

$$\psi_t(x_0, x_1) = (1 - 2t)_+^\tau \, x_0 + (2t - 1)_+^\tau \, x_1, \text{ 其中}(s)_+ = \mathrm{ReLU}(s), \tau > 2,$$

这是一个关于时间的 $C^2([0,1])$ 插值函数，对所有 x_0, x_1，它在时刻 $t = 0.5$ 将全部概率质量集中在位置 0，即 $\mathbb{P}(X_{\frac{1}{2}} = 0) = 1$. 因此，假设 $u_t(x)$ 确实生成 $p_t(x)$，它在 $t = \frac{1}{2}$ 的边缘分布就是 δ_0. 由于流既具有马尔可夫性（如 (3.17) 所示）又是确定性的，对于所有 $t > 0.5$，其边缘分布都必须是狄拉克函数. 这导致矛盾，因为 $X_1 = \psi_1(X_0, X_1) \sim q$ 一般不是狄拉克函数. Albergo and Vanden-Eijnden (2022) 和 Liu et al. (2022) 给出了一些保证 $u_t(x)$ 确实生成 $p_t(x)$ 的额外条件，但与 定理 3 的条件相比，它们较难验证. 下面将展示如何实际检验 定理 3 的条件，从而确认感兴趣的特定路径确实由相应边缘速度生成.

不过，若 $\psi_t(x_0, x_1)$ 在固定 x_1 时关于 x_0 也是微分同胚，并在固定 x_0 时关于 x_1 也是微分同胚，则这三种构造产生相同的边缘速度（由 (4.34) 定义）以及相同的边缘概率路径 p_t（由 $X_t = \psi_t(X_0, X_1) = \psi_t(X_0|X_1) = \psi_t(X_1|X_0)$ 定义），见图 11.

4.7 最优传输与线性条件流

现在的问题是：如何找到有用的条件流 $\psi_t(x|x_1)$？一种方法是将其选为某个自然代价泛函的极小化解，最好还能具有一些理想性质. 这类代价泛函的一个常见例子，是具有二次代价的动态最优

传输问题 (Villani et al., 2009; Villani, 2021; Peyré et al., 2019)，其形式为

$$(p_t^\star, u_t^\star) = \arg\min_{p_t, u_t} \int_0^1 \int \|u_t(x)\|^2 p_t(x) \mathrm{d}x \mathrm{d}t \qquad \text{（动能）} \tag{4.41a}$$

$$\text{约束为} p_0 = p, p_1 = q \qquad \text{（插值）} \tag{4.41b}$$

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t + \operatorname{div}(p_t u_t) = 0. \qquad \text{（连续性方程）} \tag{4.41c}$$

上述 $(p_t^\star, u_t^\star)$（通过 式 (3.19)）定义了如下形式的流：

$$\psi_t^\star(x) = t\phi(x) + (1-t)x, \tag{4.42}$$

它称为**OT 位移插值** (McCann, 1997)，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^d$ 是最优传输映射. 通过定义随机变量

$$X_t = \psi_t^\star(X_0) \sim p_t^\star \quad \text{当} \quad X_0 \sim p. \tag{4.43}$$

OT 位移插值也求解了流匹配问题 (4.1). 最优传输的表述促使样本沿直线轨迹运动：

$$X_t = \psi_t^\star(X_0) = X_0 + t(\phi(X_0) - X_0),$$

其速度 $\phi(X_0) - X_0$ 恒定，通常更易于用常微分方程求解器采样；特别地，这里的目标样本 X_1 可以用欧拉法 (3.21) 的单个步骤精确求出.

现在可以尝试将边缘速度公式（式 (4.34)）代入最优传输问题 (4.41)，寻找最优的 $\psi_t(x|x_1)$. 虽然这似乎颇具挑战，但可以转而寻找动能的一个上界，而这个上界的极小化解很容易得到 (Liu et al., 2022)：

$$\int_0^1 \mathbb{E}_{X_t \sim p_t} \|u_t(X_t)\|^2 \mathrm{d}t = \int_0^1 \mathbb{E}_{X_t \sim p_t} \left\| \mathbb{E}\left[\dot{\psi}_t(X_0|X_1) \,\middle|\, X_t \right] \right\|^2 \mathrm{d}t \tag{4.44}$$

$$\overset{(i)}{\leq} \int_0^1 \mathbb{E}_{X_t \sim p_t} \mathbb{E}\left[\left\| \dot{\psi}_t(X_0|X_1) \right\|^2 \,\middle|\, X_t \right] \mathrm{d}t \tag{4.45}$$

$$\overset{(ii)}{=} \mathbb{E}_{(X_0, X_1) \sim \pi_{0,1}} \int_0^1 \left\| \dot{\psi}_t(X_0|X_1) \right\|^2 \mathrm{d}t, \tag{4.46}$$

其中 (i) 使用 Jensen 不等式，(ii) 使用条件期望的塔式性质（见 式 (3.11)），并交换了对 t 的积分与期望. 现在可以对每个 (X_0, X_1) 分别最小化 (4.46) 中的被积项，这便得到关于 $\gamma_t = \psi_t(x|x_1)$ 的以下变分问题：

$$\min_{\gamma:[0,1]\to\mathbb{R}^d} \quad \int_0^1 \|\dot{\gamma}_t\|^2 \mathrm{d}t \tag{4.47a}$$

$$\text{约束为} \quad \gamma_0 = x, \gamma_1 = x_1. \tag{4.47b}$$

可以利用欧拉–Lagrange 方程 (Gelfand et al., 2000) 求解该问题；在这里，该方程的形式为 $\frac{d^2}{dt^2}\gamma_t = 0$. 结合边界条件，得到极小化解：

$$\psi_t(x|x_1) = tx_1 + (1-t)x. \tag{4.48}$$

注意，尽管没有显式施加这一约束，这一 $\psi_t(x|x_1)$ 在 $t \in [0, 1)$ 时关于 x 是微分同胚，并关于 t, x 光滑，符合条件流的要求.

可以得到以下结论：

1. 线性条件流在所有条件流中最小化动能的一个上界.
2. 当目标 q 由单个数据点 $q(x) = \delta_{x_1}(\cdot)$ 构成时，(4.48) 中的线性条件流就是最优传输 (Lipman et al., 2022). 事实上，此时 $X_t = \psi_t(X_0|x_1) \sim p_t$，而 $X_0 = \psi^{-1}(X_t|x_1)$ 是 X_t 的函数，因此 $\mathbb{E}\left[\dot{\psi}_t(X_0|x_1) \,\middle|\, X_t\right] = \dot{\psi}_t(X_0|x_1)$，从而（ii）成为等式.

   定理 5. *若 $q = \delta_{x_1}$，则动态 OT 问题 (4.41) 有解析解，由 (4.48) 中的 OT 位移插值给出.*

3. 将线性条件流代入 (4.46)，得到

$$\int_0^1 \mathbb{E}_{X_t \sim p_t} \|u_t(X_t)\|^2 \,\mathrm{d}t \le \mathbb{E}_{(X_0, X_1) \sim \pi_{0,1}} \int_0^1 \|X_1 - X_0\|^2 \,\mathrm{d}t \tag{4.49}$$

   这说明边缘速度 $u_t(x)$ 的动能不大于原始耦合 $\pi_{0,1}$ 的动能 (Liu et al., 2022).

(4.48) 中的条件流特别地是仿射的，这促使我们研究仿射条件流族，下面将对此展开讨论.

4.8 仿射条件流

上一节发现，线性（条件 OT）流在所有条件流中最小化了动能的一个上界. 线性条件流是更广泛的仿射条件流族的特例，本节将研究这一流族：

$$\psi_t(x|x_1) = \alpha_t x_1 + \sigma_t x, \tag{4.50}$$

其中 $\alpha_t, \sigma_t : [0,1] \to [0,1]$ 是满足以下条件的光滑函数：

$$\alpha_0 = 0 = \sigma_1,\ \alpha_1 = 1 = \sigma_0,\ \text{且} \dot{\alpha}_t, -\dot{\sigma}_t > 0\ \text{对} t \in (0,1). \tag{4.51}$$

称函数对 (α_t, σ_t) 为调度器. 上述导数条件保证 α_t 严格单调递增，而 σ_t 严格单调递减. 对每个 $t \in [0,1)$，条件流 (4.50) 都是关于 x 的简单仿射映射，满足条件 (4.29). 相应的边缘速度场 (4.34) 为

$$u_t(x) = \mathbb{E}\left[\dot{\alpha}_t X_1 + \dot{\sigma}_t X_0 | X_t = x\right]. \tag{4.52}$$

借助 推论 1 可以证明：若采用独立耦合，以及具有有限二阶矩、光滑且严格为正的源密度 p（例如高斯分布 $p = \mathcal{N}(\cdot|0, I)$），则 u_t 生成在 p 与 q 之间插值的概率路径 p_t. 这一结果对于流匹配应用意义重大，将其正式陈述为以下定理.

定理 6. *假设 q 具有有界支撑，p 属于 $C^1(\mathbb{R}^d)$，其密度严格为正且具有有限二阶矩，并且两者通过独立耦合 $\pi_{0,1}(x_0, x_1) = p(x_0)q(x_1)$ 联系起来. 令 $p_t(x) = \int p_{t|1}(x|x_1)q(x_1)\mathrm{d}x_1$ 由式 (4.30) 定义，其中 ψ_t 由 式 (4.50) 定义. 则边缘速度 (4.52) 生成在 p 与 q 之间插值的 p_t.*

证明. 应用 推论 1. 首先，由假设，$\pi_{0|1}(\cdot|x_1) = p(\cdot)$ 为 C^1，且处处为正. 其次，(4.50) 定义的

ψ_t 满足 (4.29). 最后，只需检验 (4.35)：

$$\begin{aligned}\mathbb{E}_{t,(X_0,X_1)}\left\|\dot{\psi}(X_0|X_1)\right\| &= \mathbb{E}_{t,(X_0,X_1)}\left\|\dot{\alpha}_t X_1 + \dot{\sigma}_t X_0\right\| \\ &\le \mathbb{E}_t|\dot{\alpha}_t|\,\mathbb{E}_{X_1}\|X_1\| + \mathbb{E}_t|\dot{\sigma}_t|\,\mathbb{E}_{X_0}\|X_0\| \\ &= \mathbb{E}_{X_1}\|X_1\| + \mathbb{E}_{X_0}\|X_0\| \\ &< \infty,\end{aligned}$$

其中最后一个不等式来自 $X_1 \sim q$ 具有有界支撑、$X_0 \sim p$ 具有有界二阶矩这两个事实. □

在这一仿射情形下，CFM 损失 (4.32) 具有如下形式：

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,(X_0,X_1)\sim\pi_{0,1}} D(\dot{\alpha}_t X_1 + \dot{\sigma}_t X_0, u_t^\theta(X_t))\,. \tag{4.53}$$

代码 5: flow_matching 程序库 中的仿射概率路径示例

```python
from flow_matching.path import AffineProbPath, CondOTPath
from flow_matching.path.scheduler import (
    CondOTScheduler, PolynomialConvexScheduler, LinearVPScheduler, CosineScheduler)

# 条件最优传输调度，α_t = t，σ_t = 1 − t
path = AffineProbPath(scheduler=CondOTScheduler())
path = CondOTPath()  # 使用上述 CondOTScheduler 的仿射路径的简写

# 多项式调度，α_t = t^n，σ_t = 1 − t^n
path = AffineProbPath(scheduler=PolynomialConvexScheduler(n=1.0))

# 线性方差保持调度，α_t = t，σ_t = √(1 − t²)
path = AffineProbPath(scheduler=LinearVPScheduler())

# 余弦调度，α_t = sin(0.5tπ)，σ_t = cos(0.5tπ)
path = AffineProbPath(scheduler=CosineScheduler())
```

4.8.1 速度参数化

在仿射情形下，边缘速度场 u_t 有多种参数化，每一种都可以通过 小节 4.5 中介绍的流匹配损失进行学习. 为推导这些参数化，将仿射路径的等价表述

$$X_t = \alpha_t X_1 + \sigma_t X_0 \quad\Leftrightarrow\quad X_1 = \frac{X_t - \sigma_t X_0}{\alpha_t} \quad\Leftrightarrow\quad X_0 = \frac{X_t - \alpha_t X_1}{\sigma_t}, \tag{4.54}$$

代入边缘速度公式 (4.52)，得到

$$u_t(x) = \dot{\alpha}_t \mathbb{E}\left[X_1|X_t = x\right] + \dot{\sigma}_t \mathbb{E}\left[X_0|X_t = x\right] \tag{4.55}$$

$$= \frac{\dot{\sigma}_t}{\sigma_t} x + \left[\dot{\alpha}_t - \alpha_t \frac{\dot{\sigma}_t}{\sigma_t}\right] \mathbb{E}\left[X_1|X_t = x\right] \tag{4.56}$$

$$= \frac{\dot{\alpha}_t}{\alpha_t} x + \left[\dot{\sigma}_t - \sigma_t \frac{\dot{\alpha}_t}{\alpha_t}\right] \mathbb{E}\left[X_0|X_t = x\right], \tag{4.57}$$

其中使用了 $\mathbb{E}[Z|Z=z]=z$ 这一事实. 随后，记以下确定性函数为：

$$x_{1|t}(x)=\mathbb{E}[X_1|X_t=x]\text{，称为}x_1\text{ 预测（目标）}, \tag{4.58}$$

$$x_{0|t}(x)=\mathbb{E}[X_0|X_t=x]\text{，称为}x_0\text{ 预测（源）}. \tag{4.59}$$

由此又得到两种参数化 u_t 的方式：通过 x_1 预测 $x_{1|t}$ 的 (4.56)，以及通过 x_0 预测 $x_{0|t}$ 的 (4.57). 表 1 给出了这些参数化之间的转换公式. 这些参数化也可以利用类似于 (4.23) 的条件匹配损失进行学习. 特别地，任意函数

$$g_t(x):=\mathbb{E}[f_t(X_0,X_1)|X_t=x], \tag{4.60}$$

都可以通过最小化如下形式的匹配损失进行学习，其中 $f_t(X_0,X_1)$ 是定义为 X_0 和 X_1 的时间依赖函数的随机变量：

$$\mathcal{L}_{\mathrm{M}}(\theta)=\mathbb{E}_{t,X_t\sim p_t}D\left(g_t(X_t),g_t^\theta(X_t)\right). \tag{4.61}$$

该损失与以下条件匹配损失具有相同的梯度：

$$\mathcal{L}_{\mathrm{CM}}(\theta)=\mathbb{E}_{t,(X_0,X_1)\sim\pi_{0,1}}D\left(f_t(X_0,X_1),g_t^\theta(X_t)\right). \tag{4.62}$$

为学习 $x_{1|t}$，条件匹配损失采用 $f_t(x_0,x_1)=x_1$，对于 $x_{0|t}$ 也类似. 这一过程由 定理 7 保证合理；它是在 命题 1 中令 $X=X_t$、$Y=f_t(X_0,X_1)$，并对 $t\sim U[0,1]$ 积分所直接得到的结果.

定理 7. 对 X_0,X_1 的任意函数 $f_t(X_0,X_1)$，匹配损失与条件匹配损失的梯度相同：

$$\nabla_\theta\mathcal{L}_M(\theta)=\nabla_\theta\mathcal{L}_{CM}(\theta). \tag{4.63}$$

特别地，条件匹配损失的极小化解是条件期望

$$g_t^\theta(x)=\mathbb{E}[f_t(X_0,X_1)|X_t=x]. \tag{4.64}$$

小节 4.8.1 展示了如何利用 `flow_matching` 程序库 训练 x_1 预测模型.

代码 6: 使用条件匹配（CM）目标训练 X_1 预测模型

```python
import torch
from flow_matching.path import AffineProbPath
from flow_matching.solver import ODESolver
from flow_matching.utils import ModelWrapper

path: AffineProbPath = ...
denoiser_model: torch.nn.Module = ...  # 初始化去噪器
optimizer = torch.optim.Adam(velocity_model.parameters())

for x_0, x_1 in dataloader:  # 来自 π_{0,1} 的样本，形状为 [batch_size, *data_dim]
    t = torch.rand(batch_size)  # 随机采样时间 t ~ U[0,1]
    sample = path.sample(t=t, x_0=x_0, x_1=x_1)  # 对条件路径采样
    cm_loss = torch.pow(model(sample.x_t, t) - sample.x_1, 2).mean()  # CM 损失
    optimizer.zero_grad()
    cm_loss.backward()
```

```
        optimizer.step()

    # 从去噪器转换为速度预测
    class VelocityModel(ModelWrapper):
        def __init__(self, denoiser: nn.Module, path: AffineProbPath):
            super().__init__(model=denoiser)
            self.path=path

        def forward(self, x: torch.Tensor, t: torch.Tensor, **extras) -> torch.Tensor:
            x_1_prediction = super().forward(x, t, **extras)
            return self.path.target_to_velocity(x_1=x_1_prediction, x_t=x, t=t)

    # 采样 X_1
    velocity_model = VelocityModel(denoiser=denoiser_model, path=path)
    x_0 = torch.randn(batch_size, *data_dim)  # 指定初始条件
    solver = ODESolver(velocity_model=velocity_model)
    num_steps = 100
    x_1 = solver.sample(x_init=x_0, method='midpoint', step_size=1.0 / num_steps)
```

速度参数化中的奇点. 表面上看，(4.56) 的系数在 $t \to 1$ 时会发散，而 (4.57) 的系数在 $t \to 0$ 时也类似. 若 $\mathbb{E}[X_1|X_0 = x]$ 和 $\mathbb{E}[X_0|X_1 = x]$ 存在，而 $p(x) > 0$ 和 $q(x) > 0$ 时确实如此，则这些奇点在理论上并非本质奇点，意味着 $x_{1|t}$ 和 $x_{0|t}$ 中的奇异性会与参数化系数中的奇异性相消. 不过，在实践中这些奇点仍可能造成问题，因为可学习的 $x^\theta_{1|t}$ 和 $x^\theta_{0|t}$ 在构造上连续，因此不能完美回归到目标 $x_{1|t}$ 和 $x_{0|t}$. 为理解如何处理这些潜在问题，回顾 (4.55)，考虑 $t \to 0$ 时的 $u_0(x) = \dot\alpha_0 \mathbb{E}[X_1|X_0 = x] + \dot\sigma_0 x$，以及 $t \to 1$ 时的 $u_1(x) = \dot\alpha_1 x + \dot\sigma_1 \mathbb{E}[X_0|X_1 = x]$. 在许多感兴趣的情形下，这些量可以计算. 回到例子 $\pi_{0,1}(x_0, x_1) = \mathcal{N}(x_0|0, I)q(x_1)$，并假设 $\mathbb{E}_{X_1} X_1 = 0$，则有 $u_0(x) = \dot\sigma_0 x$ 和 $u_1(x) = \dot\alpha_1 x$. 当 $t \to 1$ 或 $t \to 0$ 时，分别从 $x_{1|t}$ 或 $x_{0|t}$ 转换到 $u_t(x)$，可以利用这些表达式处理奇点.

4.8.2 训练后更换速度调度器

仿射条件流允许一种闭式变换：将基于调度器 (α_t, σ_t) 和任意数据耦合 $\pi_{0,1}$ 的边缘速度场 $u_t(x)$，变换为基于另一调度器 $(\bar\alpha_r, \bar\sigma_r)$ 但采用相同数据耦合 $\pi_{0,1}$ 的边缘速度场 $\bar u_r(x)$. 这种变换可以将训练好的速度场适配到不同的调度器，从而可能提高采样效率与生成质量 (Karras et al., 2022; Shaul et al., 2023b; Pokle et al., 2023). 为此，定义两个条件流之间的尺度--时间 (ST) 变换 (s_r, t_r)：

$$\bar\psi_r(x_0|x_1) = s_r \psi_{t_r}(x_0|x_1), \tag{4.65}$$

其中 $\psi_t(x_0|x_1) = \alpha_t x_1 + \sigma_t x_0$，$\bar\psi_r(x_0|x_1) = \bar\alpha_r x_1 + \bar\sigma_r x_0$，$s, t : [0, 1] \to \mathbb{R}_{\geq 0}$ 是时间与尺度的重参数化. 求解 (4.65) 得到

$$\begin{aligned} t_r &= \rho^{-1}(\bar\rho(r)) \\ s_r &= \bar\sigma_r / \sigma_{t_r}, \end{aligned} \tag{4.66}$$

其中定义信噪比为

$$\begin{aligned} \rho(t) &= \frac{\alpha_t}{\sigma_t} \\ \bar\rho(t) &= \frac{\bar\alpha_t}{\bar\sigma_t}, \end{aligned} \tag{4.67}$$

并假设它是可逆函数. 新调度器 $(\bar{\alpha}_r, \bar{\sigma}_r)$ 对应的边缘速度 $\bar{u}_r(x)$ 满足

$$\begin{aligned}\bar{u}_r(x) &= \mathbb{E}\left[\dot{\bar{X}}_r \middle| \bar{X}_r = x\right] \\ &\overset{(4.65)}{=} \mathbb{E}\left[\dot{s}_r X_{t_r} + s_r \dot{X}_{t_r} \dot{t}_r \middle| s_r X_{t_r} = x\right] \\ &= \dot{s}_r \mathbb{E}\left[X_{t_r} \middle| X_{t_r} = \frac{x}{s_r}\right] + s_r \dot{t}_r \mathbb{E}\left[\dot{X}_{t_r} \middle| X_{t_r} = \frac{x}{s_r}\right] \\ &= \frac{\dot{s}_r}{s_r} x + s_r \dot{t}_r u_{t_r}\left(\frac{x}{s_r}\right),\end{aligned}$$

其中，与前面一样，$\bar{X}_r = \bar{\psi}_r(X_0|X_1)$，$X_t = \psi_t(X_0|X_1)$. 最后一项可用于在训练后更换调度器. 小节 4.8.2 展示了如何使用 `flow_matching` 程序库，将通过方差保持调度训练的速度场更换为条件最优传输调度.

调度器的等价性. 上式另一个重要推论是：所有调度器在理论上都在时刻 $t = 1$ 产生相同的采样结果 (Shaul et al., 2023a)，即

$$\bar{\psi}_1(x_0) = \psi_1(x_0), \text{ 对所有} x_0 \in \mathbb{R}^d. \tag{4.68}$$

为说明这一点，记 $\bar{\psi}_r(x)$ 为 $\bar{u}_t(x)$ 定义的流，对 $\tilde{\psi}_r(x) \coloneqq s_r \psi_{t_r}(x)$ 关于 r 求导，并注意它也满足

$$\frac{\mathrm{d}}{\mathrm{d}t}\tilde{\psi}_r(x) = \bar{u}_r(\tilde{\psi}_r(x)). \tag{4.69}$$

因此，由常微分方程解的唯一性，有 $\bar{\psi}_r(x) = \tilde{\psi}_r(x) = s_r \psi_{t_r}(x)$. 现在，为避免处理无穷大的信噪比，假设对于任意 $\epsilon > 0$，调度器满足 $\sigma_1 = \epsilon = \bar{\sigma}_1$ (除 (4.51) 外)；那么在 $r = 1$ 时，有 $t_1 = 1$ 和 $s_1 = 1$，因而 式 (4.68) 成立.

代码 7: 训练后更换调度器

```python
import torch
from flow_matching.path import AffineProbPath
from flow_matching.path.scheduler import ScheduleTransformedModel, CondOTScheduler,
↪ VPScheduler
from flow_matching.solver import ODESolver
from flow_matching.utils import ModelWrapper

training_scheduler = VPScheduler()  # 方差保持调度
path = AffineProbPath(scheduler=training_scheduler)
velocity_model: ModelWrapper = ...  # 使用方差保持调度训练速度模型

# 将调度器从方差保持调度更换为条件 OT 调度
sampling_scheduler = CondOTScheduler()
transformed_model = ScheduleTransformedModel(
    velocity_model=velocity_model,
    original_scheduler=training_scheduler,
    new_scheduler=sampling_scheduler
)

# 使用条件 OT 调度对变换后的模型采样
solver = ODESolver(velocity_model=transformed_model)
x_0 = torch.randn(batch_size, *data_dim)  # 指定初始条件
```

```
solver = ODESolver(velocity_model=velocity_model)
num_steps = 100
x_1 = solver.sample(x_init=x_0, method='midpoint', step_size=1.0 / num_steps)
```

4.8.3 高斯路径

在本文撰写时，最常用的仿射概率路径类由独立耦合 $\pi_{0,1}(x_0, x_1) = p(x_0)q(x_1)$ 和高斯源分布 $p(x) = \mathcal{N}(x|0, \sigma^2 I)$ 构造. 由于高斯分布在仿射变换下保持高斯形式，得到的条件概率路径为

$$p_{t|1}(x|x_1) = \mathcal{N}(x|\alpha_t x_1, \sigma_t^2 I). \tag{4.70}$$

这一情形包含标准扩散模型生成的概率路径（尽管扩散的生成过程是随机的、遵循随机微分方程，但其边缘概率相同）. 两个例子是**方差保持（Variance Preserving，VP）**路径与**方差爆炸（Variance Exploding，VE）**路径 (Song et al., 2021)，它们通过选择以下调度器定义：

$$\alpha_t \equiv 1, \sigma_0 \gg 1, \sigma_1 = 0; \tag{VP}$$

$$\alpha_t = e^{-\frac{1}{2}\beta_t}, \sigma_t = \sqrt{1 - e^{-\beta_t}}, \beta_0 \gg 1, \beta_1 = 0. \tag{VE}$$

上述公式中的 "$\gg 1$" 要求标量充分大，使 $p_0(x) = \int p_{0|1}(x|x_1)q(x_1)\mathrm{d}x_1$ 在 $t = 0$ 时接近一个已知高斯分布，即 VE 情形的 $\mathcal{N}(\cdot|0, \sigma_0^2 I)$，以及 VP 情形的 $\mathcal{N}(\cdot|0, I)$. 注意，在这两种情形下，$p_t(x)$ 在 $t = 0$ 时都不精确等于 p，这与 (4.51) 中的 FM 路径不同.

在高斯情形下具有简单形式的一个有用量是**得分**，定义为对数概率的梯度. 具体而言，(4.70) 中条件路径的得分为

$$\nabla \log p_{t|1}(x|x_1) = -\frac{1}{\sigma_t^2}(x - \alpha_t x_1). \tag{4.71}$$

相应边缘概率路径 (4.4) 的得分为

$$\begin{aligned}
\nabla \log p_t(x) &= \int \frac{\nabla p_{t|1}(x|x_1)q(x_1)}{p_t(x)}\mathrm{d}x_1 && (4.72)\\
&= \int \nabla \log p_{t|1}(x|x_1)\frac{p_{t|1}(x|x_1)q(x_1)}{p_t(x)}\mathrm{d}x_1 && (4.73)\\
&= \mathbb{E}\left[\nabla \log p_{t|1}(X_t|X_1) \,|\, X_t = x\right] && (4.74)\\
&\overset{(4.71)}{=} \mathbb{E}\left[-\frac{1}{\sigma_t^2}(X_t - \alpha_t X_1) \,|\, X_t = x\right] && (4.75)\\
&\overset{(4.54)}{=} \mathbb{E}\left[-\frac{1}{\sigma_t}X_0 \,\middle|\, X_t = x\right] && (4.76)\\
&\overset{(4.59)}{=} -\frac{1}{\sigma_t}x_{0|t}(x), && (4.77)
\end{aligned}$$

其中借用了 (4.59) 中的记号 $x_{0|t}$. 扩散文献将 x_0 预测（$x_{0|t}$）称为噪声预测，或 ϵ 预测. 上式表明，得分与 x_0 预测成正比，并为高斯路径情形提供了从得分到其他参数化的转换规则，如 表 1 所示.

B \ A	速度	x_1 预测	x_0 预测	得分
速度	$0, 1$	$\frac{\dot\sigma_t}{\sigma_t}, \frac{\dot\alpha_t\sigma_t - \dot\sigma_t\alpha_t}{\sigma_t}$	$\frac{\dot\alpha_t}{\alpha_t}, \frac{\dot\sigma_t\alpha_t - \dot\alpha_t\sigma_t}{\alpha_t}$	$\frac{\dot\alpha_t}{\alpha_t}, -\frac{\dot\sigma_t\sigma_t\alpha_t - \dot\alpha_t\sigma_t^2}{\alpha_t}$
x_1 预测		$0, 1$	$\frac{1}{\alpha_t}, -\frac{\sigma_t}{\alpha_t}$	$\frac{1}{\alpha_t}, \frac{\sigma_t^2}{\alpha_t}$
x_0 预测			$0, 1$	$0, -\sigma_t$
得分				$0, 1$

表 1 不同模型参数化之间的转换：(a_t, b_t) 对应于 $f_t^B(x) = a_t x + b_t f_t^A(x)$. 颜色分别表示变换适用于 所有路径、仿射路径 和 高斯路径. 下三角部分由上三角部分通过逆变换 $f_t^A(x) = \frac{1}{b_t}\left(-a_t x + f_t^B(x)\right)$ 计算，可表示为系数对 $-\frac{a_t}{b_t}, \frac{1}{b_t}$. 注意，其中一些转换存在奇点，相关讨论见第 4.8.1 节末尾.

边缘速度的动能最优性. 上述转换公式（表 1）的一个推论是，高斯路径的边缘速度可以写成

$$u_t(x) = \frac{\dot\alpha_t}{\alpha_t} x - \frac{\dot\sigma_t\sigma_t\alpha_t - \dot\alpha_t\sigma_t^2}{\alpha_t}\nabla \log p_t(x) \tag{4.78}$$

$$= \nabla\left[\frac{\dot\alpha_t}{2\alpha_t}\|x\|^2 - \frac{\dot\sigma_t\sigma_t\alpha_t - \dot\alpha_t\sigma_t^2}{\alpha_t}\log p_t(x)\right] \tag{4.79}$$

这表明 $u_t(x)$ 是一个梯度，因此对于由 $p_{t|1}(x|x_1) = \mathcal{N}(x|\alpha_t x_1, \sigma_t^2 I)$ 定义的固定边缘化高斯概率路径 $p_t(x)$，它具有动能最优性（例如见 Villani (2021) 第 8.1.2 节，或 Neklyudov et al. (2023) 定理 2.1）.

4.9 数据耦合

在构建流匹配训练算法时，假设可以从源分布 p 与目标分布 q 的某个耦合 $\pi_{0,1}$ (x_0, x_1) 中抽取样本 $(X_0, X_1) \sim \pi_{0,1}(X_0, X_1)$. 例如，独立样本 $\pi_{0,1}(x_0, x_1) = p(x_0)q(x_1)$，这是保持边缘分布 p 和 q 的最简单耦合；或者由数据集直接提供的配对样本 $(X_0, X_1) \sim \pi_{0,1}$. 本节探讨可用于训练流匹配模型的两种具体耦合示例.

4.9.1 配对数据

非独立耦合自然出现于配对数据上的学习任务中. 例如，考虑图像修补任务，其中 q 是自然图像的分布，p 是将同一批图像的一个正方形区域遮挡后所得图像的分布. 这里的目标是学习从被遮挡图像 x_0 到其修补版本 x_1 的映射，而非将噪声变换为数据. 由于该问题是不适定的，即每个被遮挡图像 x_0 都可以对应许多修补图像 x_1，求解这一任务可以表述为学习从未知、依赖于数据的耦合 $\pi_{1|0}(x_1|x_0)$ 中采样.

基于这些认识，Liu et al. (2023); Albergo et al. (2024) 提出利用依赖于数据的耦合学习桥或流模型；这一简单修改开启了新的应用范围. 虽然不能从所关心的 $\pi_{1|0}(x_1|x_0)$ 中采样，但通常可以从反向依赖关系 $\pi_{0|1}(x_0|x_1)$ 中采样. 回到图像修补的例子，很容易通过遮挡完整图像 $X_1 \sim q$（目标样本）产生源样本 $X_0 \sim p$. 为此，指定

$$\pi_{0,1}(x_0, x_1) = \pi_{0|1}(x_0|x_1)q(x_1). \tag{4.80}$$

因此，可以通过以下步骤得到样本对 (X_0, X_1)：

1. 抽取 $X_1 \sim q$.

2. 施加预先定义的随机变换，从 X_1 得到 X_0.

为满足 推论 1 的条件（确保源分布具有密度），并促进多样性，在从 $\pi_{0|1}(x_0|x_1)$ 采样时加入噪声. (Liu et al., 2023; Albergo et al., 2024) 在图像超分辨率、修补、去模糊等多种应用中展示了这一方法的能力，优于基于引导扩散的方法 (Saharia et al., 2022).

4.9.2 多样本耦合

如 小节 4.7 所讨论，直的概率路径使常微分方程模拟的误差更小. 因此，一个自然的问题是：如何改变训练算法，使学得的速度场诱导出直的（或更直的）轨迹?

如前所述，直线轨迹与最优传输（OT）问题有关. 具体而言，考虑凸代价泛函 $c : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$，以及条件 OT 流 $\psi_t(x_0|x_1) = tx_1 + (1-t)x_0$. 则耦合的传输代价给出了边缘传输代价的上界 (Liu et al., 2022; Pooladian et al., 2023)，即：

$$\mathbb{E}\left[c(\psi_1(X_0) - X_0)\right] \leq \mathbb{E}\left[c(X_1 - X_0)\right], \tag{4.81}$$

其中 $c(x) = \|x\|^2$ 的情形可以从 (4.49) 的上界理解：在左端代入满足 $u_t(X_t) = u_t(\psi_t(x)) = \phi(x) - x$ 的 OT 解，其中 ϕ 是 OT 映射. 因此，可以通过减小耦合代价来构造低代价的边缘传输映射. 为此，Pooladian et al. (2023) 提出**多样本耦合**，通过以下过程在源分布与目标分布之间引入依赖关系，隐式构造非平凡的联合分布 $\pi_{0,1}(x_0, x_1)$：

1. 独立采样 $X_0^{(i)} \sim p$ 和 $X_1^{(i)} \sim q$，$i \in [k]$.
2. 通过 $\pi^k \coloneqq \arg\min_{\pi \in B_k} \mathbb{E}_\pi \left[c(X_0^{(i)} - X_1^{(j)})\right]$ 构造 $\pi^k \in B_k$.
3. 从满足 $\pi^k(i,j) = 1$ 的 $(X_0^{(i)}, X_1^{(j)})$ 中均匀随机采样一对 $(X_0^{(i)}, X_0^{(j)})$.

其中 B_k 是 $k \times k$ 双随机矩阵构成的多面体.

上述过程通过采样隐式定义了联合分布 $\pi_{0,1}^k(x_0, x_1)$. 这一隐式联合分布保持边缘分布，并满足最优性约束（第 2 步）(Pooladian et al., 2023). 当 $k = 1$ 时，该方法退化为独立耦合. 当 $k > 1$ 时，Pooladian et al. (2023) 表明，相比独立耦合，传输代价有所降低，即 $\mathbb{E}_{(x_0,x_1)\sim\pi_{0,1}^k(x_0,x_1)}\left[c(x_1 - x_0)\right] \leq \mathbb{E}_{X_0\sim p, X_1\sim q}\left[c(X_1 - X_0)\right]$. 此外，对于二次代价函数，当 $k \to \infty$ 时，多样本耦合趋近最优传输代价，并诱导直线轨迹 (Pooladian et al., 2023; Tong et al., 2023).

4.10 条件生成与引导

现在考虑在引导信号下训练生成模型，以进一步控制生成的样本. 这一技术已在许多实际应用中展现价值，例如图像到图像的转换 (Saharia et al., 2022)，以及文本到图像的生成 (Nichol et al., 2022; Esser et al., 2024). 本小节假设可以获得带标签的目标样本 (x_1, y)，其中 $y \in \mathcal{Y} \subseteq \mathbb{R}^k$ 是标签或引导变量.

4.10.1 条件模型

在引导下训练生成模型的一种自然方式，是学习从条件分布 $q(x_1|y)$ 中采样，扩散模型和 FM 模型都已展示这一点 (Zheng et al., 2023). 按照 图 2 中的 FM 蓝图，考虑来自条件目标分布 $q(x_1|y)$ 的样本，并指定一个简单的源分布 p，通常但不一定是高斯分布. 接下来，通过汇总条件概率路径，设计**引导概率路径**：

$$p_{t|Y}(x|y) = \int p_{t|1}(x|x_1) q(x_1|y) \mathrm{d}x_1. \tag{4.82}$$

其中假设 $p_{t,1|Y}(x,x_1|y)=p_{t|1}(x|x_1)q(x_1|y)$，意味着条件路径不依赖于 Y. 得到的引导概率路径以引导变量 $Y\sim p_Y$ 为条件，并满足边缘端点条件

$$p_{0|Y}(\cdot|y)=p(\cdot),\quad p_{1|Y}(\cdot|y)=q(\cdot|y). \tag{4.83}$$

引导速度场具有如下形式：

$$u_t(x|y)=\int u_t(x|x_1)p_{1|t,Y}(x_1|x,y)\mathrm{d}x_1, \tag{4.84}$$

其中，由贝叶斯公式，有

$$p_{1|t,Y}(x_1|x,y)=\frac{p_{t|1}(x|x_1)q(x_1|y)}{p_{t|Y}(x|y)}. \tag{4.85}$$

为证明 $u_t(x|y)$ 生成 $p_{t|Y}(x|y)$，将 (4.82) 和 (4.84) 代入 (4.14). 同时可以看到，在引导情形下 FM/CFM 损失保持不变，且可以采用 定理 4 证明中的相同步骤. 实践中，训练单个神经网络 $u_t^\theta:\mathbb{R}^d\times\mathbb{R}^k\to\mathbb{R}^d$，对所有 y 的取值建模引导边缘速度场. 随后，CFM 损失 (4.32) 的引导版本为

$$\mathcal{L}_{\mathrm{CFM}}(\theta)=\mathbb{E}_{t,(X_0,X_1,Y)\sim\pi_{0,1,Y}}D\left(\dot\psi_t(X_0|X_1),u_t^\theta(X_t|Y)\right). \tag{4.86}$$

实践中，扩散模型文献表明，在大量目标样本 X_1 共享相同引导信号 Y 的应用中，引导最为有效，例如类别引导 (Nichol and Dhariwal, 2021). 然而，当引导变量 Y 不重复且结构复杂时，例如图像描述，引导更具挑战.

4.10.2 分类器引导与无分类器引导

对于使用高斯路径训练的流，可以利用 表 1 中条件分布的速度场与得分函数之间的变换，应用分类器引导 (Song et al., 2021; Dhariwal and Nichol, 2021) 和无分类器引导 (Ho and Salimans, 2021)，见 (Zheng et al., 2023)：

$$u_t(x|y)=a_tx+b_t\nabla\log p_{t|Y}(x|y). \tag{4.87}$$

对引导概率路径使用贝叶斯公式，得到

$$p_{t|Y}(x|y)=\frac{p_{Y|t}(y|x)p_t(x)}{p_Y(y)}. \tag{4.88}$$

取对数并关于 x 求梯度，$\nabla=\nabla_x$，得到概率路径 $p_t(x)$ 与其引导版本 $p_{t|Y}(x|y)$ 的得分之间的基本关系：

$$\overbrace{\nabla\log p_{t|Y}(x|y)}^{\text{条件得分}}=\nabla\overbrace{\log p_{Y|t}(y|x)}^{\text{分类器}}+\overbrace{\nabla\log p_t(x)}^{\text{无条件得分}} \tag{4.89}$$

也就是说，两者通过分类器模型 $p_{Y|t}(y|x)$ 的得分联系起来，该分类器尝试在给定样本 x 时预测引导变量 y.

基于这一关系，Song et al. (2021) 提出**分类器引导**：利用依赖时间的分类器（给定 $x\sim p_t(x)$ 时预测引导变量 y）引导无条件模型（以 $\nabla\log p_t(x)$ 参数化），从而从条件模型 $q(x_1|y)$ 中采样. 相

应的速度场于是转化为：

$$\tilde{u}_t^{\theta,\phi}(x|y) = a_t x + b_t \left(\nabla \log p_{Y|t}^{\phi}(y|x) + \nabla \log p_t^{\theta}(x) \right) = u_t^{\theta}(x) + b_t \nabla \log p_{Y|t}^{\phi}(y|x), \tag{4.90}$$

其中 $u_t^{\theta}(x)$ 是在无条件目标 $q(x)$ 上训练的速度场，$\log p_{Y|t}^{\phi}(y|x)$ 是参数为 $\phi \in \mathbb{R}^m$、依赖时间的分类器. Dhariwal and Nichol (2021) 表明，对于类别条件和文本条件 (Nichol et al., 2022)，这种方法都优于 小节 4.10.1 中的条件模型. 实践中，由于分类器与无条件得分分别学习，通常需要如下校准分类器引导：

$$\tilde{u}_t^{\theta,\phi}(x|y) = u_t^{\theta}(x) + b_t w \nabla \log p_{Y|t}^{\phi}(y|x), \tag{4.91}$$

其中 $w \in \mathbb{R}$ 是分类器尺度，通常选择 $w > 1$ (Dhariwal and Nichol, 2021).

在后续工作中，(Ho and Salimans, 2021) 提出一种称为**无分类器引导**的纯生成式方法. 简单重排 (4.89)，得到

$$\overbrace{\nabla \log p_{Y|t}(y|x)}^{\text{分类器}} = \overbrace{\nabla \log p_{t|Y}(x|y)}^{\text{条件得分}} - \overbrace{\nabla \log p_t(x)}^{\text{无条件得分}} \tag{4.92}$$

这表明，可以通过普通概率路径与引导概率路径的得分之差，隐式近似分类器的得分. 随后，作者提出使用同一个模型同时学习条件得分和无条件得分. 对于速度，Zheng et al. (2023) 表明，也可以将 4.92 代入 4.91，并利用 表 1 中从得分到速度的转换，得到：

$$\tilde{u}_t^{\theta}(x|y) = (1-w) u_t^{\theta}(x|\emptyset) + w u_t^{\theta}(x|y), \tag{4.93}$$

其中 w 仍是引导的校准尺度. 现在仅训练单个模型 $u_t^{\theta}(x|y)$，其中 $y \in \{\mathcal{Y}, \emptyset\}$，$\emptyset$ 是表示空条件的占位值，而 $u_t^{\theta}(x|\emptyset)$ 是生成无条件概率路径 $p_t(x)$ 的速度场. 得到的损失为：

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,\xi,(X_0,X_1,Y)\sim\pi_{0,1,Y}} \left[D\left(\dot{\psi}_t(X_0|X_1), u_t^{\theta}(X_t|(1-\xi)\cdot Y + \xi \cdot \emptyset) \right) \right], \tag{4.94}$$

其中 $\xi \sim \text{Bernoulli}(p_{\text{uncond}})$，$p_{\text{uncond}}$ 是训练时抽取空条件 $\emptyset$ 的概率. CFG 实际采样的精确分布尚不清楚，一些工作为 CFG 采样提出了不同的直观或理论解释 (Dieleman, 2022; Guo et al., 2024; Chidambaram et al., 2024; Bradley and Nakkiran, 2024). 尽管如此，在本文撰写时，CFG 仍是训练条件模型最常用的方法. Esser et al. (2024); Polyak et al. (2024) 展示了应用无分类器引导训练大规模引导 FM 模型的方法.

5 非欧几里得流匹配

本节将流匹配从欧几里得空间 $\mathbb{R}^d$ 推广到一般的黎曼流形 $\mathcal{M}$. 非正式地说，黎曼流形是在局部表现得像欧几里得空间、并配备广义距离与角度概念的空间. 黎曼流形可用于对各种类型的数据建模. 例如，地球上自然现象的概率可以在球面上建模 Mathieu and Nickel (2020)，而蛋白质骨架通常以矩阵李群进行参数化 Jumper et al. (2021). 将流推广到黎曼流形的工作由 Mathieu and Nickel (2020); Lou et al. (2020) 完成. 然而，他们最初的训练算法需要代价高昂的 ODE 模拟. 遵循 Chen and Lipman (2024)，本节的流匹配方法提供了一种可扩展、无需模拟的训练算法，用于学习黎曼流形上的生成模型.

5.1 黎曼流形

我们考虑配备度量 g 的完备、连通、光滑黎曼流形 $\mathcal{M}$. 点 $x \in \mathcal{M}$ 处的切空间记为 $T_x\mathcal{M}$, 它是包含 $\mathcal{M}$ 在 x 处所有切向量的向量空间. 黎曼度量 g 在 $T_x\mathcal{M}$ 上定义一个内积, 对 $u, v \in T_x\mathcal{M}$, 记为 $\langle u, v\rangle_g$. 令 $T\mathcal{M} = \cup_{x\in\mathcal{M}} \{x\} \times T_x\mathcal{M}$ 为切丛, 它汇集了流形的所有切平面. 在下文中, 定义在切空间上的向量场是通过速度场构造流形上的流的重要对象. 我们用 $\mathcal{U} = \{u_t\}$ 表示依赖时间的光滑向量场 (VF) $u_t : [0,1] \times \mathcal{M} \to T\mathcal{M}$ 所构成的空间, 其中对所有 $x \in \mathcal{M}$, 都有 $u_t(x) \in T_x\mathcal{M}$. 此外, $\mathrm{div}_g(u_t)$ 是关于空间变量 (x) 的黎曼散度. 最后, 我们用 dvol_x 表示 $\mathcal{M}$ 上的体积元, 函数 $f : \mathcal{M} \to \mathbb{R}$ 在 $\mathcal{M}$ 上的积分记为 $\int f(x)\mathrm{dvol}_x$.

5.2 流形上的概率、流与速度

流形 $\mathcal{M}$ 上的概率密度函数是积分为 1 的连续非负函数 $p : \mathcal{M} \to \mathbb{R}_{\geq 0}$, 即 $\int_{\mathcal{M}} p(x)\mathrm{dvol}_x = 1$. 我们将随时间变化的概率路径 p_t 定义为概率空间 $\mathcal{P}$ 中依赖时间的曲线, 即 $p_t : [0,1] \to \mathcal{P}$. 与欧几里得空间的情形类似, 依赖时间的流 $\psi : [0,1] \times \mathcal{M} \to \mathcal{M}$ 对每个 t 都定义了 $\mathcal{M}$ 上的全局微分同胚.

值得关注的是, 通过速度场构造基于流的模型, 自然适用于一般黎曼流形. 形式化地, 将 Mathieu and Nickel (2020) 中的命题 1 重述如下:

定理 8 (流的局部存在性与唯一性)**.** *设 $\mathcal{M}$ 为光滑完备流形, $u_t \in \mathcal{U}$ 为速度场. 若 u 属于 $C^\infty([0,1] \times \mathcal{M}, T\mathcal{M})$ (特别地, 局部 Lipschitz), 则 (3.19) 中的 ODE 有唯一解, 该解是定义在开集 $\Omega \supset \{0\} \times \mathcal{M}$ 上的 $C^\infty(\Omega, \mathcal{M})$ 微分同胚 $\psi_t(x)$.*

与 定理 1 类似, 流的 ODE 一般只在流形上定义局部微分同胚, 这意味着对于不同的 $x \in \mathcal{M}$, $\psi_t(x)$ 可能定义在最大时间区间 $[0, t_x])$ 上. 与欧几里得情形类似, 我们采用半开时间区间 $t \in [0,1)$, 以允许 q 具有紧支集 (此时 u_t 并非处处有定义). 为确保解在所需时间区间 $[0,1)$ 上存在, 我们加入可积性约束 (见 定理 2), 并再次借助质量守恒定理. 对于配备度量 g 的黎曼流形, **黎曼连续性方程**为

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) + \mathrm{div}_g(p_t u_t)(x) = 0, \tag{5.1}$$

相应的流形质量守恒定理 (Villani et al., 2009) 表述如下.

定理 9 (流形质量守恒)**.** *设 p_t 为概率路径, $u_t \in \mathcal{U}$ 为配备度量 g 的黎曼流形 $\mathcal{M}$ 上局部 Lipschitz 且可积的向量场. 则以下命题等价:*

1. *连续性方程 (5.1) 对 $t \in [0,1)$ 成立.*
2. *u_t 在 3.24 的意义下生成 p_t.*

上述结果中, u 可积是指

$$\int_0^1 \int_{\mathcal{M}} \|u_t(x)\|\, p_t(x)\mathrm{dvol}_x \mathrm{d}t < \infty. \tag{5.2}$$

注意, 定理 9 的假设给出 $\mathcal{M}$ 上的全局微分同胚, 从而得到**黎曼瞬时变量替换**公式:

$$\frac{\mathrm{d}}{\mathrm{d}t} \log p_t(\psi_t(x)) = -\mathrm{div}_g(u_t)(\psi_t(x)). \tag{5.3}$$

最后，若

$$X_t = \psi_t(X_0) \sim p_t, \text{ 其中} X_0 \sim p. \tag{5.4}$$

则称 u_t 从 p 生成 p_t. 既然流是流形上的有效生成模型，FM 原理也应能够迁移到这一领域. 在 FM 的黎曼版本中，我们希望找到速度场 $u_t^\theta \in \mathcal{U}$，使其生成满足边缘约束 $p_0 = p$ 和 $p_1 = q$ 的目标概率路径 $p_t : [0,1] \to \mathcal{P}$，其中 p, q 表示流形 $\mathcal{M}$ 上的源分布和目标分布. 由于速度场位于流形的切空间中，**黎曼流匹配损失**使用定义在流形各个切平面上的 Bregman 散度来比较速度：

$$\mathcal{L}_{\text{RFM}}(\theta) = \mathbb{E}_{t, X_t \sim p_t} D_{X_t}\left(u_t(X_t), u_t^\theta(X_t)\right). \tag{5.5}$$

在上式中，期望现在是流形上的积分，即对于光滑函数 $f : \mathcal{M} \to \mathcal{M}$ 和随机变量 $X \sim p_X$，有 $E[f(X)] = \int_{\mathcal{M}} f(x) p_X(x) \text{dvol}_x$. Bregman 散度 D_x ($x \in \mathcal{M}$) 可以通过黎曼内积以及分配给每个切空间的严格凸函数 $\Phi_x : T_x\mathcal{M} \to T_x\mathcal{M}$ 定义，即 $D_x(u, v) := \Phi_x(u) - [\Phi_x(v) + \langle u - v, \nabla_v \Phi_x(v) \rangle_g]$. 例如，选择黎曼度量 $\Phi_x = \|\cdot\|_g^2$，则对 $u, v \in T_x\mathcal{M}$ 有 $D_x(u, v) = \|u - v\|_g^2$.

5.3 流形上的概率路径

边缘概率路径的构造与欧几里得情形 (4.4) 相同：

$$p_t(x) = \int_{\mathcal{M}} p_t(x|x_1) q(x_1) \text{dvol}_{x_1}, \tag{5.6}$$

其中 $p_{t|1}(x|x_1)$ 是定义在流形上的**条件概率路径**. 我们还要求边界约束

$$p_0 = p, \quad p_1 = q. \tag{5.7}$$

例如，可以要求条件路径 $p_{t|1}(x|x_1)$ 满足

$$p_{0|1}(x|x_1) = \pi_{0|1}(x|x_1), \text{ 且} p_{1|1}(x|x_1) = \delta_{x_1}(x), \tag{5.8}$$

来实现这些约束，其中 $\pi_{0|1}$ 是条件耦合，$\pi_{0|1}(x_0|x_1) = \pi_{0,1}(x_0, x_1)/q(x_1)$.

5.4 流形上的边缘化技巧

针对边缘速度场的边缘化技巧（定理 3）可以直接应用于黎曼情形. 考虑**条件速度场** $u_t(x|x_1) \in \mathcal{U}$，使得

$$u_t(\cdot|x_1) \text{ 生成} p_{t|1}(\cdot|x_1). \tag{5.9}$$

则**边缘速度**场 $u_t(x)$ 由如下条件速度的平均给出：

$$u_t(x) = \int_{\mathcal{M}} u_t(x|x_1) p_{1|t}(x_1|x) \text{dvol}_{x_1}, \tag{5.10}$$

其中，由概率密度函数的 Bayes 公式可得

$$p_{1|t}(x_1|x) = \frac{p_{t|1}(x|x_1) q(x_1)}{p_t(x)}, \tag{5.11}$$

该表达式对所有满足 $p_t(x) > 0$ 的 $x \in \mathcal{M}$ 有定义.

黎曼情形下的边缘化技巧（定理 3）需要将 假设 1 调整如下：

假设 2. 作为 (t, x) 的函数，$p_{t|1}(x|x_1)$ 属于 $C^\infty([0,1)\times\mathcal{M})$，$u_t(x|x_1)$ 属于 $C^\infty([0,1)\times\mathcal{M}, \mathcal{M})$. 此外，假设 q 具有有界支集，即 $q(x_1) = 0$ 在某个有界集之外成立，或者 $\mathcal{M}$ 是紧的；并且对所有 $x \in \mathcal{M}$ 和 $t \in [0,1)$，都有 $p_t(x) > 0$.

现在可以陈述**流形边缘化技巧**定理：

定理 10 (流形边缘化技巧)**.** *在假设 2 下，若 $u_t(x|x_1)$ 条件可积且生成条件概率路径 $p_t(\cdot|x_1)$，则边缘速度场 $u_t(\cdot)$ 生成边缘概率路径 $p_t(\cdot)$.*

条件可积指质量守恒定理中可积性条件 (5.2) 的条件版本：

$$\int_0^1 \int_{\mathcal{M}} \int_{\mathcal{M}} \|u_t(x|x_1)\|_g \, p_{t|1}(x|x_1) q(x_1) \mathrm{dvol}_{x_1} \mathrm{dvol}_x \mathrm{d}t < \infty \tag{5.12}$$

定理 10 的证明重复了 定理 3 的论证，见 小节 A.2.

5.5 黎曼流匹配损失

黎曼条件流匹配（RCFM）损失为

$$\mathcal{L}_{\mathrm{RCFM}}(\theta) = \mathbb{E}_{t, X_1, X_t \sim p_{t|1}(\cdot|X_1)} D_{X_t}\left(u_t(X_t|X_1), u_t^\theta(X_t)\right). \tag{5.13}$$

我们再次得到等价关系：

定理 11. *黎曼流匹配损失与黎曼条件流匹配损失的梯度一致：*

$$\nabla_\theta \mathcal{L}_{RFM}(\theta) = \nabla_\theta \mathcal{L}_{RCFM}(\theta). \tag{5.14}$$

令 $X = X_t$、$Y = u_t(X_t|X_1)$、$g^\theta(x) = u_t^\theta(x)$，使用 命题 1 并关于 $t \in [0,1]$ 积分，即可证明上述定理.

5.6 通过预度量构造条件流

在确定如何用 RCFM 损失学习流模型之后，还需指定条件概率路径及其生成速度场. 与 小节 4.6 类似，我们首先陈述相应条件流 $\psi : [0,1) \times \mathcal{M} \times \mathcal{M} \to \mathcal{M}$ 应满足的要求，使 $p_{t|1}(\cdot|x_1)$ 满足边界条件 (5.8). **条件流模型**为

$$X_{t|1} = \psi_t(X_0|x_1), \quad \text{其中} X_0 \sim \pi_{0|1}(\cdot|x_1), \tag{5.15}$$

其中**条件流**满足

$$\psi_t(x|x_1) = \begin{cases} x & t = 0 \\ x_1 & t = 1 \end{cases}, \quad \text{关于} t, x \text{ 光滑，且关于 } x \text{ 是} \mathcal{M} \text{ 上的微分同胚.} \tag{5.16}$$

我们在欧几里得空间中的分析侧重于仿射条件流，因为它们构成了一类丰富且易于计算（无需模拟）的条件流. 遗憾的是，当 $\alpha_t + \sigma_t \neq 1$ 时，组合 $\alpha_t x_1 + \sigma_t x_0$ 在流形上没有自然的定义. 对

$\alpha_t + \sigma_t = 1$ 的情形，流形上的类似构造是使用测地插值. 事实上，Chen and Lipman (2024) 提出沿测地曲线运动来构造条件流，特别是将欧几里得空间中沿直线运动的条件 OT 路径加以推广（见 定理 5）. 测地线表示流形上两点之间的最短路径，在欧几里得空间中退化为直线. 对于流形，我们将**测地条件流**定义为

$$\psi_t(x_0|x_1) = \exp_{x_0}(\kappa(t)\log_{x_0}(x_1)), \quad t \in [0,1], \tag{5.17}$$

其中 $\kappa(t) : [0,1] \to [0,1]$ 是满足 $\kappa(0) = 0$ 和 $\kappa(1) = 1$ 的单调递增调度函数，确保在 $t = 1$ 时将所有 x_0 推到 x_1. 在 $x \in \mathcal{M}$ 处的指数映射 $\exp_x : T_x\mathcal{M} \to \mathcal{M}$，$v \mapsto \exp_x(v)$，返回从 x 出发、初速度为 v 的唯一测地线在时间 $t = 1$ 的终点. 对数映射 $\log_x : \mathcal{M} \to T_xM$，$y \mapsto \log_x(y)$，是指数映射的逆映射. 在欧几里得空间中，指数映射就是向量加法，对数映射就是向量减法. 将这些映射代入 (5.17)，得到 $\psi_t(x_0|x_1) = x_0 + \kappa(t)(x_1 - x_0)$；选择 $\kappa(t) = t$，便得到条件 OT 流.

对于具有闭式指数映射和对数映射的简单流形，这种构造给出了一种无需模拟的流形上流模型训练方法，相较于流形上的扩散模型方法 (De Bortoli et al., 2022; Huang et al., 2022b; Lou et al., 2023)，这可谓明显的优势. 特别地，流形扩散模型需要在训练中进行模拟以从 p_t 采样，并且不得不使用流形上得分函数的近似.

不过，尽管构造测地条件流是自然的方法，对于没有闭式指数映射和对数映射的一般流形，测地线可能难以计算，并且 / 或者会引入不希望出现的偏置，例如将概率集中在边界点上. 为了克服测地线的计算困难，并且 / 或者引入期望的隐式偏置，可以寻找另一种光滑距离函数 $d(\cdot,\cdot) : \mathcal{M} \times \mathcal{M} \to \mathbb{R}_{\geq 0}$，并要求条件流满足

$$d(\psi_t(x_0|x_1), x_1) = \bar{\kappa}(t)d(x_0, x_1), \tag{5.18}$$

其中 $\bar{\kappa}(t) = 1 - \kappa(t)$. 若以下条件成立，就能确保条件流在时间 $t = 1$ 将所有概率集中到 x_1：

1. 非负性：对所有 $x, y \in \mathcal{M}$，$d(x, y) \geq 0$.
2. 正定性：$d(x, y) = 0$ 当且仅当 $x = y$.
3. 非退化性：$\nabla d(x, y) \neq 0$ 当且仅当 $x \neq y$.

Chen and Lipman (2024) 表明，满足 (5.18) 的流所对应的最小范数条件速度场具有如下形式：

$$u_t(x|x_1) = \frac{d\log\bar{\kappa}(t)}{\mathrm{d}t} d(x, x_1) \frac{\nabla d(x, x_1)}{\|\nabla d(x, x_1)\|_g^2}, \tag{5.19}$$

其中预度量的非退化性要求确保速度场没有不连续点，因为 $u_t(x|x_1) \propto 1/\|\nabla d(x, x_1)\|_g$. 特别地，注意到当选择 $d = d_g$ 时，(5.17) 中的测地条件流满足 (5.18)，其中 d_g 是测地距离. 另一种预度量选择的例子是在一般几何结构上使用谱距离 (Chen and Lipman, 2024)，此时条件速度提供了通过模拟从 $p_t(x|x_1)$ 采样的方法. 重要的是，尽管使用预度量的条件流与流形上的扩散模型一样，需要在训练中进行模拟，但相较于得分函数的近似，速度场仍然可以准确恢复.

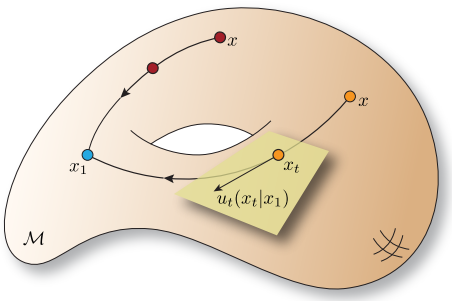


图 12 流形 $\mathcal{M}$ 上的条件流.

另一个问题是，通过测地插值和预度量定义的条件流都可能出现奇点，例如在紧流形上. 以二维球面为例，测地距离函数 $d(x, x_1)$ 在对跖点 $x = -x_1$ 处不可微. 此外，任何诸如 $x \mapsto d(x, x_1)$ 的光滑函数都会有至少

两个临界点（最大值点和最小值点），而 (5.19) 中的速度在这些点处没有定义. 然而，这类有问题的点组成的集合通常非常小（事实上，其体积通常为零）. 因此，至少在我们所知的应用场景中，这一问题并不会在实践中造成困难.

无论如何，为处理这一问题，可以在测地条件流中加入一个扩展调度函数. 也就是使用还依赖于 x, x_1 的 $\bar{\kappa}(t, x, x_1)$，以使 (5.17) 全局光滑. 为处理预度量条件流的零梯度问题，可以如下放宽非退化性要求：

3. 非退化性（放宽）：对所有 $y \in \mathcal{M}$，集合 $\mathcal{A}_y = \{x \in \mathcal{M} \,|\, \nabla d(x, y) = 0 \text{ 且} x \neq y\}$ 的体积为 0.

代码 8: 使用 CFM 目标训练球面上的测地流

```python
import torch
from flow_matching.path import GeodesicProbPath, PathSample
from flow_matching.path.scheduler import CondOTScheduler
from flow_matching.utils.manifolds import Sphere

model = ...  # 定义可训练的速度模型
optimizer = torch.optim.Adam(model.parameters())
loss_fn = torch.nn.MSELoss()  # 任意 Bregman 散度

manifold = Sphere()
scheduler = CondOTScheduler()
path = GeodesicProbPath(scheduler=scheduler, manifold=manifold)

for x_0, x_1 in dataloader:  # 来自 π_{0,1} 的样本，形状为 [batch_size, *data_dim]
    t = torch.rand(batch_size)  # 随机选取时间 t ~ U[0,1]
    sample: PathSample = path.sample(t=t, x_0=x_0, x_1=x_1)  # 对条件路径采样

    model_output = model(sample.x_t, sample.t)
    projected_model_output = manifold.proju(sample.x_t, model_output)  # 投影到切空间

    loss = loss_fn(projected_model_output, sample.dx_t)  # CFM 损失

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

6 连续时间马尔可夫链模型

本节介绍**连续时间马尔可夫链（CTMC）**，作为流之外的另一种生成模型，用于生成离散数据，即位于离散（且有限）状态空间中的数据. CTMC 是马尔可夫过程，是离散流匹配（DFM）生成模型范式的基本构件 Campbell et al. (2024); Gat et al. (2024)；我们将在 节 7 中讨论这一范式. 因此，本节与 节 3 类似，在那里我们介绍了流，作为流匹配（FM）生成模型范式的基本构件.

6.1 离散状态空间与随机变量

考虑 $\mathbb{R}^d$ 的一个有限版本作为状态空间 $\mathcal{S} = \mathcal{T}^d$，其中 $\mathcal{T} = [K] = \{1, 2, \ldots, K\}$，有时称为**词表**. 样本和状态记为 $x = (x^1, \ldots, x^d) \in \mathcal{S}$，其中 $x^i \in \mathcal{T}$ 是单个坐标，或称为**词元**. 我们也类似地使用状态 $y, z \in \mathcal{S}$. 接下来，X 表示取值于状态空间 $\mathcal{S}$ 的随机变量，其概率由**概率质量函数**

(PMF) $p_X : \mathcal{S} \to \mathbb{R}_{\geq 0}$ 给出，满足 $\sum_{x\in\mathcal{S}} p_X(x) = 1$；事件 $A \subset \mathcal{S}$ 的概率为

$$\mathbb{P}(X \in A) = \sum_{x\in A} p_X(x). \tag{6.1}$$

记号 $X \sim p_X$ 或 $X \sim p_X(X)$ 表示 X 的 PMF 为 p_X. 离散情形下的 δ PMF 定义为

$$\delta(x, z) = \begin{cases} 1 & x = z, \\ 0 & \text{其他情形}. \end{cases} \tag{6.2}$$

其中我们有时也在词元上定义 δ PMF，例如对某些 $x^i, y^i \in \mathcal{T}$ 定义 $\delta(x^i, y^i)$.

6.2 CTMC 生成模型

CTMC 模型是取值于 $\mathcal{S}$、依赖时间的随机变量族 $(X_t)_{0\leq t\leq 1}$，构成一条马尔可夫链，由如下定义的**概率转移核** $p_{t+h|t}$ 刻画：

$$p_{t+h|t}(y|x) \coloneqq \mathbb{P}(X_{t+h} = y | X_t = x) = \delta(y, x) + hu_t(y, x) + o(h), \text{ 且} \mathbb{P}(X_0 = x) = p(x), \tag{6.3}$$

其中 PMF p 表示过程在时间 $t = 0$ 的初始分布，而 $o(h)$ 是任意满足当 $t \to 0$ 时 $o(h)/h \to 0$ 的函数. $u_t(y, x)$ 的值称为**速率**或**速度**，表示概率在状态之间转移的速度随时间的变化. 完全刻画是指，对任意 $0 \leq t_1 < \cdots < t_n \leq 1$ 和 $x_i \in \mathcal{S}$ $(i \in [n])$，所有联合概率 $\mathbb{P}(X_{t_1} = x_1, \ldots, X_{t_n} = x_n)$ 都由这种方式定义.

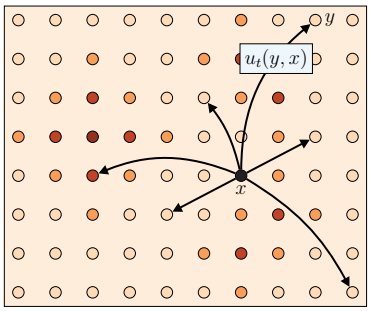


图 13 CTMC 模型通过指定状态之间的概率转移速率（速度）来定义.

为确保转移概率 $p_{t+h|t}(y|x)$ 可以通过 (6.3) 定义，速度需要满足以下**速率条件**：

$$u_t(y, x) \geq 0 \text{ 对所有} y \neq x \text{ 成立，且} \sum_y u_t(y, x) = 0. \tag{6.4}$$

若其中任一条件不成立，则对任意小的 $h > 0$，转移概率 $p_{t+h|t}(\cdot|x)$ 会出现负值，或者其和为 $c \neq 1$. 式 (6.3) 的作用，与定义流生成模型时 式 (3.16) 和 式 (3.19) 的作用相同. 过程 X_t 在时间 $t \in [0, 1]$ 的边缘概率由 PMF $p_t(x)$ 表示. 于是，与流的情形中的 式 (3.24) 类似，我们规定

$$u_t \textbf{ 生成 } p_t, \text{ 若存在} p_{t+h|t} \text{ 满足(6.3) 且边缘分布为} p_t. \tag{6.5}$$

模拟 *CTMC*. 为采样 X_t，先采样 $X_0 \sim p$，再使用**（朴素）欧拉方法**逐步推进：

$$\mathbb{P}(X_{t+h} = y \mid X_t) = \delta(y, X_t) + hu_t(y, X_t). \tag{6.6}$$

根据 (6.3)，这些步骤会给更新概率引入 $o(h)$ 的误差. 实践中，这意味着需要足够小的 $h > 0$，以确保 (6.6) 的右端仍然是有效的 PMF. 一种可能的补救办法是使用如下**欧拉方法**，它确保任

意 $h > 0$ 都得到有效的 PMF，同时保持概率的 $o(h)$ 局部误差：

$$\mathbb{P}(X_{t+h} = y \mid X_t) = \begin{cases} \exp\left[h u_t(X_t, X_t)\right] & y = X_t \\ \frac{u_t(y, X_t)}{|u_t(X_t, X_t)|}\left(1 - \exp\left[h u_t(X_t, X_t)\right]\right) & y \neq X_t \end{cases}. \tag{6.7}$$

6.3 概率路径与柯尔莫哥洛夫方程

与连续情形下的连续性方程类似，CTMC 模型 $(X_t)_{0 \le t \le 1}$ 的边缘概率 p_t 由**柯尔莫哥洛夫方程**刻画：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(y) = \sum_x u_t(y, x) p_t(x). \tag{6.8}$$

下面的经典定理（另见 Coddington et al. (1956) 中的定理 5.1 和 5.2）描述了这一线性齐次 ODE 方程组唯一解的存在性.

定理 12 (线性 ODE 的存在性与唯一性)**.** *若 $u_t(y, x)$ 属于 $C([0,1))$（关于时间连续），则柯尔莫哥洛夫方程 (6.8) 在 $t \in [0,1)$ 上存在唯一解 $p_t(x)$，并满足 $p_0(x) = p(x)$.*

对于 CTMC，可以保证解在所有时间 $t \in [0,1)$ 上存在，无需额外条件（这不同于 定理 1 中的非线性情形）. 柯尔莫哥洛夫方程与连续性方程 (3.25) 有密切联系. 利用速率条件重新整理 (6.8) 的右端，得到

$$\begin{aligned} \sum_x u_t(y, x) p_t(x) &\overset{(6.4)}{=} \overbrace{\sum_{x \neq y} u_t(y, x) p_t(x)}^{\text{流入通量}} - \overbrace{\sum_{x \neq y} u_t(x, y) p_t(y)}^{\text{流出通量}} \\ &= -\sum_{x \neq y} \left[j_t(x, y) - j_t(y, x) \right], \end{aligned}$$

其中 $j_t(y, x) \coloneqq u_t(y, x) p_t(x)$ 是概率通量，描述单位时间内从状态 x 移动到状态 y 的概率. 流出通量的净余量定义为**散度**，从而使柯尔莫哥洛夫方程具有与 小节 3.5 中连续性方程相同的结构 (Gat et al., 2024).

以下结果是在 CTMC 框架中构造概率路径与速度的主要工具：

定理 13 (离散质量守恒)**.** *设 $u_t(y, x)$ 属于 $C([0,1))$，$p_t(x)$ 是关于时间 t 属于 $C^1([0,1))$ 的 PMF. 则以下命题等价：*

1. *p_t, u_t 对 $t \in [0,1)$ 满足柯尔莫哥洛夫方程 (6.8)，且 u_t 满足速率条件 (6.4).*
2. *u_t 对 $t \in [0,1)$ 在 6.5 的意义下生成 p_t.*

定理 13 的证明见 小节 A.1.

6.3.1 保持概率的速度

由离散质量守恒（定理 13）可知，若速度 $u_t(y, x)$ 生成概率路径 $p_t(x)$，则

$$\tilde{u}_t(y, x) = u_t(y, x) + v_t(y, x) \text{ 生成} p_t(x), \tag{6.9}$$

只要 $v_t(y,x)$ 满足速率条件 (6.4)，并满足**无散速度**方程

$$\sum_x v_t(y,x)p_t(x) = 0. \tag{6.10}$$

事实上，$\tilde{u}_t(y,x)$ 满足柯尔莫哥洛夫方程：

$$\sum_x \tilde{u}_t(y,x)p_t(x) = \sum_x u_t(y,x)p_t(x) = \dot{p}_t(y),$$

这表明，可以在采样过程中加入无散速度而不改变边缘概率. 这一事实在从下面将要介绍的离散流匹配模型中采样时很有用.

7 离散流匹配

值得关注的是，图 2 中的流匹配蓝图可以从连续情形无缝迁移到离散情形，从而得到**离散流匹配（DFM）**框架 (Campbell et al., 2024; Gat et al., 2024). 与连续情形类似，首先定义一条在源 PMF p 与目标 PMF q 之间插值的概率路径 p_t. 其次，希望找到由可学习速度 u_t^θ 定义的 CTMC 模型 $(X_t)_{0\le t\le 1}$，使其生成概率路径 p_t. 最后，通过最小化定义离散流匹配损失的 Bregman 散度来训练 u_t^θ. 总而言之，这就是求解流匹配问题 (4.1) 的离散版本.

7.1 数据与耦合

我们的目标是将来自源 PMF p 的样本 $X_0 \sim p$ 转换为来自目标 PMF q 的样本 $X_1 \in q$，其中 $X_0, X_1 \in \mathcal{S}$ 是两个各自取值于状态空间 $\mathcal{S}$ 的随机变量. 源样本与目标样本可以通过独立耦合 $(X_0, X_1) \sim p(X_0)q(X_1)$ 联系起来，也可以通过一般的 PMF 耦合 $\pi_{0,1}(x_0,x_1)$ 关联. 例如，文本翻译数据使用耦合数据 (x_0,x_1)，表示以两种不同语言书写的同一文档. 另一些应用，例如文本生成，使用独立配对；此时 $p(x_0)$ 可以是 $\mathcal{S}$ 上的均匀分布，赋予所有状态相同概率，也可以在词表 $\mathcal{T}$ 中加入特殊词元 $\mathtt{m}$，即 $\mathcal{T} \cup \{\mathtt{m}\}$，并取 $\pi_{0,1}(x_0,x_1) = \delta(x_0,\mathtt{m})q(x_1)$. 任意随机变量 $X_0 \sim \delta(X_0,\mathtt{m})$ 都是常随机变量 $X_0 = (\mathtt{m},\dots,\mathtt{m})$.

7.2 离散概率路径

与往常一样，FM 方法的下一步是指定在 p 与 q 之间插值的概率路径 p_t. 遵循 小节 4.4，我们以取值于任意空间 $\mathcal{Z}$ 的一般条件随机变量 $Z \sim p_Z$ 为条件来定义这些对象. **边缘概率路径**具有如下形式：

$$p_t(x) = \sum_{z\in\mathcal{Z}} p_{t|Z}(x|z)p_Z(z), \tag{7.1}$$

其中 $p_{t|Z}(\cdot|z)$ 是条件 PMF，边缘概率路径满足边界约束 $p_0 = p$ 和 $p_1 = q$.

7.3 边缘化技巧

边缘化技巧（见 小节 4.4）可以原样迁移到离散情形 (Campbell et al., 2024; Gat et al., 2024). 假设**条件速度场** $u_t(\cdot,\cdot|z)$ 在 (6.5) 的意义下生成 $p_t(\cdot|z)$，则得到**边缘速度场**

$$u_t(y,x) = \sum_z u_t(y,x|z)p_{Z|t}(z|x) = \mathbb{E}\left[u_t(y,X_t|Z) \mid X_t = x\right], \tag{7.2}$$

它对所有满足 $p_t(x) > 0$ 的 $x, y \in \mathcal{S}$ 有定义，其中随机变量 $X_t \sim p_{t|Z}(\cdot|Z)$. 使用 Bayes 公式，得到

$$p_{Z|t}(z|x) = \frac{p_{t|Z}(x|z)p_Z(z)}{p_t(x)}. \tag{7.3}$$

为证明边缘化技巧定理（定理 3）的离散版本，作如下假设：

假设 3. $p_{t|Z}(x|z) \in C^1([0,1))$，$u_t(y,x|z) \in C([0,1))$，且对所有 $x \in \mathcal{S}$ 和 $t \in [0,1)$，都有 $p_t(x) > 0$.

与连续情形一样，假设 $p_t > 0$ 在实践中是温和的，因为我们总可以使用 $(1-(1-t)\epsilon) \cdot p_{Z|t} + (1-t)\epsilon \cdot p_{\text{uni}}$，其中 p_{uni} 是 $\mathcal{S}$ 上的均匀分布，而 $\epsilon > 0$ 可以任意小. 现在可以陈述并证明这一结果.

定理 14 (离散边缘化技巧)**.** *在假设 3 下，若 $u_t(y,x|z)$ 生成 $p_{t|Z}(x|z)$，则 (7.2) 中的边缘速度 $u_t(y,x)$ 对 $t \in [0,1)$ 生成 (7.1) 中的 $p_t(x)$.*

证明. 证明的思路与连续情形类似. 首先计算：

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}t}p_t(y) &= \sum_z \frac{\mathrm{d}}{\mathrm{d}t}p_{t|Z}(y|z)p_Z(z) \\
&\overset{(i)}{=} \sum_z \left[\sum_x u_t(y,x|z)p_{t|Z}(x|z)\right] p_Z(z) \\
&\overset{(ii)}{=} \sum_x \left[\sum_z u_t(y,x|z)\frac{p_{t|Z}(x|z)p_Z(z)}{p_t(x)}\right] p_t(x) \\
&\overset{(Bayes)}{=} \sum_x \overbrace{\sum_z u_t(y,x|z)p_{Z|t}(z|x)}^{u_t(y,x)} p_t(x),
\end{aligned}$$

等式 (i) 由 定理 13 以及 $u_t(y,x|z)$ 生成 $p_{t|Z}(y|z)$ 这一事实得到. 等式 (ii) 由同乘再同除以假设为正的 $p_t(x)$ 得到. 因此，$u_t(y,x)$ 与 p_t 满足柯尔莫哥洛夫方程. 此外，$u_t(y,x)$ 满足速率条件 (6.4)，因为每个 $u_t(y,x|z)$ 都满足这些条件. 最后，由于 $u_t(y,x|z)$ 和 $p_{Z|t}(z|x)$ 均属于 $C([0,1))$，所以 $u_t(y,x) \in C([0,1))$. 特别地，由对 $t \in [0,1)$ 假设 $p_t(x) > 0$，可得 $p_{Z|t}(z|x) \in C([0,1))$. 根据 定理 13，由于 $u_t(x,y)$ 与 p_t 满足柯尔莫哥洛夫方程，并满足速率条件，所以它在 (6.5) 的意义下生成 p_t. □

7.4 离散流匹配损失

为构造 CTMC 生成模型 $(X_t)_{0\le t\le 1}$，我们使用参数 θ 对速度场 $u_t^\theta(y,x)$ 进行参数化，例如使用神经网络. 神经网络应构造为满足速率条件 式 (6.4). 用于训练 CTMC 模型的**离散流匹配损失**定义为：

$$\mathcal{L}_{\text{DFM}}(\theta) = \mathbb{E}_{t,X_t\sim p_t} D_{X_t}(u_t(\cdot,X_t), u_t^\theta(\cdot,X_t)), \tag{7.4}$$

其中 $t \sim U[0,1]$，$u_t(\cdot, x) \in \mathbb{R}^{\mathcal{S}}$ 满足速率条件. 这意味着 $u_t(\cdot, x) \in \Omega_x$，其中

$$\Omega_x = \left\{ v \in \mathbb{R}^{\mathcal{S}} \;\middle|\; v(y) \geq 0 \; \forall y \neq x, \; \text{且} v(x) = -\sum_{y \neq x} v(y) \right\} \subset \mathbb{R}^{\mathcal{S}}, \tag{7.5}$$

是凸集，而 $D_x(u, v)$ 是使用凸函数 $\Phi_x : \Omega_x \to \mathbb{R}$ 定义的 Bregman 散度. **条件离散流匹配损失**具有如下形式：

$$\mathcal{L}_{\text{CDFM}}(\theta) = \mathbb{E}_{t, Z, X_t \sim p_{t|Z}} D_{X_t}(u_t(\cdot, X_t|Z), u_t^\theta(\cdot, X_t)). \tag{7.6}$$

再次，两个损失 (7.4) 和 (7.6) 都给出相同的学习梯度.

定理 15. *离散流匹配损失与条件离散流匹配损失的梯度一致：*

$$\nabla_\theta \mathcal{L}_{DFM}(\theta) = \nabla_\theta \mathcal{L}_{CDFM}(\theta). \tag{7.7}$$

特别地，条件离散匹配损失的极小化解为边缘速度

$$u_t^\theta(y, x) = \mathbb{E}\left[u_t(y, X_t|Z) \mid X_t = x\right]. \tag{7.8}$$

令 $X = X_t$、$Y = (X_t, Z)$，将 $f : \mathcal{S}^2 \to \mathbb{R}^{\mathcal{S}}$ 定义为 $(x, z) \mapsto u_t(\cdot, x|z) \in \mathbb{R}^{\mathcal{S}}$，应用 命题 1 并关于 $t \in [0,1]$ 积分，即可得到证明.

7.5 因子化路径与速度

若按前述方式实现 DFM，就需要一个可学习模型 $u_t^\theta(y, x)$，例如神经网络，为所有可能的状态 $y \in \mathcal{S} = \mathcal{T}^d$ 输出速率. 这将导致庞大的输出维数 K^d，对于常见序列长度 d 和词表大小 K 而言不可行. 解决这一问题的一种方法是考虑**因子化速度** (Campbell et al., 2022)：

$$u_t(y, x) = \sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x), \tag{7.9}$$

其中 $\bar{i} = (1, \ldots, i-1, i+1, \ldots, d)$ 表示除 i 之外的所有指标. 因此，上述因子化速度只在状态 x 与状态 y 至多有一个词元不同时将它们连接起来. 使用因子化速度时，只需对 $u_t^i(y^i, x)$ 建模，因为它们完全定义了 $u_t(y, x)$. 于是，每个 $u_t^i(y^i, x)$ 都是接受 $x \in \mathcal{S}$ 的可学习模型，对所有 $i \in [d] = \{1, 2, \ldots, d\}$ 和 $y^i \in \mathcal{T}$ 返回标量 $u_t^i(y^i, x) \in \mathbb{R}$. 因此，模型输出的维数为可处理的 $d \cdot K$. 因子化速度 $u_t^i(y, x)$ 的速率条件现在要求在每个维度 $i \in [d]$ 上满足：

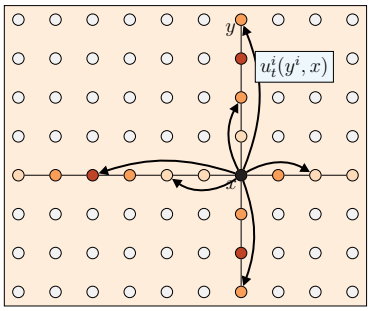


图 14 因子化 CTMC 模型仅允许在至多一个坐标（词元）不同的状态之间具有非零速率（速度）.

$$u_t^i(y^i, x) \geq 0 \text{ 对所有} y^i \neq x^i \text{ 成立，且} \sum_{y^i \in \mathcal{T}} u_t^i(y^i, x) = 0 \quad \text{对所有} x \in \mathcal{S} \text{ 成立.} \tag{7.10}$$

7.5.1 使用因子化速度模拟 CTMC

使用因子化速度时，可以逐坐标采样 CTMC 模型 (Campbell et al., 2024)：

$$\begin{aligned}\mathbb{P}(X_{t+h} = y \mid X_t = x) &= \delta(y, x) + h \sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x) + o(h) \\ &= \prod_i \left[\delta(y^i, x^i) + h u_t^i(y^i, x) + o(h)\right],\end{aligned}$$

其中第二个等式由 $\delta(y, x) = \prod_i \delta(y^i, x^i)$ 以及恒等式

$$\prod_i \left[a^i + h b^i\right] = \prod_i a^i + h \sum_i (\prod_{j \neq i} a^j) b^i + o(h).$$

得到. 因此，在 $o(h)$ 阶误差以内，转移核因子化为逐坐标的独立转移：

$$\mathbb{P}(X_{t+h}^i = y^i \mid X_t = x) = \delta(y^i, x^i) + h u^i(y^i, x) + o(h). \tag{7.11}$$

这些转移可以在每个坐标上用欧拉方法 (6.7) 进行采样. 有趣的是，连续流匹配也具有类似的因子化 $u_t(x) = [u_t^1(x), \ldots, u_t^d(x)] \in \mathbb{R}^d$，其中 $\dot{X}_t^i(x) = u_t^i(X_t)$ 决定坐标 i 的变化，并可以独立采样（连续 FM 中的 "样本" 实际上是确定性的）.

7.5.2 构造具有因子化速度的概率路径

若以特定方式构造概率路径，就会由构造本身得到因子化速度（式 (7.9)）. 下面解释这一构造. 为此，将**因子化概率路径**定义为如下形式的概率路径：

$$q_t(x) = \prod_i q_t^i(x^i). \tag{7.12}$$

下面的结果表明，这些因子化概率路径具有因子化速度.

命题 2. 设 $q_t(x)$ 是 (7.12) 中的因子化概率路径，其中 $u_t^i(y^i, x^i) \in C([0, 1))$ 生成 $q_t^i(x^i)$. 则 q_t 具有如下形式的因子化生成速度：

$$u_t(y, x) = \sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x^i). \tag{7.13}$$

为进行证明，将 PMF $q(x)$ 的边缘分布记为

$$q^i(x^i) := \sum_{x^{\bar{i}}} q(x) \quad q^{\bar{i}}(x^{\bar{i}}) := \sum_{x^i} q(x) \tag{7.14}$$

证明. 设 q_t 是 (7.12) 中的因子化概率路径. 设 $u_t^i(y^i, x^i)$ 是 $q_t^i(x^i)$ 的生成速度. 关于 t 求导得到

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}t}q_t(y) &= \sum_i q_t^{\bar{i}}(y^{\bar{i}})\frac{\mathrm{d}}{\mathrm{d}t}q_t^i(y^i)\\ &\overset{(i)}{=} \sum_i \left[\sum_{x^{\bar{i}}} \delta(y^{\bar{i}}, x^{\bar{i}}) q_t^{\bar{i}}(x^{\bar{i}})\right]\left[\sum_{x^i} u_t^i(y^i, x^i) q_t^i(x^i)\right]\\ &\overset{(ii)}{=} \sum_x \left[\sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x^i)\right] q_t(x),\end{aligned}$$

等式 (i) 由 $q_t^{\bar{i}}(y^{\bar{i}}) = \sum_{x^{\bar{i}}} \delta(y^{\bar{i}}, x^{\bar{i}}) q_t^{\bar{i}}(x^{\bar{i}})$ 以及柯尔莫哥洛夫方程 (6.8) 得到. 等式 (ii) 由交换求和次序得到; 同时注意, 根据 q_t 的定义, 有 $q_t^{\bar{i}}(x^{\bar{i}}) q_t^i(x^i) = q_t(x)$ 和 $\sum_{x^{\bar{i}}}\sum_{x^i} = \sum_x$. □

现在可以给出构造具有因子化速度、在任意 p 与 q 之间插值的路径 p_t 的主要工具 (Campbell et al., 2024; Gat et al., 2024).

定理 16 (离散因子化边缘化技巧)**.** 考虑通过如下方式构造的边缘概率路径:

$$p_t(x) = \sum_z p_{t|Z}(x|z) p_Z(z), \text{ 其中 } p_{t|Z}(x|z) = \prod_i p_{t|Z}^i(x^i|z), \tag{7.15}$$

即条件路径在 式 (7.12) 的意义下因子化. 进一步假设 $u_t^i(y^i, x^i|z)$ 属于 $C([0,1))$, 生成属于 $C^1([0,1))$ 的 $p_{t|Z}^i(x^i|z)$, 并且对所有 $x \in \mathcal{S}$ 和 $t \in [0,1)$ 有 $p_t(x) > 0$. 则边缘速度

$$u_t(y, x) = \sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x) \tag{7.16}$$

其中

$$u_t^i(y^i, x) = \sum_z u_t^i(y^i, x^i|z) p_{Z|t}(z|x) = \mathbb{E}\left[u_t^i(y^i, X_t^i|Z)|X_t = x\right] \tag{7.17}$$

生成 $p_t(x)$.

证明. 根据 命题 2, 因子化条件路径 $p_{t|Z}(x|z)$ 具有因子化生成速度

$$u_t(y, x|z) = \sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x^i|z).$$

因此,

$$\begin{aligned}u_t(y, x) &\overset{(i)}{=} \sum_z u_t(y, x|z) p_{Z|t}(z|x)\\ &\overset{(ii)}{=} \sum_z \left[\sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) u_t^i(y^i, x^i|z)\right] p_{Z|t}(x|z)\\ &\overset{(iii)}{=} \sum_i \delta(y^{\bar{i}}, x^{\bar{i}}) \left[\sum_z u_t^i(y^i, x^i|z) p_{Z|t}(z|x)\right].\end{aligned}$$

等式 (i) 由 (7.2) 得到. 等式 (ii) 由 $p_{t|Z}$ 具有因子化速度这一假设得到. 等式 (iii) 由交换求和次序得到. 由于 $p_{t|Z}^i(x^i|z) \in C^1([0,1))$ 且 $p_t(x) > 0$, 所以 $p_{t|Z}(x|z) \in C^1([0,1))$. 类似地, 由于

$u_t^i(y^i, x^i|z) \in C([0,1))$，所以 $u_t(y, x|z) \in C([0,1))$. 因此，定理 14 表明 $u_t(y,x)$ 生成 $p_t(x)$，这正是所要证明的结论. □

使用 定理 16，可以如下设计具有因子化速度、在源 PMF p 与目标 PMF q 之间插值的概率路径 p_t.

1. 找到因子化条件概率路径 $p_{t|Z}(x|z) = \prod_i p_{t|Z}^i(x^i|z)$，使边缘分布 $p_t(x)$ 满足 $p_0 = p$ 和 $p_1 = q$.
2. 找到 $p_{t|Z}^i(x^i|z)$ 的生成速度 $u_t^i(y^i, x^i|z)$. 这可以通过求解柯尔莫哥洛夫方程的解 $u_t^i(y^i, x^i|z)$ 来实现：

$$\sum_{x^i} u_t^i(y^i, x^i|z) p_{t|Z}^i(x^i|z) = \frac{\mathrm{d}}{\mathrm{d}t} p_{t|Z}^i(y^i|z), \tag{7.18}$$

对所有 $y^i \in \mathcal{T}$、固定的 $i \in [d], z \in \mathcal{Z}$ 和 $t \in [0,1)$ 均成立. 附带指出，(7.18) 是具有 $|\mathcal{T}|$ 个未知量的欠定线性方程组（未知量个数显著少于整个状态空间 $|\mathcal{S}|$）.

7.5.3 因子化速度的条件离散流匹配损失

用因子化速度 $u_t^{\theta,i}$ 表示边缘速度 u_t^θ，可得到如下条件流匹配损失：

$$\mathcal{L}_{\text{CDFM}}(\theta) = \mathbb{E}_{t,Z,X_t \sim p_{t|Z}} \sum_i D_{X_t}^i \left(u_t^i(\cdot, X_t|Z), u_t^{\theta,i}(\cdot, X_t) \right), \tag{7.19}$$

其中 $t \sim U[0,1]$，并且 $u_t^i(\cdot, x|z), u_t^{\theta,i}(\cdot, x) \in \mathbb{R}^{\mathcal{T}}$ 满足速率条件. 这意味着 $u_t^i(\cdot, x|z), u_t^{\theta,i}(\cdot, x) \in \Omega_{x^i}$；对于 $\alpha \in \mathcal{T}$，定义

$$\Omega_\alpha = \left\{ v \in \mathbb{R}^{\mathcal{T}} \;\middle|\; v(\beta) \geq 0 \; \forall \beta \in \mathcal{T} \setminus \{\alpha\}, \; \text{且} v(\alpha) = -\sum_{\beta \neq \alpha} v(\beta) \right\} \subset \mathbb{R}^{\mathcal{T}}. \tag{7.20}$$

这是凸集，$D_x^i(u,v)$ 是由凸函数 $\Phi_x^i : \Omega_{x^i} \to \mathbb{R}$ 定义的 Bregman 散度. 与之前一样，令 $X = X_t$、$Y = u_t^i(\cdot, X_t, Z) \in \mathbb{R}^{\mathcal{T}}$，取 $D_x^i(u,v)$ 为 $\Omega_{x^i} \subset \mathbb{R}^{\mathcal{T}}$ 上的 Bregman 散度，使用 命题 1 并关于 $t \in [0,1]$ 积分，即可说明这一损失的合理性.

7.5.4 混合路径

现在应用 小节 7.5.2 中的方法，构造实用的概率路径及其对应的条件速度. 遵循 Gat et al. (2024)，我们以 $Z = (X_0, X_1)$ 为条件，以适应任意的数据耦合 $(X_0, X_1) \sim \pi_{0,1}(X_0, X_1)$. 于是，将因子化条件路径

$$p_{t|0,1}(x|x_0, x_1) = \prod_i p_{t|0,1}^i(x^i|x_0, x_1) \tag{7.21}$$

构造为混合形式：

$$p_{t|0,1}^i(x^i|x_0, x_1) = \kappa_t \delta(x^i, x_1^i) + (1 - \kappa_t)\delta(x^i, x_0^i), \tag{7.22}$$

其中 $\kappa : [0,1] \to [0,1]$ 是属于 $C^1([0,1])$ 的调度函数. 注意，随机变量 $X_t^i \sim p_{t|0,1}^i(\cdot|x_0, x_1)$ 满足

$$X_t^i = \begin{cases} x_1^i & \text{以概率} \kappa_t \\ x_0^i & \text{以概率} (1-\kappa_t) \end{cases}, \tag{7.23}$$

即它以依赖于时间 t 的概率取源状态或目标状态.

若 $\kappa_0 = 0$ 且 $\kappa_1 = 1$，则 (7.1) 中的边缘分布 $p_t(x)$ 满足边界约束. 我们还需要 $p_{t|0,1}^i(x^i|x_0, x_1)$ 的生成速度 $u_t^i(y^i, x^i|x_0, x_1)$，它们是 (7.18) 的解. 推导如下：

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}t} p_{t|Z}^i(y^i|z) &\overset{(7.22)}{=} \dot{\kappa}_t \left[\delta(y^i, x_1^i) - \delta(y^i, x_0^i)\right] \\ &\overset{(7.22)}{=} \dot{\kappa}_t \left[\delta(y^i, x_1^i) - \frac{p_{t|Z}^i(y^i|z) - \kappa_t \delta(y^i, x_1^i)}{1-\kappa_t}\right] \\ &= \frac{\dot{\kappa}_t}{1-\kappa_t} \left[\delta(y^i, x_1^i) - p_{t|Z}^i(y^i|z)\right] \\ &= \sum_{x^i} \frac{\dot{\kappa}_t}{1-\kappa_t} \left[\delta(y^i, x_1^i) - \delta(y^i, x^i)\right] p_{t|Z}^i(x^i|z), \end{aligned}$$

其中，为保持记号简洁，我们交替使用 $z = (x_0, x_1)$ 和 $Z = (X_0, X_1)$. 因此，找到了生成 (7.22) 中路径的条件速度，即

$$u_t^i(y^i, x^i|x_0, x_1) = \frac{\dot{\kappa}_t}{1-\kappa_t} \left[\delta(y^i, x_1^i) - \delta(y^i, x^i)\right]. \tag{7.24}$$

小节 7.5.4 展示了如何在库 `flow_matching` 程序库 中定义混合路径.

代码 9: 离散概率路径

```python
import torch
from flow_matching.path import MixtureDiscreteProbPath
from flow_matching.path.path_sample import DiscretePathSample

# 创建离散概率路径对象
path = MixtureDiscreteProbPath(scheduler=PolynomialConvexScheduler(n=1.0))

# 对条件路径采样
# t、X_0 和 X_1 的 batch_size = 2
t = torch.tensor([0.25, 0.5])
x_0 = torch.tensor([0, 0])
x_1 = torch.tensor([1, 2])
sample: DiscretePathSample = path.sample(t=t, x_0=x_0, x_1=x_1)
sample.x_0  # X_0 为 [0, 0]
sample.x_1  # X_1 为 [1, 2]
# X_t 为
#   [以概率 0.75 为 0, 以概率 0.25 为 1,
#    以概率 0.5 为 0, 以概率 0.5 为 2]
sample.x_t
sample.t  # t 为 [0.25, 0.5]
```

速度的后验参数化. 与连续情形类似（例如 小节 4.8.1），可以选择不同方式来参数化速度 $u_t^i(y^i, x)$. 第一种方法是直接参数化，与流中的速度类似. 另一种方法，也是此处采用的方法，

其动机来自按照 (7.17) 对**混合边缘速度**进行的如下计算：

$$\begin{aligned} u_t^i(y^i, x) &= \sum_{x_0, x_1} \frac{\dot\kappa_t}{1-\kappa_t} \left[\delta(y^i, x_1^i) - \delta(y^i, x^i)\right] p_{0,1|t}(x_0, x_1|x) \\ &= \sum_{x_1^i} \frac{\dot\kappa_t}{1-\kappa_t} \left[\delta(y^i, x_1^i) - \delta(y^i, x^i)\right] p_{1|t}^i(x_1^i|x), \end{aligned} \tag{7.25}$$

其中，在第二个等式中，我们将后验 $p_{0,1|t}$ 的边缘分布记为

$$p_{1|t}^i(x_1^i|x) = \sum_{x_0, x_1^{\bar i}} p_{0,1|t}(x_0, x_1|x) \tag{7.26}$$

$$= \mathbb{E}\left[\delta(x_1^i, X_1^i) \mid X_t = x\right]. \tag{7.27}$$

这一推导使用可学习后验 $p_{1|t}^{\theta,i}(x_1^i|x)$ 表示边缘速度 $u_t^i(y^i, x)$，可理解为 x_1 预测（小节 4.8.1）的离散版本. 接下来探讨学习这一后验的损失函数.

混合路径的 CDFM 损失 我们给出学习 $p_{1|t}^{\theta,i}(x_1^i|x)$ 的两种选择，两者的合理性均由 命题 1 保证. 首先，可以用条件匹配损失学习 (7.26) 和 (7.27) 中的边缘后验：

$$\mathcal{L}_{\text{CM}}(\theta) = \mathbb{E}_{t, X_0, X_1, X_t} D_{X_t}\left(\delta(\cdot, X_1^i), p_{1|t}^{\theta,i}(\cdot|X_t)\right) \tag{7.28}$$

由于 $\delta(\cdot, X_1^i), p_{1|t}^{\theta,i}(\cdot|X_t)$ 都是 PMF，因此可以将 Bregman 散度取为比较 PMF 的 KL 散度 $D(p,q) = \sum_{\alpha\in\mathcal{T}} p(\alpha)\log\frac{p(\alpha)}{q(\alpha)}$，得到

$$\mathcal{L}_{\text{CM}}(\theta) = -\mathbb{E}_{t, X_0, X_1, X_t} \log p_{1|t}^{\theta,i}(X_1^i|X_t) + \text{常数}. \tag{7.29}$$

另一种选择是遵循 小节 7.5.3，使用 (7.19) 中的因子化损失，并以 $p_{1|t}^{\theta,i}$ 参数化 $u_t^{\theta,i}$. 此时，可以将 Bregman 散度取为比较一般向量（不一定是概率向量）$u, v \in \mathbb{R}^m_{\geq 0}$ 的广义 KL 散度：

$$D(u, v) = \sum_j u^j \log\frac{u^j}{v^j} - \sum_j u_j + \sum_j v_j. \tag{7.30}$$

对于这一 D 的选择，得到

$$D\left(u_t^i(\cdot, x^i|x_0, x_1), u_t^{\theta,i}(\cdot, x)\right) = \frac{\dot\kappa_t}{1-\kappa_t}\left[(\delta(x_1^i, x^i) - 1)\log p_{1|t}^{\theta,i}(x_1^i|x) + \delta(x_1^i, x^i) - p_{1|t}^{\theta,i}(x^i|x)\right] \tag{7.31}$$

当以 $Z = (X_0, X_1)$ 为条件时，这便实现了损失 (7.19). 广义 KL 损失 (7.31) 还给出目标分布似然的证据下界（ELBO）(Shaul et al., 2024)：

$$-\log p_1^\theta(x_1) \leq \mathbb{E}_{t, X_0, X_t \sim p_{t|0,1}} \sum_i D\left(u_t^i(\cdot, X_t^i|X_0, x_1), u_t^{\theta,i}(\cdot, X_t)\right), \tag{7.32}$$

其中 p_1^θ 是模型在时间 $t = 1$ 生成的边缘分布. 因此，除了训练之外，广义 KL 损失也常用于评估.

混合路径的采样　基于后验 $p^{\theta,i}_{1|t}$ 的参数化给出如下采样算法. 如 小节 7.5.1 所述，使用因子化速度能够实现逐坐标采样 (7.11). 根据 (7.17) 和 (7.25)，

$$\mathbb{P}(X^i_{t+h} = y^i \mid X_t = x) = \delta(y^i, x^i) + hu^i(y^i, x) + o(h) \tag{7.33}$$

$$= \sum_{x^i_1} \left[\delta(y^i, x^i) + h\frac{\dot\kappa_t}{1-\kappa_t}\left[\delta(y^i, x^i_1) - \delta(y^i, x^i)\right] + o(h)\right] p^i_{1|t}(x^i_1|x). \tag{7.34}$$

因此，给定 $X_t = x$，可以对每个 $i \in [d]$ 执行以下两个步骤来完成一步采样：

(i) 采样 $X^i_1 \sim p^i_{1|t}(X^i_1|x)$.

(ii) 根据 (6.7) 中的欧拉步，以速度 $\frac{\dot\kappa_t}{1-\kappa_t}\left[\delta(y^i, X^i_1) - \delta(y^i, x^i)\right]$ 更新 X^i_{t+h}. 直观地说，第二步决定是令 $X^i_{t+h} = X^i_1$，还是保持 $X^i_{t+h} = X^i_t$.

单侧混合路径与保持概率的速度　如 小节 6.3.1 所述，通过加入无散分量来扩展采样算法的设计空间通常很有用. 对于因子化路径，无散速度 v^i_t 需要满足 (7.18)，即

$$\sum_{x^i} v^i_t(y^i, x^i|z) p^i_{t|Z}(x^i|z) = 0. \tag{7.35}$$

一般而言，若不学习额外的量（例如 $p^i_{0|t}$），可能很难找到这样的保持概率的速度. 然而，一种能够找到闭式保持概率速度的有用情形是，假设源分布独立同分布，即 $p(x) = \prod_i p(x^i)$，且耦合独立，即 $\pi_{0,1}(x_0, x_1) = p(x_0)q(x_1)$. 此时，边缘混合路径具有如下形式：

$$p_t(x) = \sum_{x_1} p_{t|1}(x|x_1)q(x_1), \text{ 其中} p_{t|1}(x|x_1) = \prod_i p^i_{t|1}(x^i|x_1),$$

其中 $p^i_{t|1}(x^i|x_1) = \left[\kappa_t\delta(x^i, x^i_1) + (1-\kappa_t)p(x^i)\right]$. (7.24) 中的条件速度，即

$$u^i_t(y^i, x^i|x_1) = \frac{\dot\kappa_t}{1-\kappa_t}\left[\delta(y^i, x^i_1) - \delta(y^i, x^i)\right] \tag{7.36}$$

也生成 $p^i_{t|1}(x^i|x_1)$. 为找到无散速度，例如可以从这一速度中减去 $p^i_{t|1}(y^i, x^i|x_1)$ 的一个反向时间速度 $\tilde u^i_t$ (Gat et al., 2024)，其含义是：它与 $p^i_{t|1}(y^i, x^i|x_1)$ 满足柯尔莫哥洛夫方程，且 $-\tilde u^i_t$ 满足速率条件. 可以用与 式 (7.24) 类似的方式找到这样的速度：

$$\tilde u^i_t(y^i, x^i|x_1) = \frac{\dot\kappa_t}{\kappa_t}\left[\delta(y^i, x^i) - p(x^i)\right]. \tag{7.37}$$

因此，可以如下定义条件路径 $p^i_{t|1}(x^i|x_1)$ 的无散速度：

$$v^i_t(y^i, x^i|x_1) = u^i_t(y^i, x^i|x_1) - \tilde u^i_t(y^i, x^i|x_1). \tag{7.38}$$

根据 小节 6.3.1，若将无散场 $v^i_t(y^i, x^i|x_1)$ 加到速度 $u^i_t(y^i, x^i|x_1)$ 上，后者仍生成相同的概率路

径 $p^i_{t|1}(x^i|x_1)$. 因此，定理 16 表明，由下式定义的边缘速度 $u^i_t(y^i, x)$

$$\begin{aligned} u^i_t(y^i, x) &= \sum_{x_1} \left[u^i_t(y^i, x^i|x_1) + c_t v^i_t(y^i, x^i|x_1)\right] p_{1|t}(x_1|x) \\ &= \sum_{x^i_1} \left[u^i_t(y^i, x^i|x^i_1) + c_t v^i_t(y^i, x^i|x^i_1)\right] p^i_{1|t}(x^i|x), \end{aligned}$$

仍生成相同的边缘路径 $p_t(x)$；其中第二个等式来自混合路径满足 $u^i_t(y^i, x^i|x_1) = u^i_t(y^i, x^i|x^i_1)$，且 $v^i_t(y^i, x^i|x^i_1)$ 也满足类似关系. 因此，给定 $X_t = x$，广义采样算法的单步包含以下步骤：

(i) 采样 $X^i_1 \sim p^i_{1|t}(X^i_1|x)$.

(ii) 以如下速度执行 (6.7) 中的欧拉步：

$$\begin{aligned} u^i_t(y^i, x^i|x_1) = &\frac{\dot\kappa_t}{1-\kappa_t}\left[\delta(y^i, X^i_1) - \delta(y^i, x^i)\right] \\ &+ c_t\left[\frac{\dot\kappa_t}{1-\kappa_t}\left[\delta(y^i, x^i_1) - \delta(y^i, x^i)\right] - \frac{\dot\kappa_t}{\kappa_t}\left[\delta(y^i, x^i) - p(x^i)\right]\right], \end{aligned}$$

其中 $c_t > 0$ 是依赖时间的常数.

与 节 2 中的连续流匹配示例类似，我们在 小节 7.5.4 中给出了纯 PyTorch 的离散流匹配独立实现. 小节 7.5.4 展示了如何使用 `flow_matching` 程序库，在任意数据耦合下训练离散流.

代码 10: 使用混合路径与任意数据耦合进行 DFM 训练与采样.

```python
import torch

from flow_matching.path import MixtureDiscreteProbPath, DiscretePathSample
from flow_matching.path.scheduler import PolynomialConvexScheduler
from flow_matching.loss import MixturePathGeneralizedKL
from flow_matching.solver import MixtureDiscreteEulerSolver
from flow_matching.utils import ModelWrapper


model = ...  # 定义可训练的速度模型
optimizer = torch.optim.Adam(model.parameters())

scheduler = PolynomialConvexScheduler(n=1.0)
path = MixtureDiscreteProbPath(scheduler=scheduler)
loss_fn = MixturePathGeneralizedKL(path=path)  # 广义 KL Bregman 散度

for x_0, x_1 in dataloader:  # 来自 π_{0,1} 的样本，形状为 [batch_size, *data_dim]
    t = torch.rand(batch_size) * (1.0 - 1e-3)  # 随机选取时间 t ~ U[0, 1 - 10^{-3}]
    sample: DiscretePathSample = path.sample(t=t, x_0=x_0, x_1=x_1)  # 对条件路径采样
    model_output = model(sample.x_t, sample.t)

    loss = loss_fn(logits=model_output, x_1=sample.x_1, x_t=sample.x_t, t=sample.t) #
    ↪ CDFM 损失

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

```python
class ProbabilityDenoiser(ModelWrapper):
    def forward(self, x: torch.Tensor, t: torch.Tensor, **extras) -> torch.Tensor:
        logits = self.model(x, t, **extras)
        return torch.nn.functional.softmax(logits.float(), dim=-1)

# 采样 X_1
probability_denoiser = ProbabilityDenoiser(model=model)
x_0 = torch.randint(size=[batch_size, *data_dim])  # 指定初始条件
solver = MixtureDiscreteEulerSolver(
    model=probability_denoiser,
    path=path,
    vocabulary_size=vocabulary_size
)

step_size = 1 / 100
x_1 = solver.sample(x_init=x_0, step_size=step_size, time_grid=torch.tensor([0.0, 1.0-1e-3]))
```

代码 11: 离散流匹配独立代码
flow_matching/examples/standalone_discrete_flow_matching.ipynb

```python
import torch
import matplotlib.pyplot as plt
from torch import nn, Tensor
from sklearn.datasets import make_moons

class DiscreteFlow(nn.Module):
    def __init__(self, dim: int = 2, h: int = 128, v: int = 128):
        super().__init__()
        self.v = v
        self.embed = nn.Embedding(v, h)
        self.net = nn.Sequential(
            nn.Linear(dim * h + 1, h), nn.ELU(),
            nn.Linear(h, h), nn.ELU(),
            nn.Linear(h, h), nn.ELU(),
            nn.Linear(h, dim * v))

    def forward(self, x_t: Tensor, t: Tensor) -> Tensor:
        return self.net(torch.cat((t[:, None], self.embed(x_t).flatten(1, 2)), -1)).reshape(list(x_t.shape) + [self.v])

batch_size = 256
vocab_size = 128

model = DiscreteFlow(v=vocab_size)
optim = torch.optim.Adam(model.parameters(), lr=0.001)

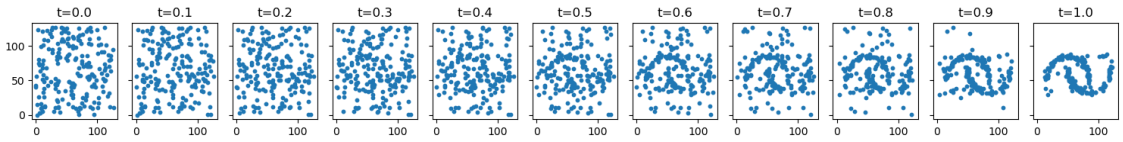
for _ in range(10000):
    x_1 = Tensor(make_moons(batch_size, noise=0.05)[0])
    x_1 = torch.round(torch.clip(x_1 * 35 + 50, min=0.0, max=vocab_size - 1)).long()
    x_0 = torch.randint(low=0, high=vocab_size, size=(batch_size, 2))

    t = torch.rand(batch_size)
    x_t = torch.where(torch.rand(batch_size, 2) < t[:, None], x_1, x_0)

```

```
    logits = model(x_t, t)
    loss = nn.functional.cross_entropy(logits.flatten(0, 1), x_1.flatten(0,
    ↪  1)).mean()
    optim.zero_grad()
    loss.backward()
    optim.step()

x_t = torch.randint(low=0, high=vocab_size, size=(200, 2))
t = 0.0
results = [(x_t, t)]
while t < 1.0 - 1e-3:
    p1 = torch.softmax(model(x_t, torch.ones(200) * t), dim=-1)
    h = min(0.1, 1.0 - t)
    one_hot_x_t = nn.functional.one_hot(x_t, vocab_size).float()
    u = (p1 - one_hot_x_t) / (1.0 - t)
    x_t = torch.distributions.Categorical(probs=one_hot_x_t + h * u).sample()
    t += h
    results.append((x_t, t))
fig, axes = plt.subplots(1, len(results), figsize=(15, 2), sharex=True, sharey=True)
for (x_t, t), ax in zip(results, axes):
    ax.scatter(x_t.detach()[:, 0], x_t.detach()[:, 1], s=10)
    ax.set_title(f't={t:.1f}')
plt.tight_layout()
plt.show()
```



8 连续时间马尔可夫过程模型

前面的各节建立了 $\mathbb{R}^d$ 和黎曼流形上的流模型（见 节 3 和 节 5），以及离散数据的 CTMC 模型（见 节 6）. 本节将统一并扩展这些模型，构建适用于一般状态空间和一般马尔可夫过程的生成模型. 这将使我们在 节 9 中将流匹配的原理推广到面向多种模态的各类生成模型.

8.1 一般状态空间与随机变量

处理一般模态. 我们的明确目标是不指定所用模态. 因此，本节始终令 $\mathcal{S}$ 为一般的 **状态空间**. 重要的例子包括 $\mathcal{S} = \mathbb{R}^d$（例如图像、向量），离散的 $\mathcal{S}$（例如语言），黎曼流形 $\mathcal{S}$（例如几何数据），以及用于联合生成多种数据模态的上述空间的乘积（多模态模型）. 对于所有模态，都可以在 $\mathcal{S}$ 上定义度量（或距离函数）$d : \mathcal{S} \times \mathcal{S} \to \mathbb{R}_{\geq 0}, (x, y) \mapsto d(x, y)$. 例如，对于离散的 $\mathcal{S}$，当 $y \neq x$ 时 $d(x, y) = 1$，并对所有 $x \in \mathcal{S}$ 令 $d(x, x) = 0$. 对于 $\mathcal{S} = \mathbb{R}^d$，使用 $d(x, y) = \|x - y\|$. 我们需要作出技术性假设：$(\mathcal{S}, d)$ 是波兰度量空间，即它是完备的（任何柯西序列都收敛）且可分的（具有可数稠密子集）. 机器学习中关注的任何模态都具有这一性质.

一般状态空间上的密度. 到目前为止，本文假设 $\mathcal{S}$ 上的概率分布 p 由密度 $p : \mathcal{S} \to \mathbb{R}_{\geq 0}$ 表示. 对于一般状态空间，使用一般的 **参考测度** ν，密度则成为 Radon–Nikodym 导数 $\frac{\mathrm{d}p}{\mathrm{d}\nu}$. 换言之，概

名称	**流**	**扩散**	**跳跃过程**	**连续时间 马尔可夫链**
空间 $\mathcal{S}$	$\mathcal{S}=\mathbb{R}^d$	$\mathcal{S}=\mathbb{R}^d$	任意 $\mathcal{S}$	$\lvert\mathcal{S}\rvert<\infty$
参数	速度场： $u_t(x)\in\mathbb{R}^d$	扩散系数： $\sigma_t(x)\in\mathbb{R}^{d\times d}$（半正定）	跳跃测度 $Q_t(\mathrm{d}y,x)$	$u_t\in\mathbb{R}^{\mathcal{S}\times\mathcal{S}}$, $1^Tu_t=0$ $u_t(x';x)\geq 0$ $(x'\neq x)$
采样	$X_{t+h}=X_t+hu_t(X_t)$	$X_{t+h}=X_t+\sqrt{h}\sigma_t(X_t)\epsilon_t$ $\epsilon_t\sim\mathcal{N}(0,I)$	$X_{t+h}=X_t$，概率为 $1-h\int Q_t(\mathrm{d}y,x)$ $X_{t+h}\sim\frac{Q_t(\mathrm{d}y,x)}{\int Q_t(\mathrm{d}y,x)}$，概率为 $h\int Q_t(\mathrm{d}y,x)$	$X_{t+h}\sim(I+hu_t)(\cdot;X_t)$
生成元 $\mathcal{L}_t$	∇f^Tu_t	$\frac{1}{2}\nabla^2 f\cdot\sigma_t^2$	$\int[f(y)-f(x)]Q_t(\mathrm{d}y,x)$	$f^Tu_t^T$
KFE （伴随形式）	连续性方程： $\partial_tp_t=-\mathrm{div}(p_tu_t)$	Fokker–Planck 方程： $\partial_tp_t=\frac{1}{2}\nabla^2\cdot(p_t\sigma_t^2)$	跳跃连续性方程：$\partial_tp_t(x)=$ $\int Q_t(x,y)p_t(y)-Q_t(y,x)p_t(x)\nu(\mathrm{d}y)$	质量守恒： $\partial_tp_t=u_tp_t$
边缘形式	$\mathbb{E}_{Z\sim p_{Z\mid t}(\cdot\mid x)}[u_t(x\mid Z)]$	$\mathbb{E}_{Z\sim p_{Z\mid t}(\cdot\mid x)}[\sigma_t^2(x\mid Z)]$	$\mathbb{E}_{Z\sim p_{Z\mid t}(\cdot\mid x)}[Q_t(y,x\mid Z)]$	$\mathbb{E}_{Z\sim p_{Z\mid t}(\cdot\mid x)}[u_t(y;x\mid Z)]$
CGM 损失 （例子）	$\lVert u_t(X\mid Z)-u_t^\theta(X)\rVert^2$	$\lVert\sigma_t^2(X\mid Z)-(\sigma_t^\theta)^2(X)\rVert_2^2$	$(\int Q_t^\theta(y;X)\nu(\mathrm{d}y)$ $-Q_t(y,X\mid Z)\log Q_t^\theta(y;X)\nu(\mathrm{d}y))$	$(\sum_{y\neq x}u_t^\theta(y;X)$ $-u_t(y;X\mid Z)\log u_t^\theta(y;X))$

表 2 CTMP 生成模型的一些例子，以及如何通过生成元匹配学习它们. 此处并未穷尽所有模型. 推导见节 8. 对于扩散，假设漂移为零，因为漂移已由 "流" 一列涵盖. KFE 以伴随形式列出，即假设相对于参考测度 ν，存在跳跃核 $Q_t(y,x)$ 和密度 $p_t(x)$. "p.s.d." 表示半正定.

率可以表示为关于 ν 的积分：

$$\mathbb{P}(A)=\int_A p(x)\nu(\mathrm{d}x)\ \text{对所有可测的}A\subset\mathcal{S}$$

对于离散的 $\mathcal{S}$，ν 是计数测度（因此积分就是求和），$p(x)$ 就是概率质量函数（PMF）. 对于 $\mathcal{S}=\mathbb{R}^d$，ν 是勒贝格测度（因此积分就是 "通常" 的积分），$p(x)$ 就是概率密度函数（PDF）. 上述表述将其推广到任意状态空间.

（可选）处理任意分布. 并非每个概率分布都有相对于参考测度的密度. 对不熟悉一般测度论的读者，只要处理的分布具有密度 $p(x)$，就可以把这种可能性视为技术性说明而略过. 然而，这些并非只是病态的例子；在机器学习应用中，也存在确实需要考虑这种情况的实例. 一个简单例子是 $\mathcal{S}=\mathbb{R}^d$ 上形如 $p_t=\delta_{(1-t)x+ty}$ 的概率路径，它沿直线连接两点 $x,y\in\mathbb{R}^d$，无法由密度表示. 另一个例子是 $\mathcal{S}=C([0,1],\mathbb{R})$ 上的概率分布，例如轨迹建模中的分布，它们往往没有相对于共同参考测度的密度. 为在数学上处理这些情形，我们针对 $\mathcal{S}$ 上的一般 **概率测度** p 建立框架. 为此使用记号 $p(\mathrm{d}x)$，其中 "$\mathrm{d}x$" 是 符号性表达式，表示对变量 x 关于 p 积分. 例如，对于有界可测函数 $f:\mathcal{S}\to\mathbb{R}$，写作

$$\mathbb{E}_{X\sim p}[f(X)]=\int f(x)p(\mathrm{d}x)$$

来表示勒贝格积分，或 f 在 p 下的期望. 与前面一样，我们稍微滥用记号，仍用 $p(x)$ 表示测度 $p(\mathrm{d}x)$ 的密度.

8.2 CTMP 生成模型

类似地，我们的明确目标是构建适用于任意演化过程的模型，无论使用流、扩散、CTMC、它们的组合，还是其他过程. 因此，本节定义一种一般的演化过程，同时满足构建生成模型所需的正则性假设. 对于 $t\in[0,1]$，令 $X_t\in\mathcal{S}$ 为随机变量. 如果 $(X_t)_{0\leq t\leq 1}$ 满足下列条件，就称其为 **连续时间马尔可夫过程（CTMP）**：

$$\mathbb{P}[X_{t_{n+1}} \in A | X_{t_1}, X_{t_2}, \ldots, X_{t_n}] = \mathbb{P}[X_{t_{n+1}} \in A | X_{t_n}] \quad (0 \le t_1 < \cdots < t_{n+1} \le 1, A \subseteq \mathcal{S}) \tag{8.1}$$

非正式地说，上述条件表示过程没有记忆. 如果已知现在，了解过去不会影响我们对未来的预测. 表 2 概述了几类重要的马尔可夫过程. 例如，流形上的流具有确定性转移；扩散的转移由布朗运动驱动；CTMC 则由速率决定（将在 小节 8.2.2 中详细解释）. 每个马尔可夫过程都有一个 **转移核** $(p_{t+h|t})_{0 \le t < t+h \le 1}$，为每个 $x \in \mathcal{S}$ 指定一个概率分布 $p_{t+h|t}(\cdot|x)$，使得

$$\mathbb{P}[X_{t+h} \in A | X_t = x] = p_{t+h|t}(A|x) \quad \text{对所有} t, h \ge 0, A \subset \mathcal{S} \text{ 可测} \tag{8.2}$$

由马尔可夫假设，马尔可夫过程由转移核和 X_0 的分布唯一确定. 反之，任意转移核和初始分布定义一个马尔可夫过程. 因此，两者存在一一对应.

接下来的目标是为 CTMP 定义速度场的相应推广. 非正式地说，它应当是转移核关于 t 的一阶导数：

$$\mathcal{L}_t \coloneqq \frac{\mathrm{d}}{\mathrm{d}h}\Big|_{h=0} p_{t+h|t} \tag{8.3}$$

我们将这一阶导数 $\mathcal{L}_t$ 称为 $p_{t+h|t}$ 的 **生成元** (Ethier and Kurtz, 2009; Rüschendorf et al., 2016). 与导数类似，生成元是一阶 线性近似，比 $p_{t+h|t}$ 更容易参数化. 我们将看到，扩散、流和其他生成模型都可以看作学习马尔可夫过程生成元的算法（见 表 2）. 由此得到 **CTMP 生成模型** 的一般形式：

$$\textbf{CTMP 模型（非正式）：} \quad p_{t+h|t}(\cdot|x) \coloneqq \delta_x + h\mathcal{L}_t(x) + o(h), \text{ 且} X_0 \sim p. \tag{8.4}$$

然而，由于转移核 $p_{t+h|t}$ 不是实函数，式 8.3 和 8.4 目前只是启发性的，还没有严格定义. 因此，本节首要目标是给出生成元和 CTMP 生成模型的 正式定义.

8.2.1 生成元的正式定义

式 (8.3) 中的第一个问题是，导数通常针对取值于向量空间的函数定义，而 $p_{t+h|t}$ 的值是分布. 不过，使用 **测试函数** 可以缓解这一问题. 测试函数提供了一种 “探测” 概率分布的方法，是使我们能够像处理实值函数一样处理分布的理论工具. 具体而言，一组测试函数是由有界可测函数 $f : S \to \mathbb{R}$ 构成的族 $\mathcal{T}$，它能完全 刻画概率分布，即对于 $\mathcal{S}$ 上的两个概率分布 μ_1, μ_2，有

$$\mu_1 = \mu_2 \quad \Leftrightarrow \quad \mathbb{E}_{X\sim\mu_1}[f(X)] = \mathbb{E}_{X\sim\mu_2}[f(X)] \quad \text{对所有} f \in \mathcal{T} \tag{8.5}$$

一般而言，选择尽可能 “良好”（或正则）的 $\mathcal{T}$. 例如，当 $\mathcal{S} = \mathbb{R}^d$ 时，紧支撑无穷次可微函数构成的空间 $\mathcal{T} = C_c^\infty(\mathbb{R}^d)$ 满足这一性质. 对于离散的 $\mathcal{S}$，$\mathcal{T} = \mathbb{R}^{\mathcal{S}}$ 就由所有函数构成（此时函数就是向量）. 令 $X_t \sim p_t$. 定义 **边缘作用** 和 **转移作用** 为

$$\langle p_t, f \rangle \coloneqq \int f(x) p_t(\mathrm{d}x) = \mathbb{E}_{X\sim p_t}[f(X)] \tag{8.6}$$

$$\langle p_{t+h|t}, f \rangle (x) \coloneqq \langle p_{t+h|t}(\cdot|x), f \rangle = \mathbb{E}[f(X_{t+h})|X_t = x] \tag{8.7}$$

其中，边缘作用将每个测试函数 f 映为标量 $\langle p_t, f \rangle \in \mathbb{R}$，而转移作用将实值函数 $x \mapsto f(x)$ 映为另一个实值函数 $x \mapsto \langle p_{t+h|t}, f \rangle (x)$. 塔式性质意味着 $\langle p_t, \langle p_{t+h|t}, f \rangle \rangle = \langle p_{t+h}, f \rangle$. 上述内积只是

“符号性”的，但如果密度 $p_t(x)$ 存在，它就成为“真正”的内积，即 $\langle p_t, f\rangle = \int f(x)p_t(x)\nu(\mathrm{d}x)$.

正式定义 式 (8.3) 中导数的第二步，是给马尔可夫过程施加某种“光滑性”要求，下面给出定义. 令 $C_0(\mathcal{S})$ 为在无穷远处消失的连续函数 $f:\mathcal{S}\to\mathbb{R}$ 构成的空间，即对所有 $\epsilon>0$，存在紧集 $K\subset\mathcal{S}$，使得对所有 $x\in\mathcal{S}\setminus K$ 都有 $|f(x)|<\epsilon$. 在 $C_0(\mathcal{S})$ 上使用上确界范数 $\|\cdot\|_\infty$. 若 CTMP X_t 满足以下两个条件，则称其为 **Feller 过程** (Feller, 1955; Rüschendorf et al., 2016)：

1. **强连续性**： $p_{t+h|t}$ 的作用关于时间连续：

$$\lim_{h'\to h, t'\to t} \| \langle p_{t'+h'|t'}, f\rangle - \langle p_{t+h|t}, f\rangle \|_\infty = 0 \quad \text{对所有} h,t\geq 0, f\in C_0(\mathcal{S})$$

2. **不从无穷远返回**： $p_{t+h|t}$ 的作用保持在无穷远处消失的函数：

$$\langle p_{t+h|t}, f\rangle \in C_0(\mathcal{S}) \quad \text{对所有} h,t\geq 0, f\in C_0(\mathcal{S})$$

假设 4. CTMP $(X_t)_{0\leq t\leq 1}$ 是 Feller 过程.

考虑到我们希望在机器学习模型中使用 X_t，这一假设是合理的：所定义的概率路径使生成过程 X_t 的分布平滑变化，而且所有数据通常都位于某个有界（紧）集内.

现在重新考察 (8.3)，尝试定义 $p_{t+h|t}$ 的导数. 从测试函数的角度出发，可以对每个 $x\in\mathcal{S}$ 求 $\langle p_{t+h|t}, f\rangle(x)$ 的导数，并定义

$$\frac{\mathrm{d}}{\mathrm{d}h}\Big|_{h=0} \langle p_{t+h|t}, f\rangle(x) = \lim_{h\to 0} \frac{\langle p_{t+h|t}, f\rangle(x) - f(x)}{h} := [\mathcal{L}_t f](x). \tag{8.8}$$

我们将这一作用称为 **生成元** $\mathcal{L}_t$，并对上述极限一致存在（即在范数 $\|\cdot\|_\infty$ 下存在）的所有 f 定义它. 直观地说，生成元被定义为 *作用于测试函数的算子*. 表 2 给出了几个生成元的例子，将在 小节 8.2.2 中推导. 生成元与 Feller 过程之间存在一一对应 (Rogers and Williams, 2000; Ethier and Kurtz, 2009; Pazy, 2012)，正如流与向量场之间存在对应一样（见 定理 1）. 这将使我们随后能够在神经网络中通过生成元参数化 Feller 过程.

采用这一定义后，(8.4) 中的 **CTMP 模型** 具有如下严格定义的形式：

$$\langle p_{t+h|t}, f\rangle = f + h\mathcal{L}_t f + o(h) (\text{对所有} f\in\mathcal{T}) \quad \text{且} X_0\sim p \tag{8.9}$$

其中 $o(h)$ 表示误差项 $E(h)\in C_0(\mathcal{S})$，满足 $\lim_{h\to 0}\frac{1}{h}\|E(h)\|_\infty = 0$. 类似于流情形的 式 (3.24) 和 CTMC 情形的 式 (6.5)，我们称

$$\mathcal{L}_t \text{ **生成** } p_t, \text{ 若存在} p_{t+h|t} \text{ 满足 (8.9)，且相应 CTMP } X_t \text{ 满足} X_t\sim p_t \tag{8.10}$$

换言之，如果马尔可夫过程以 $p=p_0$ 初始化并通过 $\mathcal{L}_t$ 模拟，其边缘分布为 p_t，那么生成元 $\mathcal{L}_t$ 就生成概率路径 p_t.

8.2.2 CTMP 模型的例子

通过几个例子说明如何计算马尔可夫过程的生成元. 本节结果汇总于 表 2.

流. 令 $\mathcal{S}=\mathbb{R}^d$，并令 $u:[0,1]\times\mathbb{R}^d\to\mathbb{R}^d,(t,x)\mapsto u_t(x)$ 为定义流 ψ_t 的时变速度场（见 节 3）. 令 $\mathcal{T}=C_c^\infty(\mathbb{R}^d)$ 为具有紧支撑的无穷次可微光滑函数空间. 于是可按下式计算生成元：

$$[\mathcal{L}_t f](x)=\lim_{h\to0}\frac{\mathbb{E}\left[f(X_{t+h})|X_t=x\right]-f(x)}{h} \tag{8.11}$$

$$\overset{(i)}{=}\lim_{h\to0}\frac{\mathbb{E}\left[f(X_t+hu_t(X_t)+o(h))|X_t=x\right]-f(x)}{h} \tag{8.12}$$

$$\overset{(ii)}{=}\lim_{h\to0}\frac{\mathbb{E}\left[f(X_t)+h\nabla f(X_t)^Tu_t(X_t)+o(h)|X_t=x\right]-f(x)}{h} \tag{8.13}$$

$$=\lim_{h\to0}\frac{f(x)+h\nabla f(x)^Tu_t(x)+o(h)-f(x)}{h} \tag{8.14}$$

$$\tag{8.15}$$

$$=\nabla f(x)^Tu_t(x), \tag{8.16}$$

其中，(i) 来自流的欧拉近似，(ii) 来自 f 在 X_t 附近的一阶泰勒近似. 因此，**流生成元** 为

$$\mathcal{L}_t f(x)=\nabla f(x)^Tu_t(x). \tag{8.17}$$

扩散. 令 $\mathcal{S}=\mathbb{R}^d$，并令 $\sigma_t:[0,1]\times\mathbb{R}^d\to\mathbb{R}^{d\times d},(t,x)\mapsto\sigma_t(x)$ 为连续取值于对称半正定矩阵 σ_t 的时变函数. 扩散系数为 σ_t 的扩散过程由随机微分方程 $dX_t=\sigma_t(X_t)dW_t$ 定义，其中 W_t 为维纳过程 (Øksendal, 2003). 该过程可以通过如下无穷小采样步骤近似：

$$X_{t+h}=X_t+\sqrt{h}\sigma_t(X_t)\epsilon_t,\quad \epsilon_t\sim\mathcal{N}(0,I) \tag{8.18}$$

仍令 $\mathcal{T}=C_c^\infty(\mathbb{R}^d)$. 则可以按下式计算生成元：

$$[\mathcal{L}_t f](x)=\lim_{h\to0}\frac{\mathbb{E}\left[f(X_t+\sqrt{h}\sigma_t(X_t)\epsilon_t+o(h))|X_t=x\right]-f(x)}{h} \tag{8.19}$$

$$\overset{(i)}{=}\lim_{h\to0}\frac{\mathbb{E}_{\epsilon_t}[f(x)+\nabla f(x)^T\sqrt{h}\sigma_t(x)\epsilon_t+\frac{1}{2}h[\sigma_t(x)\epsilon_t]^T\nabla^2f(x)[\sigma_t(x)\epsilon_t]+o(h)-f(x)]}{h} \tag{8.20}$$

$$=\lim_{h\to0}\frac{\nabla f(x)^T\sqrt{h}\sigma_t(x)\mathbb{E}_{\epsilon_t}[\epsilon_t]+\mathbb{E}_{\epsilon_t}[\frac{1}{2}h[\sigma_t(x)\epsilon_t]^T\nabla^2f(x)[\sigma_t(x)\epsilon_t]]}{h} \tag{8.21}$$

$$=\frac{1}{2}\mathbb{E}_{\epsilon_t}[\epsilon_t^T[\sigma_t(x)]^T\nabla^2f(x)[\sigma_t(x)]\epsilon_t]] \tag{8.22}$$

$$\overset{(ii)}{=}\frac{1}{2}\mathrm{tr}\left(\sigma_t(x)^T\nabla^2f(x)\sigma_t(x)\right) \tag{8.23}$$

$$\overset{(iii)}{=}\frac{1}{2}\mathrm{tr}(\sigma_t(x)\sigma_t(x)^T\nabla^2f(x)) \tag{8.24}$$

$$\overset{(iv)}{=}\frac{1}{2}\sigma_t^2(x)\cdot\nabla^2f(x) \tag{8.25}$$

其中，(i) 使用二阶泰勒近似（因为 $\mathbb{E}[\|\sqrt{h}\epsilon_t\|^2]\propto h$），(ii) 使用对 $A\in\mathbb{R}^{d\times d}$ 成立的恒等式 $\mathrm{tr}(A)=\mathbb{E}_{\epsilon_t}[\epsilon_t^TA\epsilon_t]$，(iii) 使用迹的循环性质，(iv) 使用 σ_t 的对称性. 此外，用 $A\cdot B\coloneqq\mathrm{tr}(A^TB)$ 表示矩阵 $A,B\in\mathbb{R}^{d\times d}$ 的内积. 因此，**扩散生成元** 为

$$\mathcal{L}_t f(x) = \frac{1}{2}\sigma_t^2(x) \cdot \nabla^2 f(x). \tag{8.26}$$

跳跃. 接下来令 $\mathcal{S}$ 为任意空间，并考虑 **跳跃过程**. 跳跃过程由时变核 $Q_t(\mathrm{d}y, x)$ 定义，即对每个 $0 \le t \le 1$ 和每个 $x \in \mathcal{S}$，$Q_t(\mathrm{d}y, x)$ 都是 $S \setminus \{x\}$ 上的正测度. 跳跃过程的思想是，赋予 $S \setminus \{x\}$ 的总质量

$$\lambda_t(x) = \int Q_t(\mathrm{d}y, x) \tag{8.27}$$

给出 **跳跃强度**，即发生跳跃的无穷小可能性. 此外，如果 $\lambda_t(x) > 0$，将 Q_t 归一化为概率核，就可以指定 **跳跃分布**：

$$J_t(\mathrm{d}y, x) = \frac{Q_t(\mathrm{d}y, x)}{\lambda_t(x)}. \tag{8.28}$$

跳跃过程可以通过如下无穷小采样步骤近似：

$$X_{t+h} = \begin{cases} X_t & \text{概率为} 1 - h\lambda_t(X_t) + o(h) \\ \sim J_t(\mathrm{d}y, X_t) & \text{概率为} h\lambda_t(X_t) + o(h) \end{cases} \tag{8.29}$$

关于跳跃过程的严格处理，例如可参见 (Davis, 1984). 其生成元为

$$\mathcal{L}_t f(x) = \lim_{h\to 0} \frac{\mathbb{E}[f(X_{t+h}) - f(X_t) | X_t = x]}{h} \tag{8.30}$$

$$= \lim_{h\to 0} \frac{\mathbb{E}[f(X_{t+h}) - f(X_t) | X_t = x, \text{于}[t, t+h)\text{跳跃}]\mathbb{P}[\text{于}[t, t+h)\text{跳跃} | X_t = x]}{h} \tag{8.31}$$

$$+ \lim_{h\to 0} \underbrace{\frac{\mathbb{E}[f(X_{t+h}) - f(X_t) | X_t = x, \text{于}[t, t+h)\text{不跳跃}]\mathbb{P}[\text{于}[t, t+h)\text{不跳跃} | X_t = x]}{h}}_{=0}$$

$$\tag{8.32}$$

$$= \lim_{h\to 0} \frac{\mathbb{E}_{Y\sim J_t(\mathrm{d}y,x)}\left[f(Y) - f(x)\right] h\lambda_t(x)}{h} \tag{8.33}$$

$$= \mathbb{E}_{Y\sim J_t(\mathrm{d}y,x)}\left[f(Y) - f(x)\right] \lambda_t(x) \tag{8.34}$$

$$= \int (f(y) - f(x)) Q_t(\mathrm{d}y, x) \tag{8.35}$$

其中使用了：如果 X_t 在 $[t, t+h]$ 内不发生跳跃，则 $X_{t+h} = X_t$. 因此，**跳跃生成元** 为

$$\mathcal{L}_t f(x) = \int (f(y) - f(x)) Q_t(\mathrm{d}y, x) = \lambda_t(x)\mathbb{E}_{Y\sim J_t(\mathrm{d}y,x)}[f(Y) - f(x)]. \tag{8.36}$$

连续时间马尔可夫链（*CTMC*）. 考虑离散空间 $\mathcal{S}$ 上的连续时间马尔可夫链 X_t，其中 $|\mathcal{S}| < \infty$. 实际上，这就是采用特定参数化方式的离散状态空间上的跳跃过程. 为说明这一点，考虑离散状

态空间 $\mathcal{S}$ 上由矩阵 $Q_t \in \mathbb{R}_{\geq 0}^{\mathcal{S}\times\mathcal{S}}$ 给出的普通跳跃核. 利用 式 (8.36)，生成元为

$$\mathcal{L}_t f(x) = \sum_{y\in\mathcal{S}} [f(y) - f(x)] Q_t(y,x) = \sum_{y\neq x} [f(y) - f(x)] Q_t(y,x) \quad \text{对所有} x \in \mathcal{S}, f \in \mathbb{R}^{\mathcal{S}} \tag{8.37}$$

即 $Q_t(x,x)$ 的值无关紧要，且不能唯一确定. 因此，一种自然约定是用速率重新参数化离散状态空间上的跳跃核：

$$u_t(y,x) = \begin{cases} Q_t(y,x) & \text{当} y \neq x \\ -\sum_{z\neq x} Q_t(z;x) & \text{当} y = x \end{cases}$$

由此恢复了 节 6 中的速率 $u_t(y,x)$，其构造保证满足 6.4 中的速率条件. 这表明离散空间上的跳跃模型与 CTMC 模型一致（节 6）. 将其代入 式 (8.37)，得到 **CTMC 生成元** 为

$$\mathcal{L}_t f(x) = \sum_{y\in\mathcal{S}} f(y) u_t(y,x) = f^T u_t \tag{8.38}$$

其中将 $f = (f(x))_{x\in\mathcal{S}}$ 视为列向量，将 $u_t \in \mathbb{R}^{\mathcal{S}\times\mathcal{S}}$ 视为矩阵. 因此，生成元函数就是向量从左侧相乘.

流形上的流. 接下来考虑 节 5 中的黎曼流形 $\mathcal{S} = \mathcal{M}$ 上的流. 流 $\psi : [0,1]\times\mathcal{M} \to \mathcal{M}$ 由向量场 $u : [0,1]\times\mathcal{M} \to T\mathcal{M}$ 通过 (3.19) 中的常微分方程定义. 记从时刻 s 到 t 的转移为 $\psi_{t|s}(x) = \psi_t(\psi_s^{-1}(x))$（同 (3.17)）. 则对于光滑函数 $f : \mathcal{M} \to \mathbb{R}$，**黎曼流生成元** 为

$$\mathcal{L}_t f(x) = \lim_{h\to 0} \frac{f(\psi_{t+h|t}(x)) - f(x)}{h} = \left\langle \nabla f(x), \frac{\mathrm{d}}{\mathrm{d}h}\Big|_{h=0} \psi_{t+h|t}(x) \right\rangle_g = \langle \nabla f(x), u_t(x) \rangle_g \tag{8.39}$$

其中 $\langle\cdot,\cdot\rangle_g$ 表示定义黎曼度量 g 的内积，∇f 表示 f 关于 g 的梯度. 实际上，该生成元就是函数的 李导数 (Jost, 2008)，这是微分几何中的基本概念.

8.3 概率路径与柯尔莫哥洛夫方程

对于 $\mathcal{S} = \mathbb{R}^d$ 上的流匹配，连续性方程（见 3.25）是核心数学方程，使我们能够构造生成所需概率路径的速度场（见 小节 3.5）. 本节为 CTMP 推导相应的更一般的方程. 令 X_t 为生成元是 $\mathcal{L}_t$ 的 CTMP，并令 $X_t \sim p_t$，则有：

$$\frac{\mathrm{d}}{\mathrm{d}t}\langle p_t, f\rangle = \frac{\mathrm{d}}{\mathrm{d}h}\Big|_{h=0} \langle p_{t+h}, f\rangle = \frac{\mathrm{d}}{\mathrm{d}h}\Big|_{h=0} \langle p_t, \langle p_{t+h|t}, f\rangle\rangle = \left\langle p_t, \frac{\mathrm{d}}{\mathrm{d}h}\Big|_{h=0} \langle p_{t+h|t}, f\rangle \right\rangle = \langle p_t, \mathcal{L}_t f\rangle$$

其中，利用 $\langle p_t, \cdot\rangle$ 运算的线性性交换求导次序，并利用塔式性质 $\langle p_t, \langle p_{t+h|t}, f\rangle\rangle = \langle p_{t+h}, f\rangle$. 这表明，给定马尔可夫过程 X_t 的生成元 $\mathcal{L}_t$，可以通过边缘概率的无穷小变化恢复这些概率，即得到 **柯尔莫哥洛夫前向方程（KFE）**：

$$\frac{\mathrm{d}}{\mathrm{d}t}\langle p_t, f\rangle = \langle p_t, \mathcal{L}_t f\rangle \quad \text{对所有} f \in \mathcal{T} \tag{8.40}$$

式 (8.40) 中的 KFE 确定测试函数 f 的期望如何演化. 当使用没有密度的概率分布时，这种形式是必要的. 如果密度存在，则可以使用更熟悉的 KFE 形式，直接规定概率密度的变化. 为此，引入 **伴随生成元** $\mathcal{L}_t^*$，它作用于相对于参考测度 ν 的概率密度 $p_t(x)$；具体而言，$\mathcal{L}_t^* p_t(x)$ 由下列恒等式隐式定义：

$$\int p_t(x)\mathcal{L}_t f(x)\nu(\mathrm{d}x) = \int \mathcal{L}_t^* p_t(x) f(x)\nu(\mathrm{d}x) \qquad \forall f \in \mathcal{T} \tag{8.41}$$

此外，需要假设 p_t 关于 t 可微. 将 (8.41) 应用于 KFE (8.40)，得到

$$\begin{aligned}
\int \frac{\mathrm{d}}{\mathrm{d}t} p_t(x) f(x)\nu(\mathrm{d}x) &= \frac{\mathrm{d}}{\mathrm{d}t}\int p_t(x) f(x)\nu(\mathrm{d}x) && (8.42)\\
&= \frac{\mathrm{d}}{\mathrm{d}t}\langle p_t, f\rangle && (8.43)\\
&= \langle p_t, \mathcal{L}_t f\rangle && (8.44)\\
&= \int p_t(x)\mathcal{L}_t f(x)\nu(\mathrm{d}x) && (8.45)\\
&= \int \mathcal{L}_t^* p_t(x) f(x)\nu(\mathrm{d}x) && (8.46)
\end{aligned}$$

由于这对所有测试函数 f 成立，利用 式 (8.5) 可知，它等价于 **伴随 KFE**：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) = \mathcal{L}_t^* p_t(x) \text{ 对所有} x \in \mathcal{S} \tag{8.47}$$

如下面的例子所示，伴随 KFE 推广了用于构建生成模型的许多著名方程，例如连续性方程和 Fokker–Planck 方程 (Song et al., 2021; Lipman et al., 2022)（见 表 2）. 只要概率密度存在，就使用伴随 KFE，从而避免使用测试函数，直接处理概率密度. 将这些发现总结如下：

定理 17 (一般质量守恒). *令 $\mathcal{L}_t$ 为 $(X_t)_{0\le t\le 1}$ 的生成元. 非正式地说，以下条件等价：*

1. *$p_t, \mathcal{L}_t$ 满足 KFE (8.40).*
2. *$\frac{\mathrm{d}p_t}{\mathrm{d}\nu}(x), \mathcal{L}_t$ 满足伴随 KFE (8.47).*
3. *$\mathcal{L}_t$ 在 (8.10) 的意义下生成 p_t.*

严格地说，只要 $\frac{\mathrm{d}p_t}{\mathrm{d}\nu}$ 存在且关于 t 连续可微，(1) 与 (2) 就等价. 此外，对于任意状态空间，(3) 蕴含 (1). 存在一些弱正则性假设，确保 (1) 蕴含 (3)（列表见 小节 A.3）. 本文假设这些条件成立，即 **假设 (3) 蕴含 (1)**.

据我们所知，对于抽象的一般状态空间，目前没有已知结果确保 定理 17 中的条件 (3) 蕴含 (1). 因此，这里直接将其作为假设. 对于机器学习研究者，这一假设在任何相关状态空间上都成立，因此不必担忧（见 小节 A.3）.

8.3.1 KFE 的例子

流的伴随 KFE. 令 $\mathcal{S} = \mathbb{R}^d$，并假设 p_t 相对于勒贝格测度具有有界且连续可微的密度 $p_t(x)$. 则可以按下式计算伴随生成元 $\mathcal{L}_t^*$：

$$\begin{aligned}
\langle p_t, \mathcal{L}_t f\rangle =& \mathbb{E}_{x\sim p_t}[\mathcal{L}_t f(x)] && (8.48)\\
=& \int \mathcal{L}_t f(x) p_t(x)\mathrm{d}x && (8.49)\\
\overset{(i)}{=}& \int \nabla f(x)^T u_t(x) p_t(x)\mathrm{d}x && (8.50)\\
\overset{(ii)}{=}& \int f(x) \underbrace{[-\mathrm{div}(p_t u_t)(x)]}_{=:\mathcal{L}_t^* p_t(x)} \mathrm{d}x && (8.51)\\
=& \int f(x)\mathcal{L}_t^* p_t(x)\mathrm{d}x && (8.52)
\end{aligned}$$

其中，(i) 由 式 (8.15) 得到，(ii) 由分部积分得到. 上述推导表明，伴随生成元为 $\mathcal{L}_t^* p_t = -\mathrm{div}(p_t u_t)(x)$（因为它满足 式 (8.41) 中的条件）. 利用伴随 KFE，恢复 **连续性方程**（见 式 (3.25)）：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) = -\mathrm{div}(p_t u_t)(x), \tag{8.53}$$

我们已在 小节 3.4 中详细研究了这一方程.

扩散的伴随形式. 令 $\mathcal{S} = \mathbb{R}^d$，并假设 p_t 相对于勒贝格测度具有有界且连续可微的密度 $p_t(x)$. 可以按下式计算伴随生成元 $\mathcal{L}_t^*$：

$$\begin{aligned}
\langle p_t, \mathcal{L}_t f\rangle =& \mathbb{E}_{x\sim p_t}[\mathcal{L}_t f(x)] && (8.54)\\
=& \int \mathcal{L}_t f(x) p_t(x)\mathrm{d}x && (8.55)\\
\overset{(i)}{=}& \frac{1}{2}\int \sigma_t^2(x)\cdot\nabla^2 f(x) p_t(x)\mathrm{d}x && (8.56)\\
\overset{(ii)}{=}& \int f(x) \underbrace{\frac{1}{2}\nabla^2\cdot(p_t\sigma_t^2)(x)}_{=:\mathcal{L}_t^* p_t(x)} \mathrm{d}x && (8.57)\\
=& \int f(x)\mathcal{L}_t^* p_t(x)\mathrm{d}x && (8.58)
\end{aligned}$$

其中，(i) 由 式 (8.26) 得到，(ii) 由两次分部积分得到. 上述推导表明，伴随生成元为 $\mathcal{L}_t^* p_t = \frac{1}{2}\nabla^2\cdot(p_t\sigma_t^2)(x)$（因为它满足 式 (8.41) 中的条件）. 伴随 KFE 因而恢复了著名的 **Fokker–Planck 方程**：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) = \frac{1}{2}\nabla^2\cdot(p_t\sigma_t^2)(x) \tag{8.59}$$

跳跃的伴随 KFE. 假设 p_t 相对于勒贝格测度具有有界且连续可微的密度 $p_t(x)$. 假设跳跃测度 $Q_t(\mathrm{d}y, x)$ 由核 $Q_t : \mathcal{S} \times \mathcal{S} \to \mathbb{R}_{\geq 0}, (y, x) \mapsto Q_t(y, x)$ 给出，使得

$$\int f(y) Q_t(\mathrm{d}y, x) = \int f(y) Q_t(y, x) \nu(\mathrm{d}y) \text{ 对所有可积的} f : \mathcal{S} \to \mathbb{R}. \tag{8.60}$$

于是可以如下推导伴随生成元：

$$\begin{aligned}
\langle p_t, \mathcal{L}_t f \rangle (x) &= \int \int (f(y) - f(x)) Q_t(y, x) \nu(\mathrm{d}y) p_t(x) \nu(\mathrm{d}x) && (8.61)\\
&= \int \int f(y) Q_t(y, x) p_t(x) \nu(\mathrm{d}y) \nu(\mathrm{d}x) - \int \int f(x) Q_t(y, x) p_t(x) \nu(\mathrm{d}y) \nu(\mathrm{d}x) && (8.62)\\
&\overset{(i)}{=} \int \int f(x) Q_t(x, y) p_t(y) \nu(\mathrm{d}y) \nu(\mathrm{d}x) - \int \int f(x) Q_t(y, x) p_t(x) \nu(\mathrm{d}y) \nu(\mathrm{d}x) && (8.63)\\
&= \int f(x) \underbrace{\left[\int Q_t(x, y) p_t(y) - Q_t(y, x) p_t(x) \nu(\mathrm{d}y) \right]}_{=: \mathcal{L}_t^* p_t} \nu(\mathrm{d}x) && (8.64)\\
&= \int f(x) \mathcal{L}_t^* p_t(x) \nu(\mathrm{d}x) && (8.65)
\end{aligned}$$

其中，在 (i) 中只交换了变量 x 和 y. 上述推导表明，所定义的 $\mathcal{L}_t^*$ 满足 式 (8.41) 中的条件，确实给出了跳跃的伴随生成元. 由此，伴随 KFE 成为 **跳跃连续性方程**：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) = \int \left[Q_t(x, y) p_t(y) - Q_t(y, x) p_t(x) \right] \nu(\mathrm{d}y) = \int \lambda_t(y) J_t(x, y) p_t(y) \nu(\mathrm{d}y) - \lambda_t(x) p_t(x) \tag{8.66}$$

其中使用了分解 $Q_t(y, x) = \lambda_t(x) J_t(y, x)$，将其分解为跳跃强度 λ_t 和跳跃分布 J_t（见 式 (8.28)）.

CTMC 的伴随 KFE. 对于离散的 $\mathcal{S}$ 以及 式 (8.38) 中由 $f^T u_t$ 给出的生成元，有

$$\begin{aligned}
\langle p_t, \mathcal{L}_t f \rangle = \int p_t(x) \mathcal{L}_t f(x) \nu(\mathrm{d}x) &= \sum_{x \in \mathcal{S}} p_t(x) \sum_{y \in \mathcal{S}} u_t(y, x) f(y)\\
&= \sum_{y \in \mathcal{S}} \underbrace{\left[\sum_{x \in \mathcal{S}} p_t(x) u_t(y, x) \right]}_{=: \mathcal{L}_t^* p_t(x)} f(y)\\
&= \int \mathcal{L}_t^* p_t(y) f(y) \nu(\mathrm{d}y)
\end{aligned}$$

这里 ν 就是计数测度. 因此，伴随 KFE 简单地由下式给出：

$$\frac{\mathrm{d}}{\mathrm{d}t} p_t(x) = \sum_{y \in \mathcal{S}} u_t(x, y) p_t(y) \tag{8.67}$$

这恢复了 式 (6.8) 中推导的 CTMC 的 KFE（交换了 x 与 y，以便与本节推导一致）.

8.4 普适表示定理

生成元使我们能够刻画所有可能的马尔可夫过程构成的空间. 具体而言，下面的结果不仅能刻画一大类 CTMP 生成模型，还能在 $\mathcal{S} = \mathbb{R}^d$ 或 $\mathcal{S}$ 离散时 穷尽地刻画其设计空间.

定理 18 (生成元的普适刻画)**.** *在弱正则性假设下，Feller 过程 X_t $(0 \le t \le 1)$ 的生成元具有以下形式：*

1. **离散情形** $|\mathcal{S}| < \infty$*：生成元由速率转移矩阵 u_t 给出，马尔可夫过程对应于连续时间马尔可夫链 (CTMC)* .
2. **欧几里得空间** $\mathcal{S} = \mathbb{R}^d$*：生成元可表示为 表 2 中各分量之和，即*

$$\mathcal{L}_t f(x) = \underbrace{\nabla f(x)^T u_t(x)}_{\text{流}} + \underbrace{\frac{1}{2}\nabla^2 f(x) \cdot \sigma_t^2(x)}_{\text{扩散}} + \underbrace{\int [f(y) - f(x)]\, Q_t(\mathrm{d}y, x)}_{\text{跳跃}} \tag{8.68}$$

其中，$u : [0,1] \times \mathbb{R}^d \to \mathbb{R}^d$ 为速度场，$\sigma : [0,1] \times \mathbb{R}^d \to S_d^{++}$ 为扩散系数 (S_d^{++} 表示半正定矩阵)，$Q_t(dy; x)$ 为跳跃测度；$\nabla^2 f(x)$ 表示 f 的 Hessian 矩阵，$\nabla^2 f(x) \cdot \sigma_t^2(x)$ 表示 Frobenius 内积.

证明改编自数学文献中的一个已知结果 (Courrege, 1965; von Waldenfels, 1965)，可见 (Holderrieth et al., 2024).

9 生成元匹配

本节介绍 生成元匹配 (GM) (Holderrieth et al., 2024)，一种适用于任意数据模态和一般马尔可夫过程的生成建模框架. GM 统一了近年来开发的绝大多数生成模型，包括扩散模型、“离散扩散” 模型以及前面各节介绍的流匹配变体. 为引入 GM，我们在 节 8 中定义了通过马尔可夫过程的 生成元构建的 CTMP 生成模型. GM 给出一种可扩展的生成元训练算法，方法名称由此而来. 除了提供统一框架，GM 还产生了多种新模型，允许组合不同类别的模型，并能为任意模态构建模型，包括跨越多种数据模态的模型.

9.1 数据与耦合

与前面一样，我们的目标是将分布 p 的样本 $X_0 \sim p$ 转移为目标分布 q 的样本 $X_1 \sim q$，其中 $X_0, X_1 \in \mathcal{S}$ 是两个取值于状态空间 $\mathcal{S}$ 的随机变量. 源样本与目标样本可以通过独立耦合 $(X_0, X_1) \sim p \otimes q$ (乘积分布) 关联，也可以通过一般的概率质量函数耦合 $\pi_{0,1}$ 关联，即 $\mathcal{S} \times \mathcal{S}$ 上边缘分布为 $\pi_0 = p$ 和 $\pi_1 = q$ 的分布. 与前面唯一的区别是，$\mathcal{S}$ 是一般状态空间，且 p, q 可以是任意概率测度.

9.2 一般概率路径

与前面一样，GM 构建步骤中的下一步是指定一条在 p 与 q 之间插值的概率路径 p_t. 按照 小节 4.4，使用 **条件概率路径** $p_{t|Z}(\mathrm{d}x|z)$，即一组依赖于潜在状态 $z \in \mathcal{Z}$ 的时变概率测度. 给定 $\mathcal{Z}$ 上的分布 p_Z，考虑由如下分层采样步骤定义的相应 **边缘概率路径** $p_t(\mathrm{d}x)$：

$$Z \sim p_Z, X_t \sim p_{t|Z}(\mathrm{d}x|z) \quad \Rightarrow \quad X_t \sim p_t(\mathrm{d}x)$$

即先从 p_Z 采样 Z，再从 $p_{t|Z}(\mathrm{d}x|z)$ 采样 X_t，就得到 p_t 的样本. 与前面一样，边缘概率路径的构造需满足边界约束 $p_0 = p$ 和 $p_1 = q$.

对于 $\mathcal{Z}=\mathcal{S}$ 和 $p_Z=q$，我们已经见过两种常见构造. 第一种是 $\mathcal{S}=\mathbb{R}^d$ 上的仿射条件流（用于连续流匹配，见 节 4），定义为

$$Z\sim q, X_0\sim p, X_t=\sigma_t X_0+\alpha_t Z \quad \Rightarrow \quad X_t\sim p_t(\mathrm{d}x) \tag{9.1}$$

其中 $\alpha_t,\sigma_t\in\mathbb{R}_{\geq 0}$ 是满足 $\alpha_0=\sigma_1=0$ 和 $\alpha_1=\sigma_0=1$ 的可微函数. 第二种是对任意 $\mathcal{S}$ 使用混合，正如离散状态空间上的离散流匹配所做的那样（式 (7.22)）：

$$Z\sim q, X_0\sim p, X_t\sim \begin{cases} Z & \text{概率为}\kappa_t \\ X_0 & \text{概率为}(1-\kappa_t) \end{cases} \quad \Rightarrow \quad X_t\sim p_t(\mathrm{d}x) \tag{9.2}$$

其中 $\kappa_t\in\mathbb{R}_{\geq 0}$ 是满足 $\kappa_0=0$、$\kappa_1=1$ 和 $0\leq\kappa_t\leq 1$ 的可微函数. 容易看出，仿射条件概率路径和混合概率路径都在 p 与 q 之间插值，即 $p_0=p$ 和 $p_1=q$.

9.3 通过神经网络参数化生成元

给定概率路径 p_t，目标是构造由生成元 $\mathcal{L}_t$ 指定并生成该概率路径的 CTMP 模型（见 式 (8.10)）. 为此训练神经网络之前，首先需要说明如何用参数为 θ 的神经网络 $\mathcal{L}_t^\theta$ 参数化生成元 $\mathcal{L}_t$. 本节讨论这一问题.

仍令 $\mathcal{T}$ 为测试函数族（见 小节 8.2.1）. $\mathcal{L}_t$ 的 **线性参数化** 定义如下：对每个 $x\in\mathcal{S}$，存在凸闭集 $\Omega_x\subset V_x$，其中 V_x 为带内积 $\langle\cdot,\cdot\rangle_x$ 的向量空间；并存在一个线性算子 $\mathcal{K}:\mathcal{T}\to C(\mathcal{S};V_x)$，使得所考虑的每个生成元 $\mathcal{L}_t$ 都可写为

$$\mathcal{L}_t f(x)=\langle \mathcal{K}f(x), F_t(x)\rangle_x \tag{9.3}$$

其中函数 F_t 对每个 $x\in\mathcal{S}$ 满足 $F_t(x)\in\Omega_x$. 关键在于，算子 $\mathcal{K}$ 不能依赖于 $\mathcal{L}_t$，即只需学习 F_t. 由此得到

$$\textbf{参数化生成元：}\ \mathcal{L}_t^\theta f(x)=\left\langle \mathcal{K}f(x), F_t^\theta(x)\right\rangle_x,\ \text{其中神经网络为}F_t^\theta,\ \text{参数为}\theta, \tag{9.4}$$

其中 F_t^θ 仍将元素 $x\in\mathcal{S}$ 映为 $F_t^\theta(x)\in\Omega_x$. 下面列举几个例子，使这一定义更具体.

流的线性参数化. 令 $\mathcal{S}=\mathbb{R}^d$，$\Omega_x=\mathbb{R}^d=V_x$. 考虑所有流，即生成元族由下式给出（见 式 (8.15)）：

$$\mathcal{L}_t f=\nabla f^T u_t,\quad u_t:\mathbb{R}^d\to\mathbb{R}^d. \tag{9.5}$$

取 $\mathcal{K}f=\nabla f$ 和 $F_t=u_t$，就恢复了 式 (9.3) 的形式. 这给出了通过向量场对流生成元进行的自然线性参数化.

扩散的线性参数化. 令 $\mathcal{S}=\mathbb{R}^d$，$\Omega_x=S_d^{++}\subset\mathbb{R}^{d\times d}=V_x$，其中 S_d^{++} 表示所有半正定矩阵的集合. 则扩散生成元为（见 式 (8.26)）：

$$\mathcal{L}_t f=\nabla^2 f\cdot\sigma_t^2,\quad \sigma_t:\mathbb{R}^d\to S_d^{++} \tag{9.6}$$

取 $\mathcal{K}f=\nabla^2 f$ 和 $F_t=\sigma_t^2$，就恢复了 式 (9.3) 的形式. 这给出了扩散生成元的自然线性参数化.

跳跃的线性参数化. 令 $\Omega_x = \{a : \mathcal{S} \setminus \{x\} \to \mathbb{R}_{\geq 0} \,|\, a \text{ 可积}\} \subset L^2(\mathcal{S} \setminus \{x\}) = V_x$，内积为 $\langle a, b \rangle_x = \int_{\mathcal{S}\setminus\{x\}} a(x)b(x)\nu(dx)$. 则跳跃生成元为（见 式 (8.36)）：

$$\mathcal{L}_t f(x) = \int [f(y) - f(x)] Q_t(y, x) \nu(\mathrm{d}y) = \langle \mathcal{K} f(x), Q_t(\cdot; x) \rangle_x \tag{9.7}$$

其中令 $\mathcal{K}f(x)$ 为函数 $y \mapsto f(y) - f(x)$. 取 $F_t = Q_t$，就恢复了 式 (9.3) 的形式，给出了跳跃生成元的线性参数化. 注意，上述方式只参数化具有跳跃核 $Q_t(y, x)$ 的跳跃，并不一定包含所有跳跃测度.

CTMC 的线性参数化. 令 $\mathcal{S}$ 为离散空间，$u_t \in \mathbb{R}^{\mathcal{S}\times\mathcal{S}}$ 为连续时间马尔可夫链的速率矩阵. 与离散流匹配一样（见 式 (7.5)），定义

$$\Omega_x = \left\{ v \in \mathbb{R}^{\mathcal{S}} \;\middle|\; v(y) \geq 0 \ \forall y \neq x, \ \text{且} v(x) = -\sum_{y \neq x} v(y) \right\} \subset V_x = \mathbb{R}^{\mathcal{S}}. \tag{9.8}$$

由 式 (8.38)，对于 $f \in \mathbb{R}^{\mathcal{S}}$，生成元为

$$\mathcal{L}_t f(x) = f^T u_t(\cdot, x) = \langle f, u_t(\cdot, x) \rangle_x \tag{9.9}$$

其中 $V_x = \mathbb{R}^S$，$\mathcal{K}f = f$，$\langle \cdot, \cdot \rangle_x$ 是标准欧几里得内积. 由此恢复了 式 (9.3) 的形式. 因此，这给出了通过速率 u_t 对 CTMC 进行的自然线性参数化.

流形上流的线性参数化. 令 $\mathcal{S} = \mathcal{M}$ 为黎曼流形，与 节 5 一样，考虑黎曼流形上的流. 由 式 (8.39)，生成元为

$$\mathcal{L}_t f(x) = \langle \nabla f(x), u_t(x) \rangle_g \tag{9.10}$$

其中 u_t 是时变光滑向量场 $u_t : [0, 1] \times \mathcal{M} \to T\mathcal{M}$，且对所有 $x \in \mathcal{M}$，$u_t(x) \in T_x\mathcal{M}$. 取 $\Omega_x = V_x = T_x\mathcal{M}$，并令 $\mathcal{K} = \nabla f$ 为梯度算子，就恢复了 式 (9.3) 的形式. 因此，这给出了黎曼流生成元的自然线性参数化.

9.4 边缘生成元与条件生成元

本节说明如何寻找边缘概率路径的生成元. 方法如下：先寻找条件概率路径 $p_{t|Z}(\mathrm{d}x|z)$ 的生成元（通常可解析得到），再用其构造边缘路径的生成元. 具体而言，假设对每个 $z \in \mathcal{Z}$，都找到了生成 $p_{t|Z}(\mathrm{d}x|z)$ 的条件生成元 $\mathcal{L}^z$；由 定理 17，这等价于 KFE（式 (8.40)）：

$$\frac{\mathrm{d}}{\mathrm{d}t} \langle p_{t|Z}(\cdot|z), f \rangle = \langle p_{t|Z}(\cdot|z), \mathcal{L}_t^z f \rangle \quad \text{对所有} f \in \mathcal{T}. \tag{9.11}$$

此外，假设找到了如下线性参数化（见 式 (9.3)）：

$$\mathcal{L}_t^z f(x) = \langle \mathcal{K} f(x), F_t(x|z) \rangle_x \quad z \in \mathcal{Z} \tag{9.12}$$

其中函数满足 $F_t(x|z) \in \Omega_x \subset V_x$. 例如，$F_t(x|z)$ 可以是连续流匹配中的条件速度场（见 小节 4.3），或离散流匹配中的条件速率（见 式 (7.2)）. 由此可得到生成边缘路径的生成元公式：

定理 19 (一般边缘化技巧). 边缘概率路径 $(p_t)_{0\le t\le 1}$ 由马尔可夫过程 X_t 生成，其生成元为

$$\mathcal{L}_t f(x) = \mathbb{E}_{Z\sim p_{Z|t}(\cdot|x)}[\mathcal{L}_t^Z f(x)] \tag{9.13}$$

其中 $p_{Z|t}(\mathrm{d}z|x)$ 为后验分布（即给定 x 时 z 的条件分布）. 生成元 $\mathcal{L}_t$ 具有如下线性参数化：

$$F_t(x) = \mathbb{E}_{Z\sim p_{Z|t}(\cdot|x)}[F_t(x|Z)]. \tag{9.14}$$

上述定理给出了训练目标：用神经网络近似 式 (9.13) 中的 $\mathcal{L}_t$. 前面各章的边缘化技巧（定理 3、定理 10、定理 14）都是该定理的特例. 下面给出证明，然后展示几个新实例.

证明. 为证明 $\mathcal{L}_t$ 生成 p_t，由 定理 17，需要证明 KFE 成立. 令 $p_{t+h|t}(\cdot|x,z)$ 为 $\mathcal{L}_t^z$ 的转移核. 则：

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}t}\langle p_t, f\rangle &= \lim_{h\to 0}\frac{1}{h}\left(\langle p_{t+h}, f\rangle - \langle p_t, f\rangle\right)\\
&= \lim_{h\to 0}\frac{1}{h}(\mathbb{E}_{Z\sim p_Z, X'\sim p_{t+h|Z}(\cdot|Z)}(f(X')) - \mathbb{E}_{Z\sim p_Z, X\sim p_{t|Z}(\cdot|z)}(f(X)))\\
&= \lim_{h\to 0}\frac{1}{h}(\mathbb{E}_{Z\sim p_Z, X\sim p_{t|Z}(\cdot|Z), X'\sim p_{t+h|t}(\cdot|X,Z)}(f(X')) - \mathbb{E}_{Z\sim p_Z, X\sim p_{t|Z}(\cdot|z)}(f(X)))\\
&= \lim_{h\to 0}\frac{1}{h}(\mathbb{E}_{Z\sim p_Z, X\sim p_{t|Z}(\cdot|Z), X'\sim p_{t+h|t}(\cdot|X,Z)}(f(X') - f(X)))\\
&= \lim_{h\to 0}\frac{1}{h}\mathbb{E}_{X\sim p_t}\left(\mathbb{E}_{Z\sim p_{t|Z}(\cdot|X)}\left(\left(\mathbb{E}_{(X'\sim p_{t+h|t}(\cdot|X,Z)}(f(X')) - f(X)\right)\right)\right)\\
&= \mathbb{E}_{X\sim p_t}\left(\mathbb{E}_{Z\sim p_{Z|t}(\cdot|X)}\left(\lim_{h\to 0}\frac{1}{h}\left(\mathbb{E}_{X'\sim p_{t+h|t}(\cdot|X,Z)}(f(X')) - f(X)\right)\right)\right)\\
&= \mathbb{E}_{X\sim p_t}(\underbrace{\mathbb{E}_{Z\sim p_{Z|t}(\cdot|X)}(\mathcal{L}_t^z f(X))}_{=:\mathcal{L}_t f(X)})\\
&= \langle p_t, \mathcal{L}_t f\rangle
\end{aligned}$$

F_t 的形式由下式得到：

$$\mathbb{E}_{Z\sim p_{Z|t}(\cdot|X)}(\mathcal{L}_t^z f(x)) \overset{(9.12)}{=} \mathbb{E}_{Z\sim p_{Z|t}(\cdot|X)}(\langle \mathcal{K}f(x), F_t(x|z)\rangle_x) = \left\langle \mathcal{K}f(x), \mathbb{E}_{Z\sim p_{Z|t}(\cdot|X)}(F_t(x|z))\right\rangle_x = \langle \mathcal{K}f(x), F_t(x)\rangle_x$$

其中利用内积的线性性，将其与期望交换. 这表明 F_t 是边缘生成元的线性参数化（见 式 (9.3)）. □

例子：跳跃. 令 $\mathcal{S}$ 为任意空间，$Q_t(y,x|z)$ 为 $\mathcal{S}$ 上的条件跳跃核，其中 $y,x\in\mathcal{S}, z\in\mathcal{Z}$，它生成条件概率路径 $p_{t|Z}(\mathrm{d}x|z)$. 利用跳跃核的线性参数化（见 式 (9.7)），得到 **边缘跳跃核**

$$Q_t(y,x) = \mathbb{E}_{Z\sim p_{Z|t}(\cdot|x)}[Q_t(y,x|z)].$$

生成边缘概率 $p_t(\mathrm{d}x)$.

例子：边缘扩散系数. 令 $\mathcal{S}=\mathbb{R}^d$，$\sigma_t^2(x|z)$ 为生成条件概率路径 $p_{t|Z}(\mathrm{d}x|z)$ 的扩散系数. 利用扩散系数的线性参数化（见 式 (9.6)），得到 **边缘扩散系数**

$$\sigma_t^2(x) = \mathbb{E}_{Z\sim p_{Z|t}(\cdot|x)}[\sigma_t^2(x|Z)]$$

生成边缘概率路径 $p_t(\mathrm{d}x)$.

9.5 生成元匹配损失

接下来的目标是建立学习 CTMP 模型的训练目标. 假设神经网络 F_t^θ 给出 式 (9.4) 中的生成元参数化 $\mathcal{L}_t^\theta$. 如 定理 19 所推导，目标是近似 式 (9.14) 给出的真实边缘线性参数化 F_t.

与前面一样，假设对每个 $x \in \mathcal{S}$，都有 Bregman 散度 $D_x : \Omega_x \times \Omega_x \to \mathbb{R}$，定义为

$$D_x(a,b) = \Phi_x(a) - [\Phi_x(b) + \langle a-b, \nabla\Phi_x(b)\rangle], \quad a,b \in \Omega_x \tag{9.15}$$

其中 $\Phi_x : \Omega_x \to \mathbb{R}$ 为严格凸函数（见 图 10）. 用于训练 CTMP 模型的 **生成元匹配损失** 定义为

$$\mathcal{L}_{\mathrm{GM}}(\theta) = \mathbb{E}_{t,X_t\sim p_t} D_{X_t}(F_t(X_t), F_t^\theta(X_t)), \tag{9.16}$$

其中 $t \sim U[0,1]$. 遗憾的是，上述训练目标不可计算，因为我们既不知道边缘生成元 $\mathcal{L}_t$，也不知道其参数化 F_t（只知道 式 (9.14) 中不可计算的公式）. 因此，引入 **条件生成元匹配损失** 作为可计算的替代目标，其形式为

$$\mathcal{L}_{\mathrm{CGM}}(\theta) = \mathbb{E}_{t,Z,X_t\sim p_{t|Z}} D_{X_t}(F_t(X_t|Z), F_t^\theta(X_t)). \tag{9.17}$$

这一目标可计算，因为在许多情形下都能解析地推导 $F_t(x|z)$（见 小节 9.6）. 下一个定理表明，两个损失 (9.16) 和 (9.17) 提供相同的学习梯度.

定理 20. *生成元匹配损失与条件生成元匹配损失的梯度相同：*

$$\nabla_\theta \mathcal{L}_{GM}(\theta) = \nabla_\theta \mathcal{L}_{CGM}(\theta). \tag{9.18}$$

特别地，条件生成元匹配损失的最小化解是边缘生成元的线性参数化（式 (9.14)）：

$$F_t^\theta(x) = \mathbb{E}_{Z\sim p_{Z|t}(\cdot|x)}[F_t(x|Z)]. \tag{9.19}$$

此外，要使这些性质成立，D_x **必须** *是 Bregman 散度.*

上述定理将前面各节推导的 定理 4、定理 11 和 定理 15 推广到一般 CTMP 模型. 通过最小化条件生成元匹配损失，可以轻松且可扩展地训练任何由神经网络 F_t^θ 参数化的 CTMP 模型. 此外，它还普适地刻画了损失函数空间. 定理 20 的证明与 定理 4 的证明相同，只需将 u_t 替换为 F_t. 关于 D 必须为 Bregman 散度的证明，参见 (Holderrieth et al., 2024).

例子：训练扩散系数. 下面说明 定理 20 如何使我们能够训练随机微分方程的扩散系数. 令 $\mathcal{S} = \mathbb{R}^d$，$\sigma_t^2(x|z)$ 为生成条件概率路径 $p_{t|Z}(\mathrm{d}x|z)$ 的扩散系数. 可以用神经网络 $(\sigma_t^2)^\theta(x) \in \mathbb{R}^{d\times d}$ 参数化扩散系数. 条件生成元匹配损失于是成为

$$\mathcal{L}_{\mathrm{CGM}}(\theta) = \mathbb{E}_{t,Z,X_t\sim p_{t|Z}} \|\sigma_t^2(X_t|Z) - (\sigma_t^2)^\theta(X_t)\|^2$$

其中使用均方误差作为 Bregman 散度（也可选择许多其他散度）. (Holderrieth et al., 2024) 展示了用这种方式训练模型的例子.

9.6 通过求解 KFE 寻找条件生成元

为了用条件生成元匹配损失进行可扩展训练（见 定理 20），需要能够找到求解 KFE 的条件生成元 $\mathcal{L}_t^z$：

$$\frac{\mathrm{d}}{\mathrm{d}t}\left\langle p_{t|Z}(\cdot|z), f\right\rangle = \left\langle p_{t|Z}(\cdot|z), \mathcal{L}_t^z f\right\rangle \quad \text{对所有} f \in \mathcal{T}, z \in \mathcal{Z}. \tag{9.20}$$

如果 $p_{t|Z}(\mathrm{d}x|z)$ 相对于 ν 具有密度 $p_{t|Z}(x|z)$，则可以等价地求解伴随 KFE：

$$\frac{\mathrm{d}}{\mathrm{d}t}p_{t|Z}(x|z) = [(\mathcal{L}_t^z)^* p_{t|Z}(\cdot|z)](x) \quad \text{对所有} x \in \mathcal{S}, z \in \mathcal{Z}. \tag{9.21}$$

一般而言，式 (9.20) 和 式 (9.21) 很难解析求解，也没有适用于任意生成元的通用解法. 因此，下面给出两个例子说明如何进行求解.

这里用跳跃模型来说明，因为它们适用于任意状态空间. 如 小节 8.2.2 所述，跳跃模型由可分解为如下形式的跳跃测度 Q_t 指定：

$$Q_t(\mathrm{d}y, x) = \lambda_t(x) J_t(\mathrm{d}y, x) \quad \text{对所有} x \in \mathcal{S} \tag{9.22}$$

$$\lambda_t(x) \geq 0 \quad \text{对所有} x \in \mathcal{S} \tag{9.23}$$

$$\int J_t(\mathrm{d}y, x) = 1 \quad \text{对所有} x \in \mathcal{S} \tag{9.24}$$

其中 $\lambda_t(x)$ 描述跳跃强度，J_t 描述指定跳跃分布的概率核. 为简化记号，省略对 $z \in \mathcal{Z}$ 的依赖（但在 $p_{t|Z}(\mathrm{d}x|z)$ 中保留，以免与边缘概率路径混淆）.

凸混合的跳跃模型. 考虑如下混合概率路径（见 式 (7.22)）：

$$p_{t|Z}(\mathrm{d}x|z) = \kappa_t \delta_z(\mathrm{d}x) + (1 - \kappa_t) p(\mathrm{d}x), \quad z \in \mathcal{S}. \tag{9.25}$$

利用跳跃过程生成元的形式（见 式 (8.36)），KFE 成为：

$$\frac{\mathrm{d}}{\mathrm{d}t}\left\langle p_{t|Z}(\mathrm{d}x|z), f\right\rangle = \mathbb{E}_{X \sim p_{t|Z}(\cdot|z)} \lambda_t(X) \mathbb{E}_{Y \sim J_t(\mathrm{d}y, X)}[f(Y) - f(X)] \quad \text{对所有} f \in \mathcal{T}, x \in \mathcal{S} \tag{9.26}$$

其中 λ_t, J_t 满足 式 (9.23) 和 式 (9.24) 中的约束. 我们断言，以下跳跃模型满足这一方程：

$$Q_t(\mathrm{d}y, x) = \lambda_t(x) J_t(\mathrm{d}y, x), \quad \lambda_t(x) = \frac{\dot{\kappa}_t}{1 - \kappa_t}, \quad J_t(\mathrm{d}y, x) = \delta_z(\mathrm{d}y)$$

即跳跃强度由 λ_t 给出，一旦决定跳跃，就直接跳到 $z \in \mathcal{S}$. 为证明这一点，下面验证上述跳跃过程满足 KFE. 可以推导：

$$\begin{aligned}
\mathbb{E}_{X\sim p_{t|Z}(\cdot|z)}\left[\lambda_t(X)\mathbb{E}_{Y\sim J_t(\cdot,X)}[f(Y)-f(X)]\right] &= \frac{\dot\kappa_t}{1-\kappa_t}\mathbb{E}_{X\sim p_{t|Z}(\cdot|z)}[f(z)-f(X)] \\
&= \frac{\dot\kappa_t}{1-\kappa_t}[f(z)-\mathbb{E}_{X\sim p_{t|Z}(\cdot|z)}[f(X)]] \\
&= \frac{\dot\kappa_t}{1-\kappa_t}[f(z)-[\kappa_t f(z)+(1-\kappa_t)\mathbb{E}_{X\sim p}f(X)] \\
&= \dot\kappa_t f(z)-\dot\kappa_t\mathbb{E}_{X\sim p}f(X) \\
&= \frac{\mathrm{d}}{\mathrm{d}t}\left[\kappa_t f(z)+(1-\kappa_t)\mathbb{E}_{x\sim p}[f(x)]\right] \\
&= \frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}_{X\sim p_{t|Z}(\cdot|z)}[f(X)] \\
&= \frac{\mathrm{d}}{\mathrm{d}t}\left\langle p_{t|Z}(\cdot|z), f\right\rangle .
\end{aligned}$$

因此，该过程满足跳跃 KFE（式 (9.26)），从而建立了跳跃模型. 我们在 式 (7.24) 中见过该模型在离散状态空间上的特殊例子. 这里则表明，例如也可以在欧几里得空间 $\mathbb{R}^d$ 上建立类似的跳跃模型.

具有密度的任意路径的跳跃模型. 假设概率 $p_{t|Z}(\mathrm{d}x|z)$ 相对于 $\mathcal{S}$ 上的参考测度 ν 具有密度 $p_{t|Z}(x|z)$，且关于 t 可微（注意，当 $\mathcal{S}=\mathbb{R}^d$ 时，式 (9.25) 中的混合路径不满足这一要求）. 此外，只考虑具有密度的跳跃核 $J_t(y,x)$. 于是伴随 KFE 成为跳跃连续性方程（式 (8.66)）：

$$\frac{\mathrm{d}}{\mathrm{d}t}p_{t|Z}(x|z) = \int \lambda_t(y)J_t(x,y)p_{t|Z}(y|z)dy - p_{t|Z}(x|z)\lambda_t(x) \tag{9.27}$$

$$\Leftrightarrow \quad p_{t|Z}(x|z)\left[\frac{\mathrm{d}}{\mathrm{d}t}\log p_{t|Z}(x|z)+\lambda_t(x)\right] = \int \lambda_t(y)J_t(x,y)p_{t|Z}(y|z)dy \tag{9.28}$$

令 $J_t(x,y)=J_t(x)$（“与目标状态无关”），并使用 $\partial_t=\frac{\mathrm{d}}{\mathrm{d}t}$，得到等价形式：

$$p_{t|Z}(x|z)\left[\partial_t\log p_{t|Z}(x|z)+\lambda_t(x)\right] = J_t(x)\int \lambda_t(y)p_{t|Z}(y|z)\nu(dy) \tag{9.29}$$

$$\Leftrightarrow \quad \frac{p_{t|Z}(x|z)\left[\partial_t\log p_{t|Z}(x|z)+\lambda_t(x)\right]}{\int \lambda_t(y)p_{t|Z}(y|z)\nu(dy)} = J_t(x) \tag{9.30}$$

为定义有效的跳跃过程，要求 λ_t, J_t 满足 $\lambda_t(x)\geq 0$ 和 $J_t(x)\geq 0$. 因此得到：

$$\lambda_t(x)\geq 0,\ J_t(x)\geq 0 \quad \Leftrightarrow \quad \lambda_t(x)\geq [-\partial_t\log p_t(x|z)]_+ \tag{9.31}$$

其中 $[x]_+ = \max(x, 0)$ 表示 ReLU 运算. 此外，要求 J_t 定义有效的跳跃分布，即积分为 1. 可以验证这一点：

$$
\begin{aligned}
& 1 = \int J_t(x)dx \\
\Leftrightarrow \quad & \int \lambda_t(x) p_{t|Z}(x|z)\nu(\mathrm{d}x) = \int p_{t|Z}(x|z)\left[\partial_t \log p_{t|Z}(x|z) + \lambda_t(x)\right]\nu(\mathrm{d}x) \\
\Leftrightarrow \quad & 0 = \int \partial_t p_{t|Z}(x|z)\nu(dx) \\
\Leftrightarrow \quad & 0 = \partial_t \int p_{t|Z}(x|z)\nu(dx) \\
\Leftrightarrow \quad & 0 = 0
\end{aligned}
$$

即 J_t 的积分确实为 1. 选择最小的 $\lambda_t(x)$，得到如下定义的跳跃模型：

$$
\begin{aligned}
\lambda_t(x) &= \left[-\partial_t \log p_{t|Z}(x|z)\right]_+, \\
J_t(x) &= \frac{p_{t|Z}(x|z)[\partial_t \log p_{t|Z}(x|z)]_+}{\int p_{t|Z}(y|z)[\partial_t \log p_{t|Z}(y|z)]_+\nu(\mathrm{d}y)} = \frac{[\partial_t p_{t|Z}(x|z)]_+}{\int [\partial_t p_{t|Z}(y|z)]_+\nu(\mathrm{d}y)}
\end{aligned}
$$

是跳跃连续性方程的解，因此生成条件概率路径 $p_{t|Z}(x|z)$. 起初，跳跃分布与位置无关似乎不尽如人意. 然而，将该模型扩展到多维后，跳跃分布就会依赖位置，并产生强大的生成模型 (Holderrieth et al., 2024).

9.7 组合模型

本节说明 GM 如何以不同方式组合相同状态空间 $\mathcal{S}$ 上的生成模型. 基本原理很简单：生成元是线性算子，KFE $\partial_t \langle p_t, f\rangle = \langle p_t, \mathcal{L}_t f\rangle$ 是线性方程，因此可以像在线性代数中组合矩阵方程的解一样，组合这一方程的解. 具体而言，令 $\mathcal{L}_t, \mathcal{L}'_t$ 为两个马尔可夫过程的生成元，它们都求解概率路径 p_t 的 KFE. 则对满足 $\alpha_t^1 + \alpha_t^2 = 1$ 的 $\alpha_t^1, \alpha_t^2 \in \mathbb{R}$，有：

$$
\begin{aligned}
& \left\langle p_t, (\alpha_t^1 \mathcal{L}_t + \alpha_t^2 \mathcal{L}'_t) f \right\rangle && (9.32)\\
&= \alpha_t^1 \left\langle p_t, \mathcal{L}_t f\right\rangle + \alpha_t^2 \left\langle p_t, \mathcal{L}'_t f\right\rangle && (9.33)\\
&= \alpha_t^1 \partial_t \left\langle p_t, f\right\rangle + \alpha_t^2 \partial_t \left\langle p_t, f\right\rangle && (9.34)\\
&= (\alpha_t^1 + \alpha_t^2)\partial_t \left\langle p_t, f\right\rangle && (9.35)\\
&= \partial_t \left\langle p_t, f\right\rangle, && (9.36)
\end{aligned}
$$

即 $\alpha_t^1 \mathcal{L}_t + \alpha_t^2 \mathcal{L}'_t$ 仍是 KFE 的解. 一个细微但重要的细节是，α_t^1, α_t^2 为正还是为负，以及 $\mathcal{L}_t, \mathcal{L}'_t$ 对应时间正向还是反向的马尔可夫过程. 由此得到：

命题 3 (组合模型)**.** *令 p_t 为边缘概率路径，则下列生成元都求解 p_t 的 KFE，因此定义了以 p_t 为边缘分布的生成模型：*

1. **马尔可夫叠加：** *$\alpha_t^1 \mathcal{L}_t + \alpha_t^2 \mathcal{L}'_t$，其中 $\mathcal{L}_t, \mathcal{L}'_t$ 是求解 p_t 的 KFE 的两个马尔可夫过程生成元，$\alpha_t^1, \alpha_t^2 \geq 0$ 满足 $\alpha_t^1 + \alpha_t^2 = 1$.*
2. **无散度分量：** *$\mathcal{L}_t + \beta_t \mathcal{L}_t^{\mathrm{div}}$，其中生成元 $\mathcal{L}_t^{\mathrm{div}}$ 对所有 $f \in \mathcal{T}$ 满足 $\left\langle p_t, \mathcal{L}_t^{\mathrm{div}} f\right\rangle = 0$，且 $\beta_t \geq 0$. 称这样的 $\mathcal{L}_t^{\mathrm{div}}$ 为 **无散度的**.*
3. **预测校正：** *$\alpha_t^1 \mathcal{L}_t + \alpha_t^2 \bar{\mathcal{L}}_t$，其中 $\mathcal{L}_t$ 是沿时间正向求解 p_t 的 KFE 的生成元，$\bar{\mathcal{L}}_t$ 是沿时间反向求解 KFE 的生成元，$\alpha_t^1, \alpha_t^2 \geq 0$ 满足 $\alpha_t^1 - \alpha_t^2 = 1$.*

下面用马尔可夫叠加和无散度分量的例子说明 命题 3. 关于预测校正方案的作用，可参见 (Gat et al., 2024) 中的例子.

马尔可夫叠加的例子：组合跳跃与流. 马尔可夫叠加可以组合不同类别的生成模型. 它们可以是分别训练的两个网络，也可以在一个网络中同时训练两个 GM 模型 (Holderrieth et al., 2024). 这里通过组合 $\mathcal{S} = \mathbb{R}^d$ 上的跳跃模型和流模型说明这一点. 假设有两个模型，各自都生成概率路径 p_t：流模型 u_t，以及跳跃强度为 λ_t、跳跃分布为 J_t 的跳跃模型. 由 命题 3，对于满足 $\alpha_t^1 + \alpha_t^2 = 1$ 的 $\alpha_t^1, \alpha_t^2 \geq 0$，以下生成元定义了生成 p_t 的有效 GM 模型：

$$\begin{aligned}\mathcal{L}_t f(x) &= \alpha_t^1 \mathcal{L}_t^{\text{跳跃}} f(x) + \alpha_t^2 \mathcal{L}_t^{\text{流}} f(x) \\ &= (\alpha_t^1 \lambda_t(x)) \mathbb{E}_{Y \sim J_t(\cdot, x)}[f(Y) - f(x)] + \nabla f^T(x)(\alpha_t^2 u_t(x))\end{aligned}$$

其中使用了 式 (8.17) 和 式 (8.36). 实际上，上述生成元描述一个 **分段确定性马尔可夫过程**，即组合常微分方程与跳跃的模型 (Davis, 1984). 如上式所示，需将跳跃强度乘以 α_t^1，将向量场乘以 α_t^2. 可以用下列采样步骤从所得 GM 模型采样：

$$\begin{aligned}X_0 &\sim p_0 = p \\ X_{t+h} &= \begin{cases} \sim J_t(\mathrm{d}y, X_t) & \text{概率为} h\alpha_t^1 \lambda_t(X_t) \\ X_t + h\alpha_t^2 u_t(X_t) & \text{概率为} 1 - h\alpha_t^1 \lambda_t(X_t) \end{cases}\end{aligned}$$

(Holderrieth et al., 2024) 给出了几个跳跃与流的马尔可夫叠加例子，展示了这种组合带来的性能提升.

无散度的例子：MCMC 算法. 可以用现有的马尔可夫链蒙特卡洛（MCMC）算法寻找无散度分量；这些算法都给出了寻找无散度分量的一般方法. 下面用两个著名例子说明. 假设给定具有密度 $p_t(x)$ 的一般概率路径 p_t. 则生成元 $\mathcal{L}_t^{\text{div}}$ 无散度，等价于其伴随将 p_t 映为零：

$$\left\langle p_t, \mathcal{L}_t^{\text{div}} f \right\rangle = 0 \text{ 对所有} f \in \mathcal{T} \quad \Leftrightarrow \quad [\mathcal{L}_t^{\text{div}}]^* p_t(x) = 0 \text{ 对所有} x \in \mathcal{S} \tag{9.37}$$

首先考虑 $\mathcal{S} = \mathbb{R}^d$. **朗之万动力学** 对应于速度场为 $\frac{1}{2}\beta_t^2 \nabla \log p_t(x)$、扩散系数为 β_t 的随机微分方程，即动力学为

$$dX_t = \frac{1}{2}\beta_t^2 \nabla \log p_t(x) dt + \beta_t dW_t \tag{9.38}$$

该随机微分方程的伴随生成元为

$$\begin{aligned}[\mathcal{L}_t^{\text{div}}]^* p_t &\overset{(i)}{=} -\operatorname{div}(p_t \frac{1}{2}\beta_t^2 \nabla \log p_t)(x) + \frac{1}{2}\beta_t^2 \Delta p_t(x) \\ &\overset{(ii)}{=} -\frac{1}{2}\operatorname{div}(\beta_t^2 \nabla p_t)(x) + \frac{1}{2}\beta_t^2 \Delta p_t(x) \\ &\overset{(iii)}{=} -\frac{1}{2}\beta_t^2 \Delta p_t(x) + \frac{1}{2}\beta_t^2 \Delta p_t(x) = 0\end{aligned}$$

其中，(i) 来自 小节 8.3.1 推导的流与扩散的伴随形式；(ii) 来自 $\nabla \log p_t = \nabla p_t / p_t$；(iii) 来自恒等式 $\operatorname{div}\nabla = \Delta$. 这表明朗之万动力学的生成元满足 式 (9.37)，因此在 命题 3 的意义下无散度. 这一事实广泛用于统计物理和马尔可夫链蒙特卡洛 (Roberts and Tweedie, 1996). 命题 3 表明，对任意 $\beta_t \geq 0$，都可以将这些动力学加入任意生成模型. 在 节 10 中，利用这一点推导扩散模型的随机采样.

其次，再次令 $\mathcal{S}$ 为一般状态空间. **Metropolis–Hastings** 算法 (Hastings, 1970) 给出跳跃核 $Q_t(y,x)$ 的跳跃过程构造，使其满足 **细致平衡条件**：

$$\begin{aligned} & Q_t(y,x)p_t(x) = Q_t(x,y)p_t(y) \quad \text{对所有} x,y \in \mathcal{S} \\ \Rightarrow \quad & [\mathcal{L}_t^{\text{div}}]^* p_t(x) \overset{(i)}{=} \int Q_t(y,x)p_t(x) - Q_t(x,y)p_t(y) = 0 \end{aligned}$$

其中 (i) 使用了 式 (8.66). 这表明 式 (9.37) 成立，Q_t 无散度. 命题 3 表明，可以任意地将这样的 Metropolis 方案加入任何沿概率路径 p_t 演化的 GM 模型.

9.8 多模态模型

最后讨论 GM 如何支持在多种数据模态上联合构建生成模型. 例如，可以同时生成图像及相应的文本描述. 两种模态表示为两个状态空间 $\mathcal{S}_1, \mathcal{S}_2$（例如 $\mathcal{S}_1$ 是图像，$\mathcal{S}_2$ 是文本），多模态模型则是乘积空间 $\mathcal{S} = \mathcal{S}_1 \times \mathcal{S}_2$ 上的生成模型. 由于 $\mathcal{S}$ 仍是状态空间，而 GM 适用于任意状态空间，可以像构建其他 GM 模型一样直接构建它. 不过，有一种特殊的概率路径构造，可以重用或“循环利用”为单独模态建立的 GM 模型. 例如，通过组合离散流匹配与连续流匹配模型，可以构建联合文本图像模型. 这一特殊构造依赖于 **因子化条件概率路径**. 我们已经在离散流匹配的 小节 7.5.2 中见过简单情形：因子化概率路径产生因子化速度. 对任意模态，这一结论更一般地成立. 尽管构造相当简单直观，但完全一般的表述涉及较多技术细节. 严格处理参见 (Holderrieth et al., 2024). (Campbell et al., 2024) 也在多模态蛋白质生成中实现了一个具体实例. 这说明 GM 可以有原则且严格地构建多模态模型.

10 与扩散模型及其他去噪模型的关系

最后，本节讨论与去噪扩散模型（DDM）以及非欧几里得空间上相关模型的关系. 主要关注 (Song et al., 2021) 通过随机微分方程构建扩散模型的方法，并解释如何将其纳入流匹配 / 生成元匹配模型族. 本节末尾还讨论从 DDM 获得启发的其他模态模型（“去噪模型”），以及如何将它们表述为 GM 模型.

10.1 时间约定

去噪扩散模型与流匹配的第一个简单区别，是时间参数化方式不同. 这只是一种约定，但为避免混淆需要注意. 与流匹配不同，扩散模型中的时间方向相反，取值从 0 到 ∞. 为区分这两种时间参数化，用 r 表示扩散模型的时间，用 t 表示流匹配的时间. 则有：

$$\textbf{流匹配时间 } t\text{：噪声} \equiv \text{“}t = 0\text{”}, \text{ 数据} \equiv \text{“}t = 1\text{”} \tag{10.1}$$

$$\textbf{扩散时间 } r\text{：噪声} \equiv \text{“}r = +\infty\text{”}, \text{ 数据} \equiv \text{“}r = 0\text{”} \tag{10.2}$$

$$\textbf{重新参数化：}\ r = k(t), \quad t = k^{-1}(r) \tag{10.3}$$

其中 $k : (0, 1] \to [0, +\infty)$ 是严格单调递减映射，满足 $k(1) = 0$ 和 $\lim_{t\to 0} k(t) = +\infty$.

10.2 前向过程与概率路径

去噪扩散模型的基本思想是构造一个破坏数据分布的 *前向过程*. 下面说明它如何对应于流匹配中使用的一种特定概率路径构造. 前向过程 X_r 由随机微分方程定义：

$$\mathrm{d}X_r = a_r(X_r)\mathrm{d}r + g_r \mathrm{d}W_r, \quad X_0 \sim q \tag{10.4}$$

其中 q 是数据分布，W_r 是布朗运动，$a : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ 为速度场（在随机微分方程中也称 *漂移*），$g : \mathbb{R} \to \mathbb{R}_{\geq 0}$ 为扩散系数（见 小节 8.2.2）. 每个这样的随机微分方程都定义如下条件概率路径和边缘概率路径：

$$\tilde{p}_{r|0}(x|z) = \mathbb{P}[X_t = x | X_0 = z], \qquad \tilde{p}_r(x) = \mathbb{P}[X_t = x] \tag{10.5}$$

$$p_{t|1}(x|z) = \tilde{p}_{k(t)|0}(x|z), \qquad p_t(x) = \tilde{p}_{k(t)}(x) \tag{10.6}$$

其中第二行将时间重新参数化为流匹配的时间. 可以看到，$p_{t|1}(x|z)$ 给出一条条件概率路径. 此外，前向过程的构造使得当 $R \gg 0$ 时，X_R 的分布近似为高斯分布. 因此得到：

扩散中的每个前向过程都定义了一条流匹配所使用的 “有效” 条件概率路径，即相应边缘路径在数据分布 q 和近似高斯分布 p 之间插值. 具体而言：

1. **确定性初始化：** 条件概率路径 $p_{t|1}(x|z)$ 对应于以 $X_0 = z$ 初始化时前向过程随机微分方程的分布.
2. **数据初始化：** 边缘概率路径 $p_t(x)$ 对应于以 $X_0 \sim q$ 初始化时前向过程的分布，其中 q 为数据分布.

扩散模型的一个重要要求是，能够以闭式计算条件概率 $\tilde{p}_{r|0}(x|z)$. 因此必须使用相应 KFE（即 Fokker–Planck 方程）具有解析解的随机微分方程. 所以，绝大多数文献都假设前向过程具有 **仿射漂移系数**，即 $a_r(x) = a_r x$，其中 $a : \mathbb{R} \to \mathbb{R}$ 为连续函数 (Song et al., 2021; Karras et al.,

2022). 这一假设使我们能够将给定 $X_0 = z \in \mathbb{R}^d$ 时 X_r 的条件分布表示为高斯分布 (Särkkä and Solin, 2019; Song et al., 2021; Karras et al., 2022)：

$$\tilde{p}_{r|0}(x|z) = \mathcal{N}\left(\tilde{\alpha}_r z, \tilde{\sigma}_r^2 I\right), \quad \tilde{\alpha}_r = \exp\left(\int_0^r a_w \mathrm{d}w\right), \quad \tilde{\sigma}_r^2 = \tilde{\alpha}_r^2 \int_0^r \frac{g_w^2}{\tilde{\alpha}_w^2} dw \tag{10.7}$$

$$\Rightarrow \quad p_{t|1}(x|z) = \mathcal{N}\left(\alpha_t z, \sigma_t^2 I\right) \qquad \alpha_t = \tilde{\alpha}_{k(t)}, \qquad \sigma_t = \tilde{\sigma}_{k(t)} \tag{10.8}$$

注意，我们已在 小节 4.8.3 中详细讨论了这种 **仿射高斯概率路径**，它们通过仿射条件流构造（见 小节 4.8）：

$$\psi_t(x|x_1) = \alpha_t z + \sigma_t x, \quad z \sim q,\ x \sim \mathcal{N}(\alpha_0, \sigma_0^2 I). \tag{10.9}$$

因此可以看到：

> 具有仿射漂移系数的前向过程，对应于使用 式 (10.8) 定义的 **高斯概率路径**（见 小节 4.8.3）.

注意，在上述时间参数化下，扩散模型不存在有限时刻 $r < +\infty$，使边缘分布 $\tilde{p}_r(x)$ 恰好为高斯分布.

10.3 训练扩散模型

现在讨论如何将扩散模型的训练算法恢复为流匹配训练的特例. 在 小节 4.8 中，讨论了用高斯概率路径参数化并训练流匹配模型的几种选择（见 定理 7）. 其中一种是 x_0 **预测**，通过训练神经网络 $x_{0|t}^\theta$ 近似

$$x_{0|t}^\theta \approx \mathbb{E}[X_0|X_t = x]$$

采用以下训练算法：

$$\begin{aligned}
\mathcal{L}_{\mathrm{CM}}(\theta) &= \mathbb{E}_{t,Z\sim q,X_0\sim p}\|x_{0|t}^\theta(\underbrace{\alpha_t X_0 + \sigma_t Z}_{=X_t}) - X_0\|^2 \\
&\overset{(i)}{=} \mathbb{E}_{t,Z\sim q,X_t\sim p_{t|1}(\cdot|Z)}\sigma_t^2\|s_t^\theta(X_t) - [-\frac{1}{\sigma_t^2}(X_t - \alpha_t Z)]\|^2 \\
&\overset{(ii)}{=} \mathbb{E}_{t,Z\sim q,X_t\sim p_{t|1}(\cdot|Z)}\sigma_t^2\|s_t^\theta(X_t) - \nabla \log p_{t|1}(X_t|Z)\|^2,
\end{aligned}$$

其中，(i) 通过 $s_t^\theta = -x_{0|t}^\theta/\sigma_t$ 重新参数化神经网络，(ii) 使用 式 (4.71). 上述损失也称 **去噪得分匹配** 损失 (Vincent, 2011)，是训练扩散模型的基本损失函数. 由 定理 7，最小化解 θ^* 满足

$$s_t^{\theta^*}(x) = -\frac{1}{\sigma_t}\mathbb{E}[X_0|X_t = x] \overset{(4.77)}{=} \nabla \log p_t(x), \tag{10.10}$$

即在损失最小时，s_t^θ 等于边缘概率路径的得分函数 $\nabla \log p_t(x)$. 因此，网络 s_t^θ 也称得分网络. 可以总结为：

扩散模型的训练算法 **等价于** 用 x_0 预测训练一个特定的流匹配模型. 具体而言，除了重新参数化时间，它等同于训练具有以下特征的流匹配模型：

1. **概率路径：** 使用独立耦合的高斯概率路径，通过具有仿射漂移系数的随机微分方程构造（α_t, σ_t 由 (10.7) 定义）.
2. **得分参数化：** 通过得分函数重新参数化边缘速度场.

表 1 列出了如何方便地将得分网络转换为其他速度参数化. 因此，不同参数化在理论上等价，甚至可以在训练后切换参数化（见 小节 4.8.1）. 不过，得分参数化和 x_0 预测参数化在时刻 $t = 0$（接近噪声）引入了奇点.

10.4 采样

接下来讨论从扩散模型采样，以及它与从流匹配或生成元匹配模型采样的关系.

用常微分方程确定性采样. 如果将扩散模型视为流匹配模型，就应通过边缘向量场采样. 在 (4.78) 中，我们通过得分函数表示了高斯路径的边缘向量场：

$$u_t(x) = \frac{\dot{\alpha}_t}{\alpha_t} x - \left[\dot{\sigma}_t \sigma_t - \sigma_t^2 \frac{\dot{\alpha}_t}{\alpha_t}\right] \nabla \log p_t(x). \tag{10.11}$$

将 式 (10.7) 中 α_t, σ_t 的具体形式代入，可推导出等价恒等式：

$$u_t(x) = \dot{k}(t) \left[a_t x - \frac{g_t^2}{2} \nabla \log p_t(x)\right]. \tag{10.12}$$

也可以将上述 $u_t(x)$ 代入连续性方程直接验证. u_t 对应的常微分方程也称 **概率流常微分方程** (Song et al., 2021)：

$$\mathrm{d}X_t = \dot{k}(t) \left[a_t X_t - \frac{g_t^2}{2} s_t^\theta(X_t)\right] \mathrm{d}t, \tag{10.13}$$

其中取 $s_t^\theta(x) = \nabla \log p_t(x)$ 为学习得到的得分函数. 这里对常微分方程采用随机微分方程中常用的记号，原因将在下面说明. 还需注意，由于时间重新参数化，与 (Song et al., 2021) 相比，式 (10.11) 中多了因子 $\dot{k}(t)$.

用随机微分方程随机采样. 在 小节 9.7 中，我们推导出，可以将朗之万动力学

$$\frac{1}{2}\beta_t^2 \nabla \log p_t(x) dt + \beta_t dW_t \tag{10.14}$$

加入任何 CTMP 生成模型，所得模型仍沿相同概率路径演化. 将其应用于概率流常微分方程，就得到生成概率路径 p_t 的一整族随机微分方程：

$$dX_t = \left(\dot{k}(t)\alpha_t X_t + \frac{\beta_t^2 - \dot{k}(t) g_t^2}{2} \nabla \log p_t(X_t)\right) dt + \beta_t dW_t. \tag{10.15}$$

这给出了扩散模型的 **随机采样**. 理论上，对每个 $\beta_t \geq 0$，上述所有模型都产生相同的边缘分布. 这是关于真实底层随机过程的数学事实. 实践中，需要模拟随机微分方程

$$dX_t = \left(\dot{k}(t)\alpha_t X_t + \frac{\beta_t^2 - \dot{k}(t)g_t^2}{2} s_t^\theta(X_t) \right) dt + \beta_t dW_t \tag{10.16}$$

其中使用训练后的网络 s_t^θ. 我们既有估计误差（s_t^θ 的训练不完美），也有模拟误差（对底层随机微分方程的采样不完美）. 因此，存在未知的最优噪声水平 β_t（例如见 (Albergo and Vanden-Eijnden, 2022, 式 (2.45))），可以通过经验 (Karras et al., 2022) 或理论 (Ma et al., 2024) 来确定.

因此得到：

1. **常微分方程采样**： 对于高斯源分布、独立耦合、按 (10.7) 固定的 α_t, σ_t，以及得分参数化，用概率流常微分方程从扩散模型采样，与从流匹配模型采样相同.
2. **随机微分方程采样**： 在相同条件下，通过随机微分方程随机采样从扩散模型采样，等价于从 式 (10.15) 定义的 GM 模型采样.

10.5 时间反演与反向过程的作用

为结束对扩散模型的讨论，下面说明时间反演和反向过程在扩散模型中的作用. 时间反演在扩散模型中如此核心，因此有些读者可能会惊讶于流匹配不需要它. 这里给出更详细的解释. 具体而言，为生成数据，扩散模型将训练表述为学习一个从 0 演化到 $R > 0$ 的 *反向过程* $\bar{X}_r$，使得

$$\bar{X}_r \overset{d}{=} X_{R-r} \text{ 对所有} r \in [0, R] \tag{10.17}$$

其中 $\overset{d}{=}$ 表示同分布. 找到这样的过程后，可以用高斯分布初始化 $\bar{X}_0 \overset{d}{=} X_R$，再模拟它得到 $\bar{X}_R \overset{d}{=} X_0 \sim q$，即数据分布 q 的样本. 如果 $X_r \sim \tilde{p}_r$（即 式 (10.5) 中的边缘概率路径），则 式 (10.17) 等价于：

$$\bar{X}_r \sim \bar{p}_r := \tilde{p}_{R-r} \text{ 对所有} r \in [0, R].$$

换言之，希望随机过程 $\bar{X}_r$ 生成概率路径 $\bar{p}_r$. 但这正是生成元匹配对一般马尔可夫过程的目标（见 式 (8.10)），特别地，也是流匹配对流的目标. 因此得到：

以下问题等价：

1. **时间反演的边缘分布**： 为反向过程寻找随机微分方程（或常微分方程），使其具有与前向过程相同的边缘分布，正如扩散模型所做的那样.
2. **生成概率路径**： 寻找生成由前向过程定义的概率路径的随机微分方程（或常微分方程）.
3. **求解 KFE**： 寻找求解 Fokker–Planck 方程（或连续性方程）的随机微分方程（或常微分方程）.

扩散模型最初的描述包含了随机微分方程的“完整”时间反演 (Anderson, 1982). 这一概念强于本文使用的概念，即它要求各时刻的联合分布相同：

$$\mathbb{P}[\bar{X}_{r_1} \in A_1, \dots, \bar{X}_{r_n} \in A_n] = \mathbb{P}[X_{R-r_1} \in A_1, \dots, X_{R-r_n} \in A_n] \tag{10.18}$$

$$\text{对所有} 0 \le r_1, \dots, r_n, \text{且} A_1, \dots, A_n \subset S \text{ 可测.} \tag{10.19}$$

如 Anderson (1982) 所示，对 式 (10.15) 中的 β_t 作特定选择，就可以通过反向过程得到满足上述条件的时间反演. 然而，在生成建模中，通常只使用马尔可夫过程的最终状态 X_1（例如生成的图像），并舍弃更早时刻的状态. 因此，对于许多应用，马尔可夫过程是否是“真正”的时间反演，还是仅具有相同的边缘分布（如 式 (10.17)），并不重要. 一个著名例子是概率流常微分方程：它 不是 (Anderson, 1982) 意义下扩散过程的时间反演，但遵循相同的边缘分布. 这说明寻找“真正”的时间反演在数学上更困难，但对生成建模而言往往并非必要. 在神经网络评估次数较少的采样情形下，概率流常微分方程是当前最先进的方法 (Karras et al., 2022). 这表明，与 Fokker–Planck 方程的其他解相比，“真正”的时间反演甚至可能产生次优结果 (Ma et al., 2024).

10.6 与其他去噪模型的关系

受扩散模型成功的启发，将扩散模型的方法推广到其他状态空间取得了显著进展 (Campbell et al., 2022, 2024; De Bortoli et al., 2022; Huang et al., 2022a; Benton et al., 2022). 与扩散模型类似，这些方法构造给数据加噪的前向马尔可夫过程，再对其时间反演（即“去噪”）. 因此，非正式地将这类模型称为“去噪模型”. 自然会问，这些模型与 GM（见 节 9）有何关系. 这些“去噪模型”与 GM 模型的关系，正如扩散模型与流匹配的关系：

一般而言，去噪模型是具有以下特征的生成元匹配模型：

1. **时间约定：** 使用扩散时间约定，时刻 0 对应于数据.
2. **概率路径构造：** 通过“前向”或“加噪”过程构造概率路径.
3. **求解 KFE：** 通过前向过程的时间反演找到 KFE 的一个特定解.

我们承认这是一条 非正式规则，可能存在例外. 因此，关于相关工作的更广泛而详细的讨论（包括更完整的参考文献列表），参见 (Holderrieth et al., 2024).

参考文献

Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. *arXiv preprint arXiv:2209.15571*, 2022.

Michael Samuel Albergo, Mark Goldstein, Nicholas Matthew Boffi, Rajesh Ranganath, and Eric Vanden-Eijnden. Stochastic interpolants with data-dependent couplings. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24, 2024.

Luigi Ambrosio. Transport equation and cauchy problem forbvvector fields. *Inventiones mathematicae*, 158(2), 2004.

Brian DO Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

Heli Ben-Hamu, Samuel Cohen, Joey Bose, Brandon Amos, Maximillian Nickel, Aditya Grover, Ricky T. Q. Chen, and Yaron Lipman. Matching normalizing flows and probability paths on manifolds. *Proceedings of the 39th International Conference on Machine Learning*, 162, 2022.

Joe Benton, Yuyang Shi, Valentin De Bortoli, George Deligiannidis, and Arnaud Doucet. From denoising diffusions to denoising markov models. *arXiv preprint arXiv:2211.03595*, 2022.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: Vision-language-action flow model for general robot control. *arXiv*, 2024. https://www.physicalintelligence.company/download/pi0.pdf.

Vladimir I Bogachev, Nicolai V Krylov, Michael Röckner, and Stanislav V Shaposhnikov. *Fokker–Planck–Kolmogorov Equations*, volume 207. American Mathematical Society, 2022.

Avishek Joey Bose, Tara Akhound-Sadegh, Guillaume Huguet, Kilian Fatras, Jarrid Rector-Brooks, Cheng-Hao Liu, Andrei Cristian Nica, Maksym Korablyov, Michael Bronstein, and Alexander Tong. Se(3)-stochastic flow matching for protein backbone generation. *arXiv preprint arXiv:2310.02391*, 2023.

Arwen Bradley and Preetum Nakkiran. Classifier-free guidance is a predictor-corrector. *arXiv preprint arXiv:2408.09000*, 2024.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 35:28266–28279, 2022.

Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. *arXiv preprint arXiv:2402.04997*, 2024.

Ricky T. Q. Chen and Yaron Lipman. Flow matching on general geometries. In *The Twelfth International Conference on Learning Representations*, 2024.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.

Muthu Chidambaram, Khashayar Gatmiry, Sitan Chen, Holden Lee, and Jianfeng Lu. What does guidance do? a fine-grained analysis in a simple setting. *arXiv preprint arXiv:2409.13074*, 2024.

Earl A Coddington, Norman Levinson, and T Teichmann. Theory of ordinary differential equations, 1956.

Philippe Courrege. Sur la forme intégro-différentielle des opérateurs de c_k^∞ dans c satisfaisant au principe du maximum. *Séminaire Brelot-Choquet-Deny. Théorie du Potentiel*, 10(1):1–38, 1965.

Mark HA Davis. Piecewise-deterministic markov processes: A general class of non-diffusion stochastic models. *Journal of the Royal Statistical Society: Series B (Methodological)*, 46(3):353–376, 1984.

Valentin De Bortoli, Emile Mathieu, Michael Hutchinson, James Thornton, Yee Whye Teh, and Arnaud Doucet. Riemannian score-based generative modelling. *Advances in Neural Information Processing Systems*, 35:2406–2422, 2022.

Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. In *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794. Curran Associates, Inc., 2021.

Sander Dieleman. Guidance: a cheat code for diffusion models, 2022. https://benanne.github.io/2022/05/26/guidance.html.

Ronald J DiPerna and Pierre-Louis Lions. Ordinary differential equations, transport theory and sobolev spaces. *Inventiones mathematicae*, 98(3):511–547, 1989.

Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.

Stewart N Ethier and Thomas G Kurtz. *Markov processes: characterization and convergence*. John Wiley & Sons, 2009.

William Feller. On second order differential operators. *Annals of Mathematics*, 61(1):90–105, 1955.

Alessio Figalli. Existence and uniqueness of martingale solutions for sdes with rough or degenerate coefficients. *Journal of Functional Analysis*, 254(1):109–153, 2008.

Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. *arXiv preprint arXiv:2407.15595*, 2024.

Izrail Moiseevitch Gelfand, Richard A Silverman, et al. *Calculus of variations*. Courier Corporation, 2000.

Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. *arXiv preprint arXiv:1810.01367*, 2018.

Yingqing Guo, Hui Yuan, Yukang Yang, Minshuo Chen, and Mengdi Wang. Gradient guidance for diffusion models: An optimization perspective. *arXiv preprint arXiv:2404.14743*, 2024.

W Keith Hastings. Monte carlo sampling methods using markov chains and their applications. 1970.

Eric Heitz, Laurent Belcour, and Thomas Chambon. Iterative α-(de) blending: A minimalist deterministic diffusion model. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–8, 2023.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

Peter Holderrieth, Marton Havasi, Jason Yim, Neta Shaul, Itai Gat, Tommi Jaakkola, Brian Karrer, Ricky Chen, and Yaron Lipman. Generator matching: Generative modeling with arbitrary markov processes. *Preprint*, 2024. http://arxiv.org/abs/2410.20587.

Chin-Wei Huang, Milad Aghajohari, Joey Bose, Prakash Panangaden, and Aaron C Courville. Riemannian diffusion models. *Advances in Neural Information Processing Systems*, 35:2750–2761, 2022a.

Chin-Wei Huang, Milad Aghajohari, Joey Bose, Prakash Panangaden, and Aaron C Courville. Riemannian diffusion models. In *Advances in Neural Information Processing Systems*, 2022b.

Guillaume Huguet, James Vuckovic, Kilian Fatras, Eric Thibodeau-Laufer, Pablo Lemos, Riashat Islam, Cheng-Hao Liu, Jarrid Rector-Brooks, Tara Akhound-Sadegh, Michael Bronstein, et al. Sequence-augmented se (3)-flow matching for conditional protein backbone generation. *arXiv preprint arXiv:2405.20313*, 2024.

Arieh Iserles. *A first course in the numerical analysis of differential equations*. Cambridge university press, 2009.

Jürgen Jost. *Riemannian geometry and geometric analysis*, volume 42005. Springer, 2008.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.

Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in Neural Information Processing Systems*, 35:26565–26577, 2022.

Diederik Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. *Advances in neural information processing systems*, 34:21696–21707, 2021.

Thomas G Kurtz. Equivalence of stochastic equations and martingale problems. *Stochastic analysis 2010*, pages 113–130, 2011.

Matthew Le, Apoorv Vyas, Bowen Shi, Brian Karrer, Leda Sari, Rashel Moritz, Mary Williamson, Vimal Manohar, Yossi Adi, Jay Mahadeokar, et al. Voicebox: Text-guided multilingual universal speech generation at scale. *Advances in neural information processing systems*, 36, 2024.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

Guan-Horng Liu, Arash Vahdat, De-An Huang, Evangelos A. Theodorou, Weili Nie, and Anima Anandkumar. I2sb: image-to-image schrödinger bridge. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

Lynn Harold Loomis and Shlomo Sternberg. *Advanced calculus*. World Scientific, 1968.

Aaron Lou, Derek Lim, Isay Katsman, Leo Huang, Qingxuan Jiang, Ser-Nam Lim, and Christopher De Sa. Neural manifold ordinary differential equations. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 2020.

Aaron Lou, Minkai Xu, Adam Farris, and Stefano Ermon. Scaling riemannian diffusion models. *Advances in Neural Information Processing Systems*, 36:80291–80305, 2023.

Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. *arXiv preprint arXiv:2401.08740*, 2024.

Emile Mathieu and Maximilian Nickel. Riemannian continuous normalizing flows. In *Advances in Neural Information Processing Systems*, 2020.

Paul C Matthews. *Vector calculus*. Springer Science & Business Media, 2012.

Robert J McCann. A convexity principle for interacting gases. *Advances in mathematics*, 128(1):153–179, 1997.

Kirill Neklyudov, Rob Brekelmans, Daniel Severo, and Alireza Makhzani. Action matching: Learning stochastic dynamics from samples. In *International conference on machine learning*, pages 25858–25889. PMLR, 2023.

Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8162–8171. PMLR, 18–24 Jul 2021.

Alexander Quinn Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob Mcgrew, Ilya Sutskever, and Mark Chen. GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16784–16804. PMLR, 17–23 Jul 2022.

Bernt Øksendal. *Stochastic differential equations*. Springer, 2003.

Amnon Pazy. *Semigroups of linear operators and applications to partial differential equations*, volume 44. Springer Science & Business Media, 2012.

Stefano Peluchetti. Non-denoising forward-time diffusions. *arXiv preprint arXiv:2312.14589*, 2023.

Lawrence Perko. *Differential equations and dynamical systems*, volume 7. Springer Science & Business Media, 2013.

Gabriel Peyré, Marco Cuturi, et al. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019.

Ashwini Pokle, Matthew J Muckley, Ricky T. Q. Chen, and Brian Karrer. Training-free linear image inversion via flows. *arXiv preprint arXiv:2310.04432*, 2023.

Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, David Yan, Dhruv Choudhary, Dingkang Wang, Geet Sethi, Guan Pang, Haoyu Ma, Ishan Misra, Ji Hou, Jialiang Wang, Kiran Jagadeesh, Kunpeng Li, Luxin Zhang, Mannat Singh, Mary Williamson, Matt Le, Matthew Yu, Mitesh Kumar Singh, Peizhao Zhang, Peter Vajda, Quentin Duval, Rohit Girdhar, Roshan Sumbaly, Sai Saketh Rambhatla, Sam Tsai, Samaneh Azadi, Samyak Datta, Sanyuan Chen, Sean Bell, Sharadh Ramaswamy, Shelly Sheynin, Siddharth Bhattacharya, Simran Motwani, Tao Xu, Tianhe Li, Tingbo Hou, Wei-Ning Hsu, Xi Yin, Xiaoliang Dai, Yaniv Taigman, Yaqiao Luo, Yen-Cheng Liu, Yi-Chiao Wu, Yue Zhao, Yuval Kirstain, Zecheng He, Zijian He, Albert Pumarola, Ali Thabet, Artsiom Sanakoyeu, Arun Mallya, Baishan Guo, Boris Araya, Breena Kerr, Carleigh Wood, Ce Liu, Cen Peng, Dimitry Vengertsev, Edgar Schonfeld, Elliot Blanchard, Felix Juefei-Xu, Fraylie Nord, Jeff Liang, John Hoffman, Jonas Kohler, Kaolin Fire, Karthik Sivakumar, Lawrence Chen, Licheng Yu, Luya Gao, Markos Georgopoulos, Rashel Moritz, Sara K. Sampson, Shikai Li, Simone Parmeggiani, Steve Fine, Tara Fowler, Vladan Petrovic, and Yuming Du. Movie gen: A cast of media foundation models, 2024. https://arxiv.org/abs/2410.13720.

Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *International Conference on Machine Learning*, 2023.

Jan Prüss, Mathias Wilke, and Mathias Wilke. *Ordinary differential equations and dynamic systems*. Springer, 2010.

Gareth O Roberts and Richard L Tweedie. Exponential convergence of langevin distributions and their discrete approximations. *Bernoulli 2(4): 341-363 (December 1996)*, 1996.

Leonard CG Rogers and David Williams. *Diffusions, markov processes, and martingales: Volume 1, foundations*. Cambridge university press, 2000.

Noam Rozen, Aditya Grover, Maximilian Nickel, and Yaron Lipman. Moser flow: Divergence-based generative modeling on manifolds. *Advances in Neural Information Processing Systems*, 34:17669–17680, 2021.

Ludger Rüschendorf, Alexander Schnurr, and Viktor Wolf. Comparison of time-inhomogeneous markov processes. *Advances in Applied Probability*, 48(4):1015–1044, 2016.

Chitwan Saharia, William Chan, Huiwen Chang, Chris Lee, Jonathan Ho, Tim Salimans, David Fleet, and Mohammad Norouzi. Palette: Image-to-image diffusion models. In *ACM SIGGRAPH 2022 Conference Proceedings*, SIGGRAPH '22. Association for Computing Machinery, 2022.

Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022.

Simo Särkkä and Arno Solin. *Applied stochastic differential equations*, volume 10. Cambridge University Press, 2019.

Neta Shaul, Ricky T. Q. Chen, Maximilian Nickel, Matthew Le, and Yaron Lipman. On kinetic optimal probability paths for generative models. In *International Conference on Machine Learning*, pages 30883–30907. PMLR, 2023a.

Neta Shaul, Juan Perez, Ricky T. Q. Chen, Ali Thabet, Albert Pumarola, and Yaron Lipman. Bespoke solvers for generative flow models. *arXiv preprint arXiv:2310.19075*, 2023b.

Neta Shaul, Itai Gat, Marton Havasi, Daniel Severo, Anuroop Sriram, Peter Holderrieth, Brian Karrer, Yaron Lipman, and Ricky T. Q. Chen. Flow matching with general discrete paths: A kinetic-optimal perspective, 2024. https://arxiv.org/abs/2412.03487.

Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet. Diffusion schrödinger bridge matching. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pages 2256–2265. PMLR, 2015.

Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, pages 11895–11907, 2019.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

Alexander Tong, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Kilian Fatras, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

Cédric Villani. *Topics in optimal transportation*, volume 58. American Mathematical Soc., 2021.

Cédric Villani et al. *Optimal transport: old and new*, volume 338. Springer, 2009.

Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674, 2011.

Wilhelm von Waldenfels. Fast positive operatoren. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 4:159–174, 1965.

Apoorv Vyas, Bowen Shi, Matthew Le, Andros Tjandra, Yi-Chiao Wu, Baishan Guo, Jiemin Zhang, Xinyue Zhang, Robert Adkins, William Ngan, et al. Audiobox: Unified audio generation with natural language prompts. *arXiv preprint arXiv:2312.15821*, 2023.

Jason Yim, Andrew Campbell, Andrew YK Foong, Michael Gastegger, José Jiménez-Luna, Sarah Lewis, Victor Garcia Satorras, Bastiaan S Veeling, Regina Barzilay, Tommi Jaakkola, et al. Fast protein backbone generation with se (3) flow matching. *arXiv preprint arXiv:2310.05297*, 2023.

Qinqing Zheng, Matt Le, Neta Shaul, Yaron Lipman, Aditya Grover, and Ricky T. Q. Chen. Guided flows for generative modeling and decision making, 2023. https://arxiv.org/abs/2311.13443.

A 补充证明

A.1 离散质量守恒

引理 1 (满足速率条件的柯尔莫哥洛夫方程的概率质量函数解). *考虑柯尔莫哥洛夫方程 (6.8) 的解 $f_t(x)$，初始条件为 $f_0(x) = p(x)$，其中 p 是概率质量函数，$u_t(y, x)$ 关于时间 t 属于 $C([0, 1])$，并满足速率条件 (6.4). 则对所有 $t \in [0, 1]$，$f_t(x)$ 都是概率质量函数 (PMF).*

证明. 令 $f_t(x)$，$t \in [0, 1]$，为柯尔莫哥洛夫方程的解，定理 12 保证其存在且唯一. $f_t(x)$ 是概率质量函数，当且仅当它满足

$$f_t(x) \geq 0, \text{ 且} \sum_x f_t(x) = 1. \tag{A.1}$$

对柯尔莫哥洛夫方程两边求和，可知解满足如下关系，从而证明后一个条件成立：

$$\frac{\mathrm{d}}{\mathrm{d}t} \sum_x f_t(x) = \sum_x \sum_z u_t(x, z) p_t(z) = 0$$

其中第二个等号来自速率条件中的 $\sum_y u_t(y, x) = 0$. 由于 $\sum_x f_0(x) = \sum_x p(x) = 1$，对所有 $t \in [0, 1]$ 都有 $\sum_x f_t(x) \equiv 1$.

为证明对所有 $x \in \mathcal{S}$ 都有 $f_t(x) \geq 0$，将使用关于动力系统凸不变集的一个结果. 具体而言，Prüss et al. (2010) 中的定理 7.3.4 断言：只要 $f_0 = p$ 满足这一条件 (确实满足)，并且每当 $w(z)$ 位于约束边界，即 w 是概率质量函数且对某个 $z \in \mathcal{S}$ 有 $w(z) = 0$ 时，与约束外法向的内积非正，即 $\sum_{x,y} u_t(y, x) w(x) \delta(y, z) \geq 0$，就能推出对所有 $t \in [0, 1]$ 和 $x \in \mathcal{S}$，解满足 $f_t(x) \geq 0$. 下面检验这一条件：

$$\sum_{x,y} u_t(y, x) w(x) \delta(y, z) = \sum_x u_t(z, x) w(x) = \sum_{x \neq z} u_t(z, x) w(x) \geq 0,$$

其中，第二个等号使用了 $w(z) = 0$，最后一个不等式使用了速率条件 (6.4) 中当 $z \neq x$ 时 $u_t(z, x) \geq 0$，以及对所有 y 都有 $w(y) \geq 0$.

□

定理 13 (离散质量守恒). *令 $u_t(y, x)$ 关于时间 t 属于 $C([0, 1))$，$p_t(x)$ 是关于 t 属于 $C^1([0, 1))$ 的概率质量函数. 则以下条件等价：*

1. *p_t, u_t 在 $t \in [0, 1)$ 上满足柯尔莫哥洛夫方程 (6.8)，且 u_t 满足速率条件 (6.4).*
2. *u_t 在 6.5 的意义下，对 $t \in [0, 1)$ 生成 p_t.*

证明. 首先假设条件 2 成立. 此时，概率转移核 $p_{t+h|t}(y|x)$ 满足 (6.3)：

$$p_{t+h|t}(y|x) = \delta(y, x) + h u_t(y, x) + o(h). \tag{A.2}$$

利用全概率公式表示边缘分布 $p_t(y)$，得到

$$p_{t+h}(y) = \sum_x p_{t+h|t}(y|x)\, p_t(x). \tag{A.3}$$

将 (A.2) 代入 (A.3) 并整理，得到

$$\frac{p_{t+h}(y)-p_t(y)}{h}=\sum_x u_t(y,x)p_t(x)+o(1),$$

其中，按照 $o(h)$ 的定义，当 $h\to 0$ 时，$o(1)=o(h)/h\to 0$. 取极限 $h\to 0$，得到 (p_t,u_t) 满足柯尔莫哥洛夫方程 (6.8). 接下来证明 u_t 满足速率条件 (6.4). 若对某些 $y\neq x$ 有 $u_t(y,x)<0$，由 (A.2)，对充分小的 $h>0$ 有 $p_{t+h|t}(y|x)<0$，这与 $p_{t+h|t}$ 为概率核矛盾. 若 $\sum_y u_t(y,x)=c\neq 0$，由 A.2 得 $1=\sum_x p_{t+h|t}(y|x)=1+hc+o(h)$，当 $h>0$ 充分小时产生矛盾.

反过来，现假设条件 1 成立. 即 (u_t,p_t) 满足初始条件为 $p_0=p$ 的柯尔莫哥洛夫方程 (6.8). 由定理 12，令 $p_{s|t}(y|x)$ 为柯尔莫哥洛夫方程的唯一解：

$$\frac{\mathrm{d}}{\mathrm{d}s}p_{s|t}(y|x)=\sum_z u_t(y,z)p_{s|t}(z|x), \tag{A.4}$$

初始条件为 $p_{t|t}(y,x)=\delta(y,x)$，其中 $0\le t\le s<1$，并固定 t 和 y. 由 引理 1，$p_{s|t}(\cdot|x)$ 是概率质量函数. $\sum_x p_{s|t}(y|x)p(x)$ 也满足柯尔莫哥洛夫方程，因为

$$\frac{\mathrm{d}}{\mathrm{d}t}\sum_x p_{s|t}(y|x)p(x)=\sum_x\left[\sum_z u_t(y,z)p_{s|t}(z|x)\right]p(x)=\sum_z u_t(y,z)\left[\sum_x p_{s|t}(z|x)p(x)\right], \tag{A.5}$$

此时初始条件为 $\sum_x p_{t|t}(y|x)p(x)=p(y)$. 由柯尔莫哥洛夫方程解的唯一性（定理 12），得到所需等式 $\sum_x p_{s|t}(y|x)p(x)=p_s(y)$. 最后，对于 $0\le t\le r\le s<1$，转移核的半群性质 $\sum_z p_{s|r}(y|z)p_{r|t}(z|x)=p_{s|t}(y|x)$，可以以 $p_{r|t}$ 为时刻 r 的初始条件，重复 A.5 中的论证来证明. 由此找到了生成 p_t 的转移核 $p_{t+h|t}$，完成证明. □

A.2 流形边缘化技巧

定理 *10* (流形边缘化技巧). 在假设 2 下，如果 $u_t(x|x_1)$ 条件可积并生成条件概率路径 $p_t(\cdot|x_1)$，那么边缘速度场 $u_t(\cdot)$ 生成边缘概率路径 $p_t(\cdot)$.

证明. 为证明 $u_t(\cdot)$ 生成 $p_t(\cdot)$，再次验证它们满足质量守恒定理中的条件. 首先检验 $u_t(x)$ 与 $p_t(x)$ 满足连续性方程 (5.1)：

$$\frac{\mathrm{d}}{\mathrm{d}t}p_t(x)\overset{(i)}{=}\int_{\mathcal{M}}\frac{\mathrm{d}}{\mathrm{d}t}p_{t|1}(x|x_1)q(x_1)\mathrm{dvol}_{x_1} \tag{A.6}$$

$$\overset{(ii)}{=}-\int_{\mathcal{M}}\mathrm{div}_g\left[u_t(x|x_1)p_{t|1}(x|x_1)\right]q(x_1)\mathrm{dvol}_{x_1} \tag{A.7}$$

$$\overset{(i)}{=}-\mathrm{div}_g\int_{\mathcal{M}}u_t(x|x_1)p_{t|1}(x|x_1)q(x_1)\mathrm{dvol}_{x_1} \tag{A.8}$$

$$\overset{(iii)}{=}-\mathrm{div}_g\left[u_t(x)p_t(x)\right], \tag{A.9}$$

其中，在 (i) 中交换了微分（$\frac{\mathrm{d}}{\mathrm{d}t}$ 和 div_g）与积分的次序，其依据是莱布尼茨法则，以及 $p_{t|1}(x|x_1)$ 和 $u_t(x|x_1)$ 关于 t,x 属于 C^1，且 q 具有有界支撑或 $\mathcal{M}$ 紧. 在 (ii) 中，使用了 $u_t(\cdot|x_1)$ 生成 $p_{t|1}(\cdot|x_1)$ 的事实和 定理 9. 在 (iii) 中，乘以再除以 $p_t(x)$（按假设它严格为正），并使用 (5.10) 中 u_t 的公式. 最后，采用与 定理 3 证明中相同的论证，可知 u_t 可积且局部 Lipschitz 连续. □

A.3 KFE 的正则性假设

注意，假设 5 在相对较弱的假设下成立，数学文献中有许多结果，在不同情形下证明 KFE 关于 p_t 的解的唯一性. 然而，据我们所知，目前没有已知结果给出一般状态空间和马尔可夫过程的正则性假设，因此这里直接将其表述为假设. 对于机器学习实践者，该假设在任何相关状态空间上都成立. 为说明这一点，下面列出数学文献中的丰富资料，它们证明唯一性，并给出特定空间和马尔可夫过程类别的正则性假设：

1. $\mathbb{R}^d$ 与流形上的流：(Villani et al., 2009, 质量守恒公式，第 15 页)、(DiPerna and Lions, 1989)、(Ambrosio, 2004).
2. $\mathbb{R}^d$ 与流形上的扩散：(Villani et al., 2009, 扩散定理，第 16 页).
3. $\mathbb{R}^d$ 上的一般 Itô 随机微分方程：(Figalli, 2008, 定理 1.3 和 1.4)、(Kurtz, 2011, 推论 1.3)、(Bogachev et al., 2022).
4. 离散状态空间：此时 KFE 为线性常微分方程，在系数连续的假设下具有唯一解（见 定理 13）.

术语对照表

中文术语	英文术语
流匹配	Flow Matching
条件流匹配	Conditional Flow Matching
黎曼流匹配	Riemannian Flow Matching
离散流匹配	Discrete Flow Matching
生成元匹配	Generator Matching
流	flow
连续归一化流	Continuous Normalizing Flow
速度场；向量场	velocity field; vector field
概率路径	probability path
源分布；目标分布	source distribution; target distribution
条件概率密度	conditional probability density
边缘分布	marginal distribution
边缘化技巧	Marginalization Trick
推前	push-forward
微分同胚	diffeomorphism
连续性方程	Continuity Equation
瞬时变量替换公式	Instantaneous Change of Variables
最优传输	Optimal Transport
耦合	coupling
仿射条件流	affine conditional flow
Bregman 散度	Bregman divergence
无分类器引导	classifier-free guidance
得分函数	score function
预度量	premetric
连续时间马尔可夫链	Continuous Time Markov Chain
连续时间马尔可夫过程	Continuous Time Markov Process
生成元	generator
跳跃过程	jump process
漂移；扩散系数	drift; diffusion coefficient
柯尔莫哥洛夫前向方程	Kolmogorov Forward Equation
柯尔莫哥洛夫后向方程	Kolmogorov Backward Equation
半群	semigroup
去噪器	denoiser
塔式性质	tower property